# Vantage: Availability-Graded Broadcast for Signature-Free BFT

NIKITA POLYANSKII, IOTA Foundation, Germany

Digital signatures make blocks and votes transferable evidence: one party can prove to another what a third party said. Authenticated channels convince only the direct receiver. The high-throughput signature-free protocols we survey therefore complete an availability vote or a broadcast instance for a data block before any proposer may order it.

We present Vantage, a partially synchronous Byzantine fault-tolerant (BFT) protocol for $n \geq 3f+1$ parties that uses only authenticated channels and collision-resistant hashing. Parties publish blocks on hash-linked author lanes; each view's proposer pairs a quorum-available *core manifest* of lane frontiers with an optimistic *tip manifest* of freshly received blocks. A new primitive, *Availability-Graded Broadcast* (AGB), makes the core irrevocable on a quorum of first-hand responses while grading, rather than blocking on, the tip manifest. Unresolved tips are sealed later by resolutions that every correct party checks against its own recorded responses; when all parties are correct, all $n$ exact grade-1 ECHOs fast-seal a qualifying proposal with no resolution metadata (a data-only proposal) within two message delays of its send. A crash-only silent view is skipped by a quorum of skip votes without waiting for the control log; partial and mixed failures retain the control-log resolution path. AGB allows a published data block to become proposal-eligible after one message delay, matching signed optimistic-tip designs. In the all-correct data-only responsive suffix, the block is sequenced within $4\delta$ of publication, and within $3\delta$ when publication is aligned with the next proposal, where $\delta$ is the actual message delay after the Global Stabilization Time (GST). We prove safety under asynchrony, while liveness holds after GST. We compare Vantage with the nearest signature-based and signature-free protocols: it has the lowest median latency at every load satisfying our 95% acceptance criterion, and its peak accepted throughput is within 3% of the best signature-free configuration. In particular, with 100 parties on an emulated ten-region wide-area network (WAN), Vantage sequences 239k 512-byte transactions per second with median end-to-end latency below 500 ms.

CCS Concepts: • **Security and privacy** → **Distributed systems security**; • **Theory of computation** → *Distributed algorithms.*

Additional Key Words and Phrases: Byzantine fault tolerance, atomic broadcast, availability-graded broadcast, signature-free protocols, partial synchrony

## 1 INTRODUCTION

State-machine replication (SMR) lets a group of replicas maintain one transaction log even when some replicas are faulty. In a traditional design, one leader collects the transactions and sends the next block. Its outgoing link can limit throughput. Modern high-throughput systems instead let every replica publish transaction blocks in parallel. Consensus then orders short references to those blocks [6, 16, 20, 33].

*When may a block be referenced?* There are two basic choices. With *certified admission*, a proposer waits until a quorum has confirmed that it holds the block. This is robust, but the confirmation round must finish before ordering can begin. Because every reference is already confirmed, voters can treat the proposal as *indivisible*: one value, accepted or not as a whole. With *uncertified admission*, the next proposer references the block as soon as it receives it from the author. This removes that delay when all messages arrive. A voter that missed one block in an indivisible proposal can only wait for it, fetch it, or reject the proposal and every other block in it.

Digital signatures make fetched bytes attributable: after receiving them from any party, the voter can verify who signed them. Authenticated channels do not provide transferable attribution. They tell a receiver who sent a message, but the receiver cannot pass that proof to anybody else. A hash proves that fetched bytes match a reference; it does not prove who published them. Data fetched during repair can therefore restore missing bytes, but does not establish

Author's address: Nikita Polyanskii, IOTA Foundation, Berlin, Germany, nikitapolyansky@gmail.com.

authorship. Avoiding signatures removes the protocol's dependence on a public-key infrastructure (PKI) and signature processing, but also removes this transferable proof.

Without transferable attribution, waiting can stall the view, repair supplies data but not authorship evidence, and rejection discards otherwise available blocks. Vantage instead divides each proposal into a part supported by first-hand availability evidence and an optional part admitted after direct publication. The high-throughput signature-free systems we survey use certified admission instead, certifying each orderable block or waiting for availability votes before proposing it [34, 42].

*Our approach.* Vantage uses quorum acknowledgments to distinguish the two parts, because a voter can count them without transferable proof. Each Vantage party publishes its blocks as a hash-linked chain, and a correct proposer sends two manifests:

- the *quorum-available core manifest* lists lane frontiers for which the proposer has already received acknowledgments from $n - f$ of the $n$ parties, of which at most $f$ may be faulty; and
- the *optimistic tip manifest* lists newer lane frontiers received directly from their authors.

Completion makes the core irrevocable. The tip is included only when the responses support it; otherwise the core can be retained without the tip.

On a proposal that carries no resolution metadata, a voter holding both manifests supports the full proposal. A voter that can check the core but still lacks a tip block at its echo deadline supports the core alone, without waiting for the missing block or rejecting the other blocks. A voter that cannot check the core refuses. Every receiver counts only responses received directly from their senders.

We call this standalone two-level primitive *Availability-Graded Broadcast* (AGB), and use *Direct AGB* when distinguishing it from the Vantage-level shortcuts and resolver. A READY quorum completes the proposal and makes its core irrevocable. A homogeneous READY quorum directly seals either the full proposal or the core; mixed grades leave only the tip open. A separate signature-free control log resolves open or silent views. A grounded post-ready skip-vote shortcut bypasses that log for a clean crash-only silent view; the Direct AGB READY path and the control-log resolution path proceed independently.

*Latency.* Safety does not depend on timing; liveness holds after GST. Let $\delta$ be the actual message delay after GST. When all parties are correct, a new block can enter a proposal one delay after publication. The proposal then seals in two more delays when all $n$ responses support it, or three through the ordinary Direct AGB READY path. The two-delay seal is optimal: with $f \geq 1$, no broadcast reaches a decision one message delay after its send, even in all-correct executions [2].

Assuming earlier views have already been output, the new block is sequenced with the full outcome in $3\delta$ when the next proposal is sent as soon as the block is eligible, and in at most $4\delta$ for any arrival time. Using the quorum-available core adds one delay to each figure. These bounds hold at any $n \geq 3f+1$, including the optimal resilience $n = 3f+1$ for partial synchrony.

A faulty author can make one tip entry verifiable only at the correct proposer by publishing its lane-tip block there and withholding matching acknowledgments from every other correct party. At $n = 3f + 1$, the proposer is then the only correct grade-1 ECHO author; even if all $f$ Byzantine parties also send grade 1, at most $1 + f < 2f + 1$ grade-1 ECHOs exist. Thus neither the all-$n$ fast-seal shortcut nor a grade-1 READY quorum can form. ECHO and READY quorums still count both grades, so completion keeps its ordinary bound. If grade 0 later reaches a quorum, a homogeneous

refinement seals the core; otherwise the direct result may remain open-tip and ordered output waits for the control log. A correct party that emits READY-mix omits the author of each non-quorum tip entry from its own later tip manifests until the corresponding prefix becomes quorum-available or enters a containing core. This prevents correct proposers from repeatedly creating open views from the same unresolved lane, but a Byzantine proposer need not obey the discipline; the general resolver queue remains unbounded (Theorem B.8 and section 8).

For a clean crash-only silent proposer, the post-ready skip shortcut removes that queueing source: after an ECHO-SKIP quorum and its own NO-READY, each correct party casts one grounded skip vote, and a quorum seals the view one actual delay later. This is not a bound for partial dissemination or Byzantine recovery schedules, which continue through the control log.

*Scope of the claim.* To our knowledge, Vantage is the first high-throughput protocol in our survey to admit another author's block after one message, without signatures and without first completing a separate availability-vote or broadcast protocol for that block. This is a scoped data-path claim, not a claim that Vantage has better faulty-view or control-log resolution latency than signed protocols. Section 3 gives the exact comparison conditions.

Our contributions are:

- **Availability-Graded Broadcast.** We define a non-total broadcast primitive with nested outcomes: completion fixes one core, while homogeneous first-hand grades select either that core with its tip or the core alone. We prove timing-independent value and grade agreement, publication and availability witnesses, and conditional termination for a correct proposer. Its graded Complete–Adopt relation links completion to correct adopters, while a quorum of no-adopt responses excludes completion.
- **Signature-free SMR at $n \geq 3f + 1$.** We instantiate AGB with authenticated author lanes and non-transferable acknowledgment quorums, then compose its direct results with a locally checked resolver backed by a validated Bracha/Simple-IT control log. We prove one common terminal outcome per view, prefix-comparable output with a common limit, author-backed retrievability, and eventual inclusion of every valid block from a correct author.
- **One-hop admission and latency.** A directly published cross-author lane tip is proposal-eligible after $\delta$. In the all-correct data-only responsive suffix, where proposals contain no resolution metadata, earlier views are output, and consecutive proposal opportunities are at most one actual delay apart, all $n$ exact grade-1 ECHOs fast-seal a prefix-extending proposal within $2\delta$ of its send. This gives $3\delta$ aligned publication-to-sequencing latency. At $n = 3f + 1$, this matches the lowest aligned endpoint in our normalized survey, including signed paths. One AGB view uses $O(n^2)$ messages and $O(n^3\lambda)$ control bits with the proved encoding, where $\lambda$ is the hash security parameter; the evaluated wire encoding lowers the aligned common case to $O(n^2\lambda + n^3)$ control bits per view at an unchanged worst case (Appendix F.3). Publication and repair are accounted for separately.
- **Implementation and evaluation.** One Rust binary runs VANTAGE, two Autobahn variants, and two Simple-IT variants. With 100 parties on an emulated ten-region WAN, VANTAGE has the lowest median latency at every accepted load among all compared configurations, and its peak throughput is within 3% of the best signature-free configuration (Section 7).

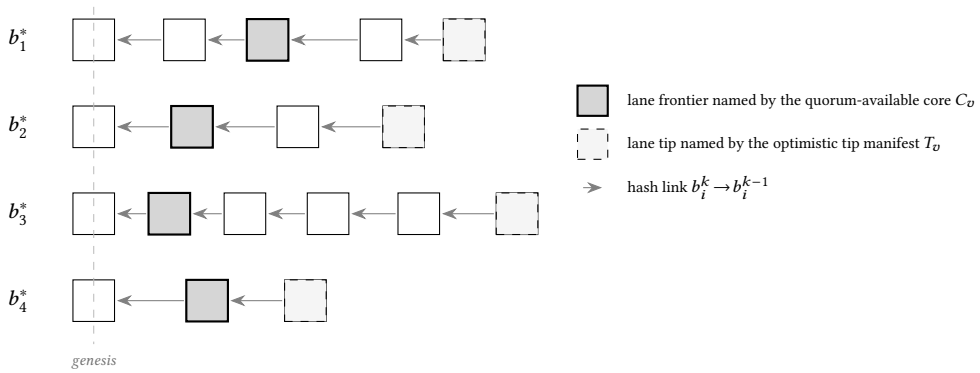


Fig. 1. Each party publishes an author lane $b_i^1, b_i^2, \ldots$ from a shared genesis, advancing asynchronously to a ragged frontier. The snapshot is from the point of view of Proposer$(v)$. Proposer$(v)$ proposes the view proposal $B_v = (C_v, T_v, M_v)$, naming data blocks by author-indexed references: the quorum-available core $C_v$ (gray) and the optimistic tip manifest $T_v$ (dashed).

## 2 OVERVIEW

This section describes one all-correct data-only view of Vantage and then the unresolved cases. Section 3 compares the nearest protocols, Section 4 makes the model precise, Section 5 defines and proves the broadcast primitive, and Section 6 gives resolution, output, and the background control log.

*Lifecycle vocabulary.* We distinguish five lifecycle actions. *Complete* fixes a view's proposal and makes its core irrevocable. A *direct-seal* selects the full or core outcome from a homogeneous READY quorum; the Vantage-only *fast-seal* selects the full outcome from all $n$ exact grade-1 ECHOs; and the Vantage-only *skip-seal* selects skip from a quorum of grounded post-ready skip votes. The terminal *seal* records, once per view, the first compatible result from these mechanisms or the resolver (Section 6). *Sequence* assigns blocks to the canonical cross-view order; after earlier views are sealed, a completed core may be sequenced before its own tip is sealed. *Output* is local delivery after retrieval. We reserve *commit* for the imported Simple-IT control log of Section 6.1, except inside the formal outcome names commit-full and commit-core.

*Data path: author lanes.* Every party continuously publishes its data blocks on its own *author lane*, a hash chain sent block by block to all parties (Figure 1). A party that has received a valid lane prefix directly from its author broadcasts a short acknowledgment for it. Direct receipt establishes provenance locally, and $f$+1 matching acknowledgments convince a counting party that at least one correct party received the prefix directly from its author and retains it. No block waits for a broadcast or certification round before it may be proposed.

*All-correct data-only case: one view.* Views have round-robin proposers. The proposer of view $v$ broadcasts one *view proposal* $B_v = (C_v, T_v, M_v)$ with two *manifests*, each naming at most one lane reference per author: the *core manifest* $C_v$ lists lane frontiers the proposer received directly and counted $n-f$ acknowledgments for, and the optimistic *tip manifest* $T_v$ lists newer lane tips received directly from their authors, each strictly extending its author's core entry (or genesis when the author has none). The metadata field $M_v$ is empty in this case. Every party answers in Bracha's ECHO/READY pattern [9] (Figure 2):

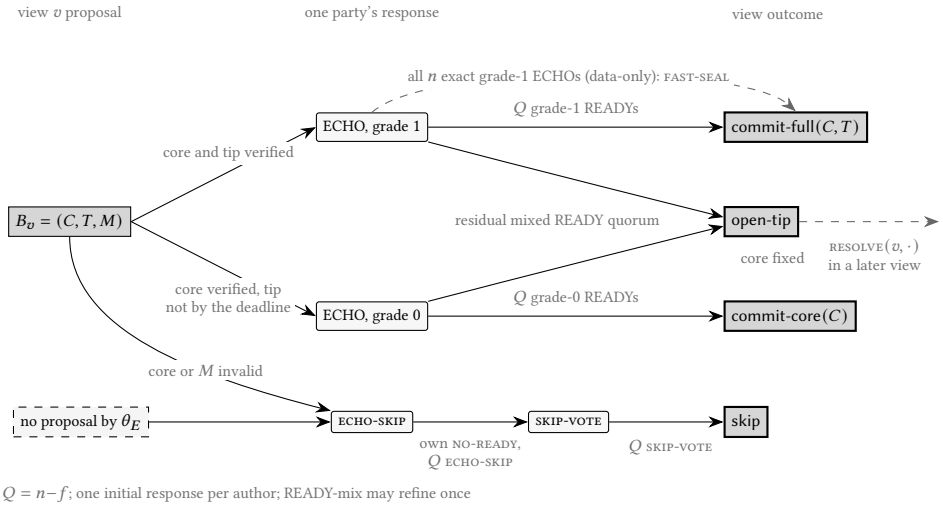


Fig. 2. Vantage's response and outcome flow around Direct AGB for one view $v$. Each party emits one Direct AGB echo-stage response (left); the composition of the counted quorum selects the view outcome (right). Every READY quorum completes the view and fixes the core; homogeneous grades add a Direct AGB result, while a residual mixed quorum leaves the tip open for a later-view resolution (Figure 4). The shortcut branches are Vantage shortcuts: grounded skip votes seal a crash-only silent view, and all $n$ exact grade-1 ECHOs fast-seal a data-only proposal as commit-full one message delay before READYs.

- an *ECHO* of $B_v$, carrying grade 1 if the party verified both core and tip, and grade 0 if it verified the core but could not verify the tip by its local deadline — missing, forked, or not extending the core, a failed tip lowers the grade instead of blocking the view;
- a *READY* for $B_v$ once $n-f$ ECHOs name $B_v$, carrying grade 1 or 0 when it counted $n-f$ ECHOs of that single grade for $B_v$, and the mixed grade otherwise. A mixed READY is sent immediately; until the refinement window closes, it may be followed by one same-$B_v$ homogeneous refinement if later ECHOs make one grade reach $n-f$.

Counting $n-f$ READYs for $B_v$ — grades ignored — *completes* the view: quorum intersection makes $B_v$ the only completable proposal for $v$, and its core becomes an irrevocable prefix of the view's output. A homogeneous READY quorum also *seals* the view directly: grade 1 selects the full outcome (commit-full), while grade 0 selects the core outcome (commit-core). For a data-only proposal, the all-$n$ fast-seal shortcut acts one message delay earlier: all $n$ exact grade-1 ECHOs fast-seal the full outcome. In the all-correct responsive suffix, this occurs two delays after the send of a proposal whose manifests extend the already-output lane prefixes (Figure 3). Views are pipelined: in the all-correct responsive suffix the least not-yet-proposed view follows within one delay of the latest send, and entering a later view never waits for an older view's response windows.

*Recovery: open and silent views.* Direct AGB does not immediately seal two cases. READY grades that remain mixed after the permitted refinements leave a completed view *open*: its core is fixed, but its tip is neither included nor discarded. A faulty proposer can leave a view *silent*: parties time out and emit the refusal responses ECHO-SKIP and NO-READY. A clean crash-only silent view can use the grounded skip-vote shortcut: after its own NO-READY, a party that counted an ECHO-SKIP quorum and has not endorsed a non-skip recovery broadcasts SKIP-VOTE; a quorum of those votes seals skip without a resolution-bearing proposal or control-log application (Figure 2). The grounded vote and an ECHO for a

non-skip resolution-bearing proposal are persistently exclusive at each correct party. In every other unresolved case, a later proposal carries a *resolution* entry in its metadata field, asking to seal a target view as commit-full, commit-core, or skip; a bounded vector may batch only manifest-free skip entries. The resolution needs no transferable certificate: a party echoes the resolution-bearing proposal only if the entry is consistent with its *own* final echo- and ready-stage responses for the target view, so that proposal's ECHO quorum intersects every target-view READY quorum in a correct party whose recorded response contradicts any conflicting outcome. A background signature-free log orders *anchor hashes*, which name completed resolution-bearing proposals, and every party applies the first logged resolution per target (Figure 4). The all-$n$ fast-seal shortcut is guarded the same way: sending a grade-1 ECHO for a data-only proposal installs an *optimistic lock* under which the party refuses every conflicting resolution until $f$+1 first-hand echo-stage responses prove that an all-$n$ exact grade-1 ECHO set can no longer form.

## 3 CLOSEST COMPARISONS

We compare the time from a correct nonleader publishing a block to a different correct party proposing the object that orders it. We call that proposal or protocol object the *carrier*; in Vantage, it names a lane prefix that contains the block. We assume every earlier ordering position has already been resolved and output, and that the next proposal extends this prefix without waiting for an older response window. Reliable broadcast (RBC), optimistic RBC (Opt-RBC), and best-effort broadcast (BEB) are abbreviated below. We also use proof of availability (PoA), Byzantine Broadcast with Complete-Adopt (BBCA), and directed acyclic graph (DAG). We write $A$ for publication-to-carrier latency and $L$ for publication-to-sequencing latency.

*Comparison assumptions.* Every row uses the same favorable setting: all parties are correct after GST, there is no queueing, and earlier output is complete. The Vantage row also assumes the data-only responsive suffix of Corollary B.9. Publication is aligned with the next carrier opportunity, so $A$ and $L$ exclude an extra wait for the protocol's cadence. "Signature-free" means no digital signatures or transferable signature certificates; every row still assumes authenticated channels. Here $\delta$ is the actual post-GST message-delay bound and $\Delta$ is its known configured upper bound.

Starred values are derived zero-fault fast paths that require matching responses from all $n$ parties; one non-voter disables them. FinWhale defines $n = 3f + 2p - 1$ and uses $n - p$ references for its fast decision. Its displayed $p = 1$ value is not starred: it runs at $n = 3f$+1 and survives one non-voter [23].

The last column is a per-view or per-round advancement bound. In the view-based rows, once every correct party has entered view $v$ or higher, it upper-bounds the additional time until every correct party has entered a view strictly higher than $v$, following the definition of [42]. The FinWhale row gives the corresponding maximum time a correct party can remain in one round under the adopted Push pacemaker. Thus one failed view or round can delay progression by at most the listed amount after the bound applies; each view or round in a run of failures incurs its own delay. This is not a bound on resolving the failed value or on the time to process a recovery queue. Vantage's value follows Lemma B.5; the Simple-IT, Sailfish++, and Autobahn values come from [42], whose Sailfish++ bound applies after a correct leader. The FinWhale value is the worst-case round duration proved for the Push pacemaker with leader timeout $2\Delta$ [28], which FinWhale adopts; Mysticeti alone states no such bound. The daggered BBCA-Chain value is derived from its timer and synchronization rules rather than stated in that form [26]. Its general bound is $\Delta_\Gamma + \delta$: by $\Delta_\Gamma$ after synchronized entry, every lagging correct party has completed or adopted, or has probed and sent a new-view block; one actual delay then delivers either a complete/adopt certificate or $2f + 1$ correct noadopt blocks everywhere. The paper requires $\Delta_\Gamma \geq \Delta_{\text{sync}} + 3\Delta$ and proves $\Delta_{\text{sync}} = \Delta$, so the minimum admissible timer gives the reported $4\Delta + \delta$. In

Table 1. Aligned cross-author data latency in the common experiment above. $A$ is publication-to-carrier latency and $L$ is publication-to-sequencing latency. The final column is the maximum per-view/round advancement delay defined above, not failed-value recovery or sequencing latency. Lower is better; the comparison assumptions above apply.

publish $\xrightarrow{A}$ first legal carrier $\longrightarrow$ sequenced at $L$

| Protocol / path | Signature-free | Required before carrier | Admission $A$ | Aligned total $L$ | Max one-view/ round advance |
|---|---|---|---|---|---|
| Mempool-Simple-IT (Opt-RBC) [42] | Yes | Availability-vote quorum | $2\delta$ | $5\delta$ | $8\Delta+4\delta$ |
| Sailfish++ (Opt-RBC) [34, 42] | Yes | Opt-RBC for every vertex | $2\delta$ | $5\delta$ ordinary<br>$7\delta$ late | $8\Delta+5\delta$ |
| **Vantage (optimistic lane tip)** | **Yes** | **Directly published lane tip; no PoA** | $\boldsymbol{\delta}$ | $\mathbf{3\delta^*}$ all-to-all fast<br>$\mathbf{4\delta}$ READY path | $\mathbf{4\Delta+\delta}$ |
| FinWhale (Mysticeti + fast path) [6, 23] | No | Direct receipt of signed DAG block | $\delta$ | $3\delta$ fast<br>$4\delta$ ordinary | $2\Delta+2\delta$ |
| Autobahn (optimistic lane tip) [20] | No | Current lane tip; parent has PoA | $\delta$ | $3\delta^*$ all-to-all fast<br>$4\delta^*$ linear fast | $10\Delta+\delta$ |
| BBCA-Chain [26] | No | One-trip BEB block | $\delta$ | $4\delta$ | $4\Delta+\delta^\dagger$ |

particular, Vantage's control-log resolution may take longer than view advancement and has no fixed bound (Section 8). The table compares latency under these assumptions, not overall dominance or fault tolerance.

The table selects five representative baselines. The quorum-available core and optimistic tip are two manifests in the same Vantage proposal. The highlighted row isolates lane-tip admission, which constitutes the one-hop result. Its $3\delta$ all-to-all fast value assumes that the carrier is sent immediately when the tip becomes eligible; the Direct AGB READY path gives $4\delta$. The Autobahn cell likewise gives its all-to-all fast path first, followed by its main linear fast path. The derivations and protocol-by-protocol qualifications appear in Appendix A. Without the alignment assumption, Corollary B.9 bounds the Vantage tip-manifest latency by $4\delta$ in the worst phase, via the all-$n$ fast-seal shortcut.

The signature-free baselines require two delays before a carrier: an availability-vote quorum in Mempool-Simple-IT and Opt-RBC for each Sailfish++ vertex. Vantage instead admits a directly published lane tip after one delay, then grades a receiver that lacks it rather than suppressing that receiver's ECHO. This comparison concerns the cross-author path only: the certified baselines provide stronger pre-carrier availability, while a Vantage tip manifest may remain open and use the resolver.

The signed one-hop rows separate three tradeoffs. Autobahn also admits a current lane tip after one delay, but requires a PoA for its parent and uses signed votes; its displayed $3\delta$ endpoint, like Vantage's, needs all $n$ participants. FinWhale's $p = 1$ fast path reaches the same endpoint and survives one non-voter, using signed DAG blocks. BBCA-Chain's carrier is a one-trip BEB block followed by a three-trip BBCA decision; its displayed advancement value is derived from its minimum admissible synchronization timer. At $n = 3f + 1$, Vantage therefore *matches* the lowest aligned endpoint in this survey without signatures; it does not strictly dominate these protocols or prove a lower bound.

## 4 MODEL AND PRELIMINARIES

*Parties and faults.* The protocol runs among a fixed set $\Pi = \{p_1, \ldots, p_n\}$ of $n \geq 3f+1$ parties. A party is *correct* if it follows the protocol and *Byzantine* otherwise. A static adversary chooses before execution a fixed set of $f_{\text{act}} \leq f$ Byzantine parties, which may deviate arbitrarily and are coordinated by a single computationally bounded adversary. Thus a correct party is one that is never corrupted during the execution. As $n \geq 3f+1$, at least $n - f \geq 2f+1$ parties are correct.

*Communication.* Every pair of parties shares a reliable, authenticated point-to-point channel: the recipient of a message learns the identity of its sender, the adversary can neither forge nor tamper with messages between correct parties, and every message sent between correct parties is eventually delivered. Channels are not assumed to be first-in, first-out (FIFO). Authentication is *not transferable*: that $q$ received $m$ on its channel from $p$ is not evidence $q$ can relay to a third party. This is the relevant difference from the signature-based setting, and the reason non-equivocation must be re-established through quorums. A concrete link may use a pairwise message-authentication code (MAC), but protocol messages contain no transferable vector of recipient-specific MACs: forwarded authentication tags are never accepted or counted as evidence. A broadcast includes its sender: a party delivers its own message to itself immediately and counts it as a first-hand message from itself.

*Timing.* We assume partial synchrony [17]: there is an unknown Global Stabilization Time (GST) and a known bound $\Delta$ such that every message between correct parties sent at time $t$ arrives by $\max(t, \text{GST}) + \Delta$. Safety never rests on timing; liveness is guaranteed after GST. For a particular execution, let $\delta \leq \Delta$ denote its actual upper bound after GST: every correct-to-correct message sent at time $t$ arrives by $\max(t, \text{GST}) + \delta$. The protocol knows $\Delta$ but not $\delta$; latency statements use $\delta$ to state the execution's actual delay. This is an explicit *in-flight-message convention*: even a correct-to-correct message sent before GST is delivered by GST + $\delta$ in the execution-specific analysis. Local clocks are monotone throughout the execution and measure elapsed real time without drift after GST; they need not have a common origin. Thus a timer armed for duration $\theta$ before GST has at most $\theta$ local time remaining at GST and expires within at most $\theta$ real time thereafter. At a local deadline, a party processes every message delivered by that deadline, and evaluates the positive rules those messages enable, before taking the timeout rule. The protocol uses the constants

$$\theta_E = 3\Delta, \qquad \theta_R = 4\Delta$$

for a missing echo-stage response and a missing initial ready-stage response — the smallest values for which the derivations in Theorem B.8 and lemma B.6 hold as written, tight when $\delta = \Delta$; they are not claimed to be protocol lower bounds. These constants affect only timer-triggered responses; positive evidence fires a rule as soon as its local condition holds.

*Execution fairness.* Liveness is stated for fair, non-Zeno executions. Every finite real-time interval contains only finitely many protocol events; local time and armed timers at correct parties advance; every action that remains enabled at a correct party is eventually taken; and a correct holder eventually serves a request issued by a correct party. These assumptions rule out an execution that performs infinitely many view transitions in finite time or permanently starves an enabled local action. They impose no delivery deadline before GST.

*Cryptography.* Beyond the ideal authenticated-channel abstraction, the protocol's sole cryptographic primitive is a collision-resistant hash function $\mathcal{H}\colon \{0,1\}^* \to \{0,1\}^\lambda$ with security parameter $\lambda$. For a post-quantum instantiation,

collision resistance is required against quantum polynomial-time adversaries. The protocol uses no digital signatures, public-key infrastructure, or threshold cryptography. Relative to the ideal channel abstraction, all guarantees hold except with negligible probability in $\lambda$, with a hash collision as the only protocol-level computational failure event. A concrete MAC-based channel implementation additionally relies on pairwise-MAC unforgeability; a post-quantum implementation requires that property against quantum polynomial-time adversaries as well. We treat $\mathcal{H}(x)$ as a succinct, collision-free reference to $x$.

*Instance and wire encoding.* An execution fixes a session identifier sid, which binds the membership, fault threshold, and application epoch; a maximum data payload length $\ell_{\max}$; and a deterministic, state-independent validity predicate BlockOK for data blocks (state-dependent transaction checks are performed later by deterministic execution). Every protocol message carries sid; a party rejects a message from another session before storing or counting it. All messages use one canonical, prefix-free byte encoding enc with author-indexed vectors in increasing author order, and every hash input begins with a distinct fixed domain tag and sid (for example, `data-block` or `view-proposal`), so objects of different types cannot share an encoding. Messages with a noncanonical encoding, more entries than the type-specific caps below, or a data payload longer than $\ell_{\max}$ are malformed and are never counted.

*Quorums.* A *quorum* is any set of $n-f$ parties. We use repeatedly that, for $n \geq 3f+1$:

(i) every quorum contains at least $n - 2f \geq f+1$ correct parties, and the correct parties always form a quorum; and

(ii) any two quorums $Q_1, Q_2$ satisfy $|Q_1 \cap Q_2| \geq 2(n-f) - n = n - 2f \geq f+1$, hence share at least one correct party.

## 5 AVAILABILITY-GRADED BROADCAST

The proposer of view $v$ broadcasts $B_v = (C_v, T_v, M_v)$. The quorum-available core manifest $C_v$ contains at most one lane frontier per author and, for a correct proposer, each entry has $n-f$ author-bound acknowledgments. The optimistic tip manifest $T_v$ contains newer lane frontiers received directly from their encoded authors; it is not certified before proposal. The auxiliary field $M_v$ is empty in data-only proposals and carries bounded resolution metadata in resolution-bearing proposals. Every manifest is sent by value in the proved protocol. Full lane syntax, publication, acknowledgment, repair, and the value-domain hooks appear in Appendix B.

Direct AGB separates *completion* from *sealing*. Completion fixes one proposal and makes its core irrevocable. A homogeneous grade-1 READY quorum seals commit-full$(C_v, T_v)$, a homogeneous grade-0 quorum seals commit-core$(C_v)$, and a residual mixed quorum leaves open-tip$(C_v, T_v)$ for the Vantage composition. Direct AGB never outputs skip and does not promise totality for a Byzantine proposer.

### 5.1 Compact protocol

All statements are broadcast over authenticated channels and counted only when received directly from distinct authors. Let $Q = n - f$. Each party sends one immutable echo-stage response and one initial ready-stage response per view; an initial READY-mix may refine once to a homogeneous READY for the same proposal. The complete event and direct result follow these four rules.

**R1. Propose.** The designated proposer sends one well-formed $(C, T, M)$. A correct Vantage proposer forms $C$ from directly published, $Q$-acknowledged lane frontiers and $T$ from directly published extensions. Mixed-tip quarantine may omit from later tip manifests an author recorded by an earlier READY-mix, but never from the core.

**R2. Echo.** A receiver fixes the first proposal received directly from the proposer. After the proposal becomes active through formal entry or the responsive frontier, the receiver immediately ECHOs grade 1 if core, tip, and metadata pass their local checks. If it has not sent that ECHO by its bounded echo fallback deadline, it sends grade 0 when the core and metadata pass, or ECHO-SKIP otherwise. If no active well-formed proposal exists by the absolute deadline $e_i(v) + 3\Delta$, it sends ECHO-SKIP. Repair can provide bytes but cannot create direct provenance or acknowledgments.

**R3. Ready.** On $Q$ ECHOs for one proposal and a satisfied metadata guard, a party broadcasts READY-1 or READY-0 if that grade already has $Q$ ECHOs, and READY-mix otherwise. A provisional mix refines once if one grade reaches $Q$ before all ECHO responses or the deadline; otherwise the refinement window closes. If the party has sent no READY by $e_i(v) + 4\Delta$, it emits NO-READY.

**R4. Complete and direct-seal.** Any $Q$ READYs naming one proposal, grades ignored and each author counted once, complete it. A homogeneous $Q$ additionally emits the corresponding full or core direct result. Late READYs continue to be counted after completion.

The WISH pacemaker is the only formal entry mechanism. A WISH message carries a view high-watermark. A party relays that high-watermark after first-hand support from $f + 1$ authors and, after support from $Q$ authors, enters each missing view up to that high-watermark. Immediately before sending either its echo-stage response for view $v - 1$ or its initial ready-stage response for view $v - 2$, a party checks whether that send completes the pair; if so, it raises its own WISH high-watermark through $v + 1$. A separate responsive frontier activates consecutive proposals as soon as they arrive; it is scheduling state, never counted evidence. After GST, correct entries to a view lie within $2\delta$, within $\delta$ when all parties are correct, and one view can delay advancement by at most $4\Delta + \delta$. The complete pacemaker and timer proof are in Appendix B.3.

Vantage adds one data-only shortcut outside Direct AGB. Before sending an exact grade-1 ECHO for $B = (C, T, \emptyset)$, a party installs an optimistic lock on that full payload. The lock releases permanently after $f + 1$ first-hand nonmatching echo-stage responses. Counting all $n$ exact grade-1 ECHOs submits commit-full$(C, T)$ to Vantage's TRYSEAL arbiter without creating completion, a direct result, or a completion report. The ordinary Direct AGB READY path continues in parallel.

### 5.2 Guarantees

The *pacemaker cutoff* below is the finite view after which the pacemaker supplies the entry and proposal-timing premises of conditional termination.

THEOREM 5.1 (DIRECT AGB AND FAST-SEAL SUMMARY). *Without timing assumptions, no two correct parties complete different proposals in one view; every direct result is compatible with the unique completion, and a completed core is valid, author-backed, and permanently retrievable. Every correct-proposer view past the pacemaker cutoff completes at all correct parties when its persistent core and metadata checks and its READY guard hold. In an all-correct, data-only responsive suffix, a prefix-extending proposal fast-seals full within $2\delta$ of its send; the Direct AGB READY path seals full within $3\delta$.*

The safety proof is quorum intersection over first-hand, one-shot ECHOs and READYs. A READY quorum contains a correct sender grounded in an ECHO quorum; two such quorums intersect in a correct immutable response. For the Vantage-only shortcut, the all-$n$ exact grade-1 ECHO set fixes every correct ECHO, while the optimistic locks exclude a

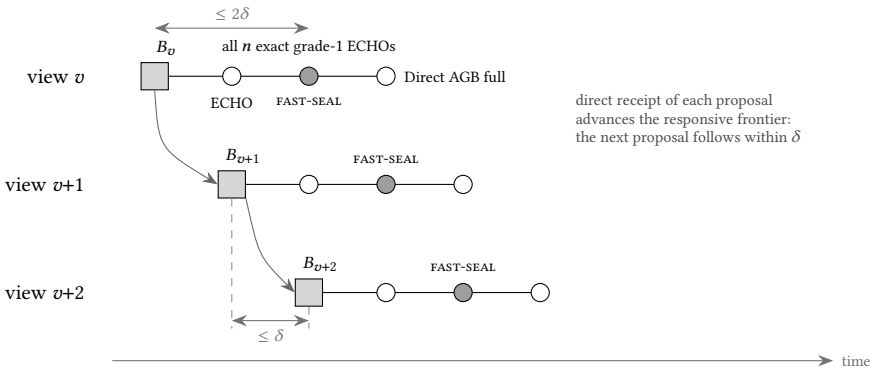


Fig. 3. Pipelined Vantage views in the all-correct data-only case. Every exact grade-1 ECHO arrives everywhere within two delays of proposal send and fast-seals commit-full; the Direct AGB READY result follows within three. Consecutive fresh proposal opportunities are at most one actual delay apart and do not wait for older response windows.

conflicting resolution-bearing proposal's ECHO quorum in either temporal order. The full safety, conditional-termination, and lock-compatibility proofs are in Appendix B.5 and lemma D.4.

For a block aligned with the next proposal opportunity, tip admission costs $\delta$ and the fast seal costs another $2\delta$, giving the $3\delta$ endpoint in Table 1; the READY endpoint is $4\delta$. A quorum-available core needs one additional publication delay. With arbitrary arrival phase, the proved tip endpoints are $4\delta$ in the worst phase and $3.5\delta$ in expectation under the uniform-phase convention; the corresponding core endpoints are $5\delta$ and $4.5\delta$. These latencies do not bound mixed or Byzantine resolution. Direct AGB uses $O(n^2)$ logical messages per view — at most $3n^2 + O(n)$ with READY refinements — and the proved by-value protocol uses $O(n^3\lambda)$ bits, excluding lane publication, repair, and the control log.

## 6 VANTAGE RESOLUTION, CONTROL, AND OUTPUT

A completed view can remain open when its READY grades do not become homogeneous; a silent or partially disseminated view may not complete at all. Vantage combines four compatible submissions in its TRY-SEAL arbiter: a Direct AGB full/core result, the all-$n$ fast-seal full result, a grounded skip result, or the first applicable control-log anchor. The arbiter stores the first submission and treats later compatible submissions as idempotent. It is Vantage composition state, not a second Direct AGB input. For target view $u$, the first applicable anchor is the earliest control-log position naming a proposal view $w \geq u + 3$ whose metadata resolves $u$.

The reusable resolver contract is per target: all correct calls choose one exact outcome; a full, core, or skip call permanently excludes every future incompatible READY quorum; a non-skip call supplies author-backed manifests; and every correct party eventually obtains the chosen result. Direct AGB composed with any service satisfying these properties has one compatible terminal outcome, and a completed view can never resolve as skip. Vantage realizes the contract with the guards below and the first applicable anchor; the formal resolver properties and the generic composition theorem are in Appendix C.1.

Every target view has a persistent stance $z_i(u) \in \{\mathsf{free}, \mathsf{non\text{-}skip}, \mathsf{skip\text{-}voted}\}$. Before ECHOing a resolution-bearing proposal with a non-skip entry, a correct party atomically claims the non-skip stance and latches the metadata checks; before sending SKIP-VOTE, it instead claims the free stance as skip-voted. The two sends are therefore mutually exclusive in either order. A non-skip resolution-bearing proposal is also checked against the party's final target ECHO/READY

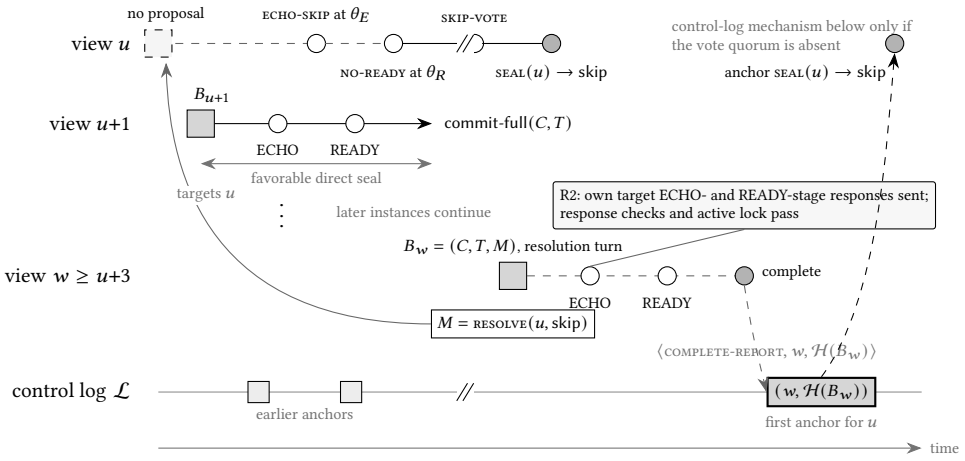


Fig. 4. Resolution mechanisms for a silent or open view. A clean crash-only view can skip after grounded post-ready votes. Other unresolved views use a later resolution-bearing proposal and the sparse control log; the first applicable anchor seals the target. Later data-plane views continue in parallel, but ordered output cannot cross the unresolved cursor position.

**Algorithm 1:** Compact Vantage composition at party $p_i$; $Q = n - f$

**upon** $f + 1$ *direct* WISH *high-watermarks support* $x$ *above the own wish* **do**
  persist and broadcast the own WISH high-watermark through $x$

**upon** $Q$ *direct* WISH *high-watermarks support* $x$ **do**
  invoke ENTER$(v)$ for every locally missing $v \leq x$, in order

**upon** *Direct AGB emits* COMPLETE$(v) \to B_v = (C_v, T_v, M_v)$ **do**
  store the irrevocable core; **if** $M_v \neq \emptyset$ **then**
    broadcast $\langle$COMPLETE-REPORT, $v, \mathcal{H}(B_v)\rangle$

**upon** *a Direct result, all* $n$ *exact data-only grade-*1 *ECHOs, or* $Q$ *grounded* SKIP-VOTE *statements* **do**
  submit respectively commit-full/commit-core, commit-full, or skip to TRYSEAL

**upon** *own* NO-READY *and* $Q$ *direct* ECHO-SKIP *for target* $u$*, with* $z_i(u) =$ free *and no non-skip seal* **do**
  atomically persist $z_i(u) \leftarrow$ skip-voted and broadcast $\langle$SKIP-VOTE, $u\rangle$

**upon** *R2 is about to ECHO a resolution-bearing proposal with a non-skip entry for* $u$ **do**
  require final target ECHO and READY stages, compatible local history and lock, and $z_i(u) \neq$ skip-voted; atomically persist $z_i(u) \leftarrow$ non-skip, latch the successful metadata checks, and emit the proposal ECHO

**upon** *the next contiguous control-log value* $(w, \mathcal{H}(B_w))$ *is delivered* **do**
  fetch and verify $B_w$; process its metadata; for every target $u$ whose first applicable anchor this is, invoke APPLY-ANCHOR$(u, X_u)$ and submit $X_u$ to TRYSEAL

**function** TRYSEAL$(u, X)$:
  **if** $u$ *has no terminal outcome* **then**
    persist $X$ and fire SEAL$(u) \to X$
  run the output cursor in view order: emit a completed core after all earlier views are sealed; stop at an open tip; after a full/core/skip seal append the verified tip, omit it, or emit nothing, then advance

history and any active optimistic lock. Its ECHO carries origin value 1 for a full entry only if the sender previously sent a grade-1 target ECHO for the exact payload, and for a core entry only if it sent a target ECHO of either grade for that payload. The carrying view's READY guard requires $f + 1$ such first-hand origin bits, ensuring that one correct target ECHO sender checked the output manifests and established their publication provenance. Full guards and candidate-selection rules are given in Appendix C.

### 6.1 Sparse control log

Only residual resolution-bearing proposals use the control plane. Here "sparse" refers to nonempty anchors: the log still runs serial rounds when their values are empty. A genuine Direct AGB completion of such a proposal broadcasts a

completion report; fast and skip seals do not. After $Q$ reports and body validation, the pair $(w, \mathcal{H}(B_w))$ is eligible for a validated Bracha broadcast embedded in the non-speculative Simple-IT log. The log orders only anchor hashes. Correct parties consume one contiguous log prefix, fetch every body before processing it, and apply the first logged proposal view at least $u + 3$ whose metadata resolves target $u$. Direct homogeneous results and the clean crash-only skip do not wait for this log.

The validated broadcast requires a correct ECHO author to have first-hand completion evidence and the proposal body. Bracha uniqueness and totality therefore preserve a common anchor sequence, while the imported Simple-IT parent and notification rules ensure that every resolution-bearing proposal which completes everywhere is eventually anchored. The complete adapter, reliable-notification rules, body-repair protocol, and imported-theorem mapping are reproduced in Appendix D.

Theorem 6.1 (Sparse control-log summary). *Correct parties observe prefixes of one anchor sequence and eventually observe every anchor. Every nonempty anchor names a proposal that completed at a correct party and therefore carries echo-accepted metadata. Every resolution-bearing proposal that completes at all correct parties is eventually anchored.*

### 6.2 End-to-end guarantees

Theorem 6.2 (Vantage correctness). *Safety holds without timing assumptions: every correct party seals each view at most once, no two correct parties seal different outcomes, and their output sequences are always prefix-comparable. Every output block is valid, published with its lane prefix by its encoded author to a correct party, permanently retrievable, and obtained before local output. Under partial synchrony, the fixed static fault set, fair repair, and the non-Zeno execution assumption, every view is eventually sealed, every finite common output prefix is eventually produced everywhere, and every valid block of a correct author is eventually included.*

The proof separates compatibility across sealing mechanisms from totality. Quorum intersection makes direct results agree; optimistic locks exclude conflicts between fast-seal and resolver submissions; persistent stances exclude a skip-vote quorum from any ECHO quorum for a non-skip resolution-bearing proposal. The common anchor order makes the resolver choice identical at all correct parties, after which the prefix cursor gives one prefix-comparable output stream. For a clean crash-only silent view whose first correct entry is $s \geq GST$ and whose correct entries finish by $s + 2\delta$, all correct parties skip-seal by $s + 4\Delta + 3\delta$; aligned entry gives $4\Delta + \delta$. This is a per-view bound, not a bound for a burst of failures. Full statements and proofs appear in Appendix E.

## 7 IMPLEMENTATION AND EVALUATION

### 7.1 Implementation and methodology

We implement Vantage in Rust. One deterministic task handles AGB, lane frontiers, view changes, recovery, and ordered output; networking and storage run separately. The same binary runs optimistic all-to-all Autobahn, seamless Autobahn, and two Simple-IT variants on the same Transmission Control Protocol (TCP), framing, batching, worker, storage, client, and metrics stack. Bluestreak and Sailfish++ use the separate Starfish artifact and its protocol stack. No Vantage protocol step performs a digital-signature or public-key operation. Canonical one-byte committee indices (compact IDs) reduce wire size but provide no authentication: the benchmark TCP stack does not cryptographically bind a connection to its declared committee identity, so the measurements omit the pairwise channel-authentication cost assumed by the model.

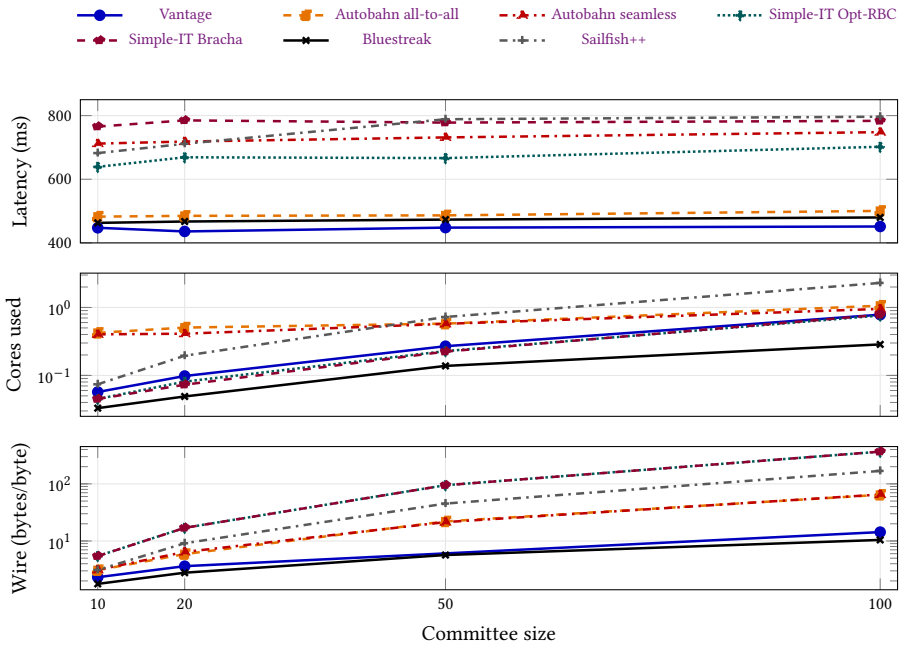


Fig. 5. Fault-free committee scaling at 100 transactions per second. Latency, processor use, and comparison-protocol wire points are cloud deployment medians from one measured window. Vantage wire uses compact-ID local measurements at $n = 10, 20$ and the separate throughput campaign at $n = 100$; no $n = 50$ wire marker is reported. Processor use and wire use logarithmic scales.

Validators run in the Amazon Web Services (AWS) `eu-west-1a` availability zone over private addresses while netem injects the ten-region round-trip time (RTT) matrix of Table 2. Transactions are random, effectively incompressible 512-byte values submitted open loop and evenly across validators. Rates are reported in transactions per second (tx/s). Every reported point has a 30-second warmup and one 120-second measurement window. Latency runs from client submission to local sequencing plus byte retrieval; we report the median validator's latency, central processing unit (CPU) use, and outbound framed bytes per sequenced byte. We write p50, p90, and p99 for the 50th, 90th, and 99th percentiles, and use the same notation for other percentiles. One run per point means the results are descriptive and have no confidence intervals. Exact instance types, the RTT matrix, workload and metric definitions, artifacts, and all supplementary measurements are in Appendix F.

The implementation carries acknowledgments positionally in ECHO envelopes, names repeated single-proposal statements by digest, batches transport, and uses checkpoints, bounded reconnect replay, and a 200-view garbage-collection (GC) window. These are representation and engineering choices, not additional evidence. The aligned single-proposal encoding uses $O(n^2\lambda + n^3)$ control bits, while ragged exceptions and resolution traffic retain the proved $O(n^3\lambda)$ worst case. The formal protocol still assumes unbounded retention.

### 7.2 Q1: Fault-free committee scaling

We run seven configurations at $n \in \{10, 20, 50, 100\}$ under a fixed aggregate 100 tx/s. Each validator uses one `c5d.xlarge` instance. All 28 points completed on their first attempt, and every validator sustained at least 99.2% of offered throughput.


Fig. 6. One Vantage view at $n = 20$. Top: p50 interval bars with p90 circles and p99 crosses; intervals describe different event populations and are not stacked. Bottom: outgoing typed bytes and logical unicast messages by module.

Vantage has the lowest p50 and p99 latency at every committee size. Its p50 changes from 447.5 ms at $n = 10$ to 451.5 ms at $n = 100$. At $n = 100$ it is 28.4 ms faster at p50 and 23.8 ms faster at p99 than Bluestreak, while using 2.74× its CPU and 1.37× its measured compact-ID wire cost. Against optimistic all-to-all Autobahn, it is 49.0 ms faster at p50, uses 26% less CPU, and has 78% lower measured wire cost. Bluestreak has the lowest CPU and wire cost. The fixed low load measures committee-scale background traffic; it is not a capacity or empirical-asymptotic experiment.

### 7.3 Q2: Inside one Vantage view

To measure the claimed overlap directly, we run Vantage at $n = 20$ and 100 tx/s with the same RTT matrix in a controlled local mechanism study. Each of three 60-second runs sequences all 6,000 submitted transactions.

The interval medians describe different event populations and are not additive. The median worker-batch wait is 11 ms, block-publication interval is 51 ms, block-to-sequencing interval is 368 ms, and local-materialization interval is 6 ms. A block is covered by a proposal after 175 ms but becomes quorum-available after 289 ms: proposal coverage therefore precedes quorum availability in the measured one-hop execution. The end-to-end median is 444 ms. In a matched network-delay control (Figure 8), the all-$n$ fast-seal shortcut supplies 70.0–70.5% of local seals under the heterogeneous matrix and every observed seal under a uniform RTT; all outcomes in both experiments are full.

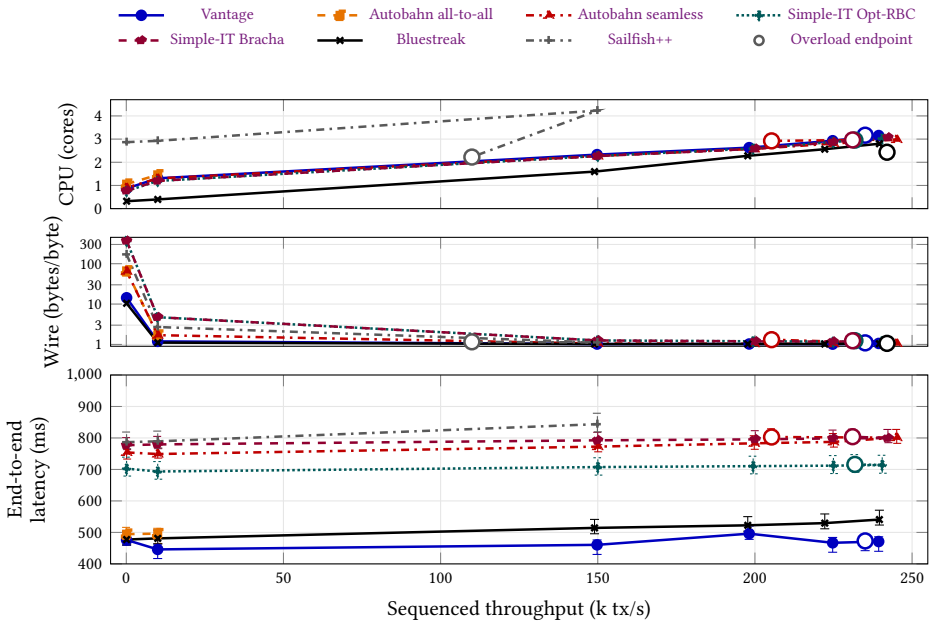


Fig. 7. Throughput sweep at $n = 100$, one run per point. Most points use `c5d.2xlarge`; Bluestreak and Sailfish++ use the separate `m5d.2xlarge` rerun from 200k. The x-axis is measured sequenced throughput; latency is submission-to-materialization p50 with p5–p95 validator whiskers. Open markers are the first point below 95% of offered load.

### 7.4 Q3: Throughput–latency at $n = 100$

We increase offered load from 100 to 275,000 tx/s. Most points use `c5d.2xlarge`; after payload compaction, Bluestreak and Sailfish++ use a separate `m5d.2xlarge` fleet from 200k tx/s. Both have eight virtual CPUs (vCPUs), but their CPU families and memory differ, so the combined high-load curves are descriptive rather than hardware-identical. A point is accepted when median sequenced throughput reaches at least 95% of offered load; the first lower point is retained as the overload endpoint.

Vantage has the lowest p50 latency at every accepted offered rate, remaining between 446 and 496 ms through 250k tx/s. At that load it sequences 239.4k tx/s with 471 ms p50 and 646 ms p99. Bluestreak also reaches 250k on the separate m5d fleet, with 540.8 ms p50 and 1,478.8 ms p99, and has the lowest observed CPU and wire cost among accepted configurations. Vantage's peak accepted throughput is within 3% of the best signature-free configuration. Vantage and Bluestreak overload at 275k; Sailfish++ overloads at 200k. Optimistic all-to-all Autobahn has no accepted point at 150k or above. These single-run endpoints characterize the measured deployments, not protocol capacity bounds.

## 8 DISCUSSION AND LIMITATIONS

*Reuse boundary.* Direct AGB applies when one committee handles a value with two parts: a stable prefix and an optional suffix that can be omitted safely. A shared mempool or a checkpoint with a speculative log tail may have this structure, but each use must define its own eligibility rules and prove availability. This paper proves only the author-lane instance. AGB alone is not reliable broadcast, a cross-committee availability certificate, or consensus. Vantage adds the resolver, deterministic order, repair, and input fairness identified in Corollary C.2.

The grounded skip-vote shortcut is deliberately narrow. Relaying a skip vote after $f + 1$ votes is unsafe if a party has already echoed a non-skip proposal. Requiring a free stance avoids that conflict but may leave too few parties to complete the relay. Vantage therefore uses the grounded shortcut only for a clean crash-only silent view and uses the ordered resolver for all other failed views.

*Tip spoiling.* One unverifiable entry makes the entire optional tip manifest fail its local check, so a faulty author can prevent other authors' fresh blocks in that manifest from being included. The faulty author publishes its lane-tip block only to the correct proposer and withholds matching acknowledgments from every other correct party. Repair may supply the bytes, but it cannot recreate direct proof of authorship. This is the same withholding problem that Multimmit calls easy spoiling in Raptr's prefix voting [24, 37].

The attack does not block a correct proposer's completion or its core. At $n = 3f + 1$, the proposer is the only correct party that can send grade 1 for the resulting tip manifest. Even if all $f$ Byzantine parties also send grade 1, at most $1 + f < 2f + 1$ grade-1 ECHOs exist. Thus neither the all-$n$ fast-seal shortcut nor a grade-1 READY quorum can form. A mixed ECHO quorum sends READY-mix immediately, and ECHO and READY quorums count both grades, so ordinary completion remains available. If grade 0 later reaches $n-f$, each provisional sender can refine to READY-0 and the Direct AGB READY path seals commit-core; otherwise the direct result may remain open-tip. Neither case treats repaired possession as proof of authorship.

A fully Byzantine schedule can still withhold enough ECHOs that neither grade reaches $n-f$ before the refinement window closes. The residual READY-mix then leaves the view open. To stop correct proposers from producing an open view from the same lane on every turn, a party that emits the initial mix records every author whose tip entry is not quorum-available. It omits that author's later tip entries until the recorded prefix becomes quorum-available or enters a containing core. This quarantine state stores one reference per author; ordinary inclusion and the all-correct grade-1 case do not add entries. This is a correct-proposer discipline, not a receiver-verifiable formation rule, so Byzantine proposers can ignore it and continue creating residual open views on their own turns. A Byzantine proposal can also cause a correct party to record a reference that its encoded author never published. That entry never clears, but it suppresses only the affected author's tip entries when that party proposes; the author's blocks still enter every core through quorum acknowledgment.

A separate possible extension is one grade bit per tip entry. Quorum intersection can then select an outcome for each two-valued coordinate, producing $(C, S)$ for some $S \subseteq T$ and confining simple withholding to one author's lane. This does not remove the resolver: Byzantine ECHOs can still make coordinates mixed, and the optimistic lock and resolution checks would need a new per-coordinate proof. We have not carried out that verification. A monotone bitmap over several heights may extend the same idea; unlike Multimmit's rank rule, the individual questions remain two-valued at $n = 3f + 1$.

The Direct AGB homogeneous path does not wait for the control log, but both still share CPU and network resources. The formal construction runs serial Simple-IT rounds even when their values are empty; "sparse" refers to nonempty anchors, not to round activity. An open tip also blocks ordered output until its resolution is logged. Mixed-tip quarantine prevents repeated open views from correct proposers but does not bound adversarial proposals or the service time of existing anchors. If residual open views arrive faster than the log resolves them, the queue can still grow and long-run throughput can fall to the log's service rate. Fair selection eventually resolves every fixed view, but gives no bound on this queue or on general failed-view latency. Only the clean crash-only case of Corollary D.10 has a direct bound. A fault experiment must measure this interference.

The formal protocol also does not bound storage. Correct parties retain lane prefixes, response and timer state, completion bodies, and control-validation data. State is finite at every finite time but can grow without bound over an infinite execution. The implementation's checkpoints, garbage collection, and admission control therefore need a separate safety argument.

The provenance guarantee concerns validator-authored blocks: an output block under index $i$ was published by $p_i$ to a correct party. A Byzantine author may still invent payload bytes or publish forks. Authenticating external clients is an application concern, and state-dependent invalid or replayed transactions must be handled deterministically during execution. These boundaries do not weaken common block ordering, but they are not client-level no-creation guarantees.

## 9 CONCLUSION

Vantage shows that a signature-free multi-author data path can admit an author's block after receiving it directly over an authenticated channel, without first certifying that block. Each proposal pairs a quorum-available core manifest with an optimistic tip manifest. AGB fixes the core and uses first-hand responses to determine whether the tip is included. When all parties respond, Vantage seals a data-only proposal within two message delays of its send, matching the proposal-relative latency of signed all-to-all fast paths while using only authenticated channels and hashes.

Every output block is valid, author-backed, and retrievable, and every valid block from a correct author is eventually included. A clean crash-only silent view is skipped without waiting for the control log. In our $n = 100$ experiments, Vantage has the lowest median latency at every accepted load and sequences 239k tx/s with median latency below 500 ms. Its peak is within 3% of the best measured signature-free configuration. The tradeoffs remain important: the control log consumes resources, general control-log resolution latency is unbounded, and the formal protocol assumes unbounded retention. Repeated runs and fault experiments are the next empirical steps.

## ACKNOWLEDGMENTS

Models from Anthropic and OpenAI assisted with code development and grammatical editing of the manuscript under the author's direction.

## REFERENCES

[1] Ittai Abraham, Sourav Das, Yuval Efron, and Jovan Komatovic. 2026. Forget-IT: Optimal Good-Case Latency for Information-Theoretic BFT. Cryptology ePrint Archive, Paper 2026/355.

[2] Ittai Abraham, Ling Ren, and Zhuolun Xiang. 2022. Good-Case and Bad-Case Latency of Unauthenticated Byzantine Broadcast: A Complete Categorization. In *International Conference on Principles of Distributed Systems (OPODIS) (Leibniz International Proceedings in Informatics, Vol. 217)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 5:1–5:20. https://doi.org/10.4230/LIPIcs.OPODIS.2021.5

[3] Ittai Abraham and Gilad Stern. 2021. Information Theoretic HotStuff. In *International Conference on Principles of Distributed Systems (OPODIS) (Leibniz International Proceedings in Informatics, Vol. 184)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 11:1–11:16. https://doi.org/10.4230/LIPIcs.OPODIS.2020.11

[4] Michael Anoprenko, Andrei Tonkikh, Alexander Spiegelman, Petr Kuznetsov, Anatoliy Zinovyev, and Konstantin Shprenger. 2025. DAGs for the Masses. https://doi.org/10.48550/arXiv.2506.13998 arXiv:2506.13998v4, revised 1 March 2026.

[5] Balaji Arun, Zekun Li, Florian Suri-Payer, Sourav Das, and Alexander Spiegelman. 2025. Shoal++: High Throughput DAG BFT Can Be Fast!. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, Philadelphia, PA, USA, 813–826. arXiv:2405.20488.

[6] Kushal Babel, Andrey Chursin, George Danezis, Anastasios Kichidis, Lefteris Kokoris-Kogias, Arun Koshy, Alberto Sonnino, and Mingwei Tian. 2025. Mysticeti: Reaching the Latency Limits with Uncertified DAGs. In *Network and Distributed System Security Symposium (NDSS)*. Internet Society, Reston, VA, USA, 18 pages. https://doi.org/10.14722/ndss.2025.240929

[7] Kushal Babel, Fatima Elsheimy, Lioba Heimbach, Mohammad Mussadiq Jalalzai, Tobias Klenze, Jovan Komatovic, Jason Milionis, Mike Setrin, and Victor Shoup. 2026. Cadence: Extreme Pipelining with Multiple Concurrent Proposers. arXiv:2607.02275 [cs.DC] arXiv:2607.02275v2.

[8] Leemon Baird and Atul Luykx. 2020. The Hashgraph Protocol: Efficient Asynchronous BFT for High-Throughput Distributed Ledgers. In *2020 International Conference on Omni-layer Intelligent Systems (COINS)*. IEEE, Piscataway, NJ, USA, 1–7. https://doi.org/10.1109/COINS49042.2020.9191430

[9] Gabriel Bracha. 1987. Asynchronous Byzantine Agreement Protocols. *Information and Computation* 75, 2 (1987), 130–143.

[10] Manuel Bravo, Gregory Chockler, and Alexey Gotsman. 2022. Liveness and Latency of Byzantine State-Machine Replication. In *36th International Symposium on Distributed Computing (DISC) (Leibniz International Proceedings in Informatics, Vol. 246)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Wadern, Germany, 12:1–12:19. https://doi.org/10.4230/LIPIcs.DISC.2022.12

[11] Christian Cachin and Stefano Tessaro. 2005. Asynchronous Verifiable Information Dispersal. In *IEEE Symposium on Reliable Distributed Systems (SRDS)*. IEEE Computer Society, Los Alamitos, CA, USA, 191–202.

[12] Miguel Castro. 2001. *Practical Byzantine Fault Tolerance.* Ph. D. Dissertation. Massachusetts Institute of Technology. Technical Report MIT-LCS-TR-817; the MAC-authenticated variant.

[13] Miguel Castro and Barbara Liskov. 1999. Practical Byzantine Fault Tolerance. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, New Orleans, LA, USA, 173–186.

[14] Benjamin Y. Chan and Rafael Pass. 2023. Simplex Consensus: A Simple and Fast Consensus Protocol. In *Theory of Cryptography Conference (TCC) (Lecture Notes in Computer Science, Vol. 14372)*. Springer, Cham, Switzerland, 452–479. https://doi.org/10.1007/978-3-031-48624-1_17

[15] Xiaohai Dai, Zhaonan Zhang, Jiang Xiao, Jingtao Yue, Xia Xie, and Hai Jin. 2023. GradedDAG: An Asynchronous DAG-Based BFT Consensus with Lower Latency. In *IEEE International Symposium on Reliable Distributed Systems (SRDS)*. IEEE Computer Society, Los Alamitos, CA, USA, 107–117. https://doi.org/10.1109/SRDS60354.2023.00020

[16] George Danezis, Lefteris Kokoris-Kogias, Alberto Sonnino, and Alexander Spiegelman. 2022. Narwhal and Tusk: A DAG-Based Mempool and Efficient BFT Consensus. In *European Conference on Computer Systems (EuroSys)*. Association for Computing Machinery, New York, NY, USA, 34–50. https://doi.org/10.1145/3492321.3519594

[17] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. 1988. Consensus in the Presence of Partial Synchrony. *J. ACM* 35, 2 (1988), 288–323. https://doi.org/10.1145/42282.42283

[18] Paul Feldman and Silvio Micali. 1988. Optimal Algorithms for Byzantine Agreement. In *Proceedings of the 20th Annual ACM Symposium on Theory of Computing (STOC)*. Association for Computing Machinery, New York, NY, USA, 148–161.

[19] Yingzi Gao, Yuan Lu, Zhenliang Lu, Qiang Tang, Jing Xu, and Zhenfeng Zhang. 2022. Dumbo-NG: Fast Asynchronous BFT Consensus with Throughput-Oblivious Latency. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. Association for Computing Machinery, New York, NY, USA, 1187–1201. https://doi.org/10.1145/3548606.3559379

[20] Neil Giridharan, Florian Suri-Payer, Ittai Abraham, Lorenzo Alvisi, and Natacha Crooks. 2024. Autobahn: Seamless High Speed BFT. In *ACM Symposium on Operating Systems Principles (SOSP)*. Association for Computing Machinery, New York, NY, USA, 1–23. https://doi.org/10.1145/3694715.3695942

[21] Idit Keidar, Lefteris Kokoris-Kogias, Oded Naor, and Alexander Spiegelman. 2021. All You Need Is DAG. In *ACM Symposium on Principles of Distributed Computing (PODC)*. Association for Computing Machinery, New York, NY, USA, 165–175. https://doi.org/10.1145/3465084.3467905

[22] Idit Keidar, Oded Naor, Ouri Poupko, and Ehud Shapiro. 2023. Cordial Miners: Fast and Efficient Consensus for Every Eventuality. In *International Symposium on Distributed Computing (DISC) (Leibniz International Proceedings in Informatics, Vol. 281)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 26:1–26:22. https://doi.org/10.4230/LIPIcs.DISC.2023.26

[23] Razya Ladelsky and Roy Friedman. 2026. FinWhale: An Optimally Resilient Two-Round Terminating DAG Protocol. arXiv:2606.26292 [cs.DC]

[24] Andrew Lewis-Pye and Patrick O'Grady. 2026. Multimmit: Extending Blocks for Faster Finality. arXiv:2607.21021 [cs.DC] arXiv:2607.21021.

[25] Andrew Lewis-Pye and Ehud Shapiro. 2026. Morpheus Consensus: Excelling on Trails and Autobahns. In *29th International Conference on Principles of Distributed Systems (OPODIS 2025) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 361)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 35:1–35:20. https://doi.org/10.4230/LIPIcs.OPODIS.2025.35

[26] Dahlia Malkhi, Chrysoula Stathakopoulou, and Maofan Yin. 2025. BBCA-CHAIN: Low Latency, High Throughput BFT Consensus on a DAG. In *Financial Cryptography and Data Security (Lecture Notes in Computer Science, Vol. 14744)*. Springer, Cham, Switzerland, 51–73. https://doi.org/10.1007/978-3-031-78676-1_4 28th International Conference, FC 2024; arXiv:2310.06335.

[27] Achour Mostéfaoui, Hamouma Moumen, and Michel Raynal. 2014. Signature-Free Asynchronous Byzantine Consensus with $t < n/3$ and $O(n^2)$ Messages. In *ACM Symposium on Principles of Distributed Computing (PODC)*. Association for Computing Machinery, New York, NY, USA, 2–9. https://doi.org/10.1145/2611462.2611468

[28] Nikita Polyanskii, Sebastian Müller, and Ilya Vorobyev. 2025. Making Uncertified DAG BFT Provably Live with Linear Payload and Quadratic Metadata Communication. Cryptology ePrint Archive, Paper 2025/567. https://eprint.iacr.org/2025/567

[29] Nikita Polyanskii, Ilya Vorobyev, and Sebastian Müller. 2026. Bluestreak: Scaling DAG BFT by Sparsifying Metadata. Cryptology ePrint Archive, Paper 2026/898.

[30] Nikita Polyanskii, Ilya Vorobyev, and Sebastian Müller. 2026. Starfish-S: Optimistic Payload Sequencing at Leader-Commit Latency. Manuscript.

[31] Mayank Raikwar, Nikita Polyanskii, and Sebastian Müller. 2024. SoK: DAG-based Consensus Protocols. In *2024 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, Piscataway, NJ, USA, 1–18. https://doi.org/10.1109/ICBC59979.2024.10634358

[32] Victor Shoup. 2024. Sing a Song of Simplex. In *International Symposium on Distributed Computing (DISC) (Leibniz International Proceedings in Informatics, Vol. 319)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 37:1–37:22. https://doi.org/10.4230/LIPIcs.DISC.2024.37 Introduces DispersedSimplex; full version: Cryptology ePrint 2023/1916.

[33] Nibesh Shrestha, Rohan Shrothrium, Aniket Kate, and Kartik Nayak. 2025. Sailfish: Towards Improving the Latency of DAG-based BFT. In *IEEE Symposium on Security and Privacy (S&P)*. IEEE Computer Society, Los Alamitos, CA, USA, 1928–1946. https://doi.org/10.1109/SP61157.2025.00021 Full version: Cryptology ePrint 2024/472.

[34] Nibesh Shrestha, Qianyu Yu, Aniket Kate, Giuliano Losa, Kartik Nayak, and Xuechao Wang. 2025. Optimistic, Signature-Free Reliable Broadcast and Its Applications. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. Association for Computing Machinery, New York, NY, USA, 3780–3794. https://doi.org/10.1145/3719027.3765220 Introduces Sailfish++, a signature-free DAG BFT protocol; use arXiv:2505.02761 v4 or later.

[35] Alexander Spiegelman, Balaji Arun, Rati Gelashvili, and Zekun Li. 2025. Shoal: Improving DAG-BFT Latency and Robustness. In *Financial Cryptography and Data Security (FC) (Lecture Notes in Computer Science, Vol. 14744)*. Springer, Cham, Switzerland, 92–109. 28th International Conference, FC 2024; arXiv:2306.03058.

[36] Alexander Spiegelman, Neil Giridharan, Alberto Sonnino, and Lefteris Kokoris-Kogias. 2022. Bullshark: DAG BFT Protocols Made Practical. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. Association for Computing Machinery, New York, NY, USA, 2705–2718. https://doi.org/10.1145/3548606.3559361

[37] Andrei Tonkikh, Balaji Arun, Zhuolun Xiang, Zekun Li, and Alexander Spiegelman. 2025. Raptr: Prefix Consensus for Robust High-Performance BFT. arXiv:2504.18649 [cs.DC] arXiv:2504.18649.

[38] Yann Vonlanthen, Jakub Sliwinski, Massimo Albarello, and Roger Wattenhofer. 2024. Banyan: Fast Rotating Leader BFT. In *Proceedings of the 25th International Middleware Conference*. ACM, New York, NY, USA, 494–507. https://doi.org/10.1145/3652892.3700788

[39] Feifan Wang, Jingfan Yu, Zixi Cai, and Zhixuan Fang. 2026. Clownfish: Scaling DAG-based BFT Consensus via Sparse Edges. arXiv:2606.04687. https://doi.org/10.48550/arXiv.2606.04687

[40] Qianyu Yu, Giuliano Losa, Nibesh Shrestha, and Xuechao Wang. 2026. Angelfish: Leader, DAG, or Anywhere in Between. https://arxiv.org/abs/2509.15847 To appear at ACM CCS 2026; full version, arXiv:2509.15847v3.

[41] Qianyu Yu, Giuliano Losa, and Xuechao Wang. 2024. TetraBFT: Reducing Latency of Unauthenticated, Responsive BFT Consensus. In *ACM Symposium on Principles of Distributed Computing (PODC)*. Association for Computing Machinery, New York, NY, USA, 257–267. https://doi.org/10.1145/3662158.3662783 Full version: arXiv:2405.02615.

[42] Qianyu Yu, Juan Villacis, Giuliano Losa, Zhuolun Xiang, and Xuechao Wang. 2026. Simple-IT: Practical Low-Latency Signature-Free BFT Consensus. arXiv:2606.14404 [cs.DC] arXiv:2606.14404.

## A COMPARISON DETAILS

This appendix expands the normalized comparison of Table 1. The calculations use the same aligned cross-author experiment and do not turn favorable endpoints into general fault-path bounds.

*Mempool-Simple-IT.* A leader names a batch only after collecting $2f + 1$ availability votes, and a receiver checks $f + 1$ locally [42]. This two-delay gate, two-delay Opt-RBC, and one commit-vote delay give $5\delta$ under the roughly $5n/6$ optimistic threshold. It certifies availability before proposal; Vantage admits a directly published tip after one delay but may leave it open.

*Sailfish++.* Opt-RBC makes each nonleader vertex referenceable after two delays. Then $n - f - 1$ ordinary correct nonleaders sequence in $5\delta$ and the remaining $f$ late ones in $7\delta$—two classes, not a range [34, 42]. This supplies per-vertex RBC; Vantage's graded tip instead gives the $3\delta$ all-to-all or $4\delta$ READY endpoint and may need an anchor.

*Autobahn.* The leader may reference a directly received current lane tip after one delay while its parent remains PoA-certified [20]. Linear fast consensus adds three delays, giving $4\delta$. On the sketched all-to-all path, Prepare is the carrier; one delay spreads every vote and another lets each party form the all-$n$ fast certificate locally. Thus the derived total is $\delta + 2\delta = 3\delta$, without a final certificate broadcast. The paper does not state this endpoint; like Vantage's all-$n$ fast-seal shortcut, one non-voter disables it. The optimistic tip is outside Autobahn's seamlessness guarantee: a missing receiver fetches it and withholds its vote, while weak/strong refinement still needs $f + 1$ strong assertions in the PrepareQC. Vantage lowers the grade without suppressing ECHO or READY, but an open tip has no constant resolver bound.

*FinWhale, Mysticeti, and compressed variants.* Cordial Miners begins this best-effort-DAG line [22]; Mysticeti pipelines leaders [6]; Starfish supplies the live Push pacemaker and multisigned Mysticeti-L [28]; Bluestreak compresses with ordinary signatures [29]; and FinWhale adds a two-round fast path [23]. Cross-author admission costs one delay, after which Mysticeti's three-message decision gives $4\delta$. FinWhale commits a round-$r$ leader after $n - p$ round-$(r + 1)$ references, where $n = 3f + 2p - 1$ and $1 \le p \le f$; creating and receiving them gives $3\delta$. At $p = 1$, $n = 3f + 1$ and one non-voter is tolerated. Its adopted Push pacemaker bounds a round by $2\Delta + 2\delta$ after GST.

Bluestreak sends one advance and block per party per round; Vantage sends one ECHO and initial READY per party per view plus a proposal. Both favorable patterns are $2n^2 + O(n)$; READY refinement raises Vantage's general bound to $3n^2 + O(n)$. Bluestreak's happy case is $O(n^2\lambda + nP)$; Vantage's positional bitmaps give $O(n^2\lambda + n^3)$ aligned, while its proved by-value and ragged/batch worst case remains $O(n^3\lambda)$.

*BBCA-Chain.* One BEB trip plus the referencing leader's three-trip BBCA gives the $4\delta$ favorable nonleader path [26]. Timely correct-leader BBCA tolerates $f$ faults; Vantage's two-delay carrier-to-seal shortcut assumes all parties correct, and its READY path takes three delays. BBCA-Chain waits for the preceding BBCA and recovers through signed complete, adopt, or no-adopt evidence. Vantage overlaps views with an at-most-$\delta$ fresh-opportunity gap but has no general recovery bound. The $4\delta$ excludes BBCA-Chain's optional blob-availability layer.

## B DIRECT AGB: FULL SPECIFICATION AND PROOFS

Vantage runs one pipelined AGB instance per view. Its round-robin proposer $\mathsf{Proposer}(v) = p_{1+((v-1) \bmod n)}$ broadcasts $B_v = (C_v, T_v, M_v)$. The core $C_v$ and tip $T_v$ are author-indexed lane-frontier manifests, encoded in increasing author order with at most one reference $(k, h)$ per author in each (Figure 1); we omit the coordinate and height when clear. These fields

are self-asserted and do not alone prove authorship. A correct proposer places an entry in its quorum-available core only after receiving its valid lane prefix directly from the encoded author and counting $n-f$ author-bound acknowledgments. For a faulty proposer, "quorum-available" is not certified; R2 instead checks that every output-relevant entry is author-backed, which suffices for the retrievability theorem below. Hashes are distinct within the proposal and disjoint across $C_v, T_v$; ordered output removes any cross-view repetition. The metadata $M_v$ is empty, one general resolution entry, or a bounded vector of manifest-free skips for earlier views (Section 6); it is hash-bound but is not payload. The proposal identifier $\mathcal{H}(B_v)$ abbreviates $\mathcal{H}(\texttt{view-proposal} \parallel \mathsf{enc}(\mathsf{sid}, v, C_v, T_v, M_v))$ and therefore binds the view, manifests, and metadata.

At its abstraction boundary, Direct AGB is parameterized by a deterministic syntactic predicate $\textsc{Formed}_v(C, T, M)$, local predicates $\textsc{CoreOK}_i(C)$ and $\textsc{TipOK}_i(C, T)$, and an opaque auxiliary field $M$. The value domain also supplies a stable item identity and a finite, deterministic item sequence $\mathsf{Items}(X)$ for each formed manifest. For every prior-item set $D$, $\mathsf{Core}_D(C)$ is $\mathsf{Items}(C)$ after removing identities in $D$ and repeated identities, and $\mathsf{Full}_D(C, T)$ appends the likewise deduplicated items of $\mathsf{Items}(T)$. Thus the core expansion is a prefix of the full expansion, and every full item comes from $C$ or $T$. Write $\mathsf{Witnessed}(X)$ for the value-domain guarantee that every item in $\mathsf{Items}(X)$ is externally valid and permanently retrievable. A conforming eligibility instantiation must prove

$$\textsc{CoreOK}_i(C) \Rightarrow \mathsf{Witnessed}(C), \qquad \textsc{TipOK}_i(C, T) \Rightarrow \mathsf{Witnessed}(T)$$

whenever $p_i$ is correct. Agreement uses only locality and determinism of these predicates; availability inherits the displayed witness implications.

The auxiliary field is supported by three hooks. A finite reference set $\textsc{AuxRefs}(M)$ names what a stored proposal authorizes for repair, and a local predicate $\textsc{AuxOK}_i(v, M)$ joins the echo gate. The *ready guard* consists of a local annotation $\textsc{Ann}_i(v, M)$ that a party attaches to its proposal echo — immutable once attached — and a deterministic predicate $\textsc{ReadyOK}(M, \mathcal{E})$ over the set $\mathcal{E}$ of annotated proposal echoes counted for the candidate proposal at ready time. A conforming guard is deterministic and monotone: for all annotated-echo sets $\mathcal{E} \subseteq \mathcal{E}'$ with at most one echo per author, $\textsc{ReadyOK}(M, \mathcal{E})$ implies $\textsc{ReadyOK}(M, \mathcal{E}')$. Empty $M$ has no references, passes the gate, carries an empty annotation, and satisfies the guard. Direct AGB otherwise treats $M$ opaquely; its syntax is checked inside $\textsc{Formed}_v(C, T, M)$.

For conditional termination, CoreOK, TipOK, and AuxOK are persistent for a fixed proposal: once true at a correct party, they remain true. A conforming value domain also provides authorization and fair repair: once a correct party authorizes a witnessed manifest or its root, it eventually authorizes and obtains every item in that manifest's deterministic expansion in every fair execution.

Vantage instantiates $\textsc{CoreOK}_i(C)$ as $\textsc{AuthorOK}_i(C)$ and $\textsc{TipOK}_i(C, T)$ as $\textsc{AuthorOK}_i(T) \wedge \textsc{TipAnchored}_i(C, T)$. Under this instantiation, $\mathsf{Witnessed}(X)$ means that every named valid lane prefix was published by its encoded author to a correct permanent holder; Lemmas B.2 and B.3 prove the witness implications. The author-indexed syntax in Appendix B.2 instantiates Formed, and the hash-chain traversal in Section 6 instantiates the expansion contract. That section also defines Vantage's auxiliary hooks for resolution entries. The standalone safety arguments use only this contract and the initial-response and same-proposal refinement rules.

### B.1 Data Dissemination

Payloads are disseminated in data blocks below AGB. An *author lane* is the author-indexed publication structure formed by one party's data blocks. A height-$k$ block of party $p_i$ has the canonical form $b_i^k = (\mathsf{sid}, i, k, h_i^{k-1}, x)$, where $|x| \le \ell_{\max}$ and $h_i^{k-1}$ is the predecessor hash (the session fixes the shared height-0 genesis). Its identifier is

$h_i^k = \mathcal{H}(\texttt{data-block} \parallel \mathrm{enc}(b_i^k))$, abbreviated $\mathcal{H}(b_i^k)$. A *valid lane prefix* is a linear hash chain whose blocks share one author index, have consecutive heights and matching predecessor hashes back to genesis, and satisfy BlockOK. A correct author's valid lane prefixes are mutually comparable; a Byzantine author may publish forked lane prefixes. A correct party creates only valid extensions of its current lane frontier. To publish a new block it sends $\langle \textsc{publish}, b_i^k \rangle$ to all parties. A recipient treats this as author publication only when the authenticated channel sender and the block's encoded author are both $p_i$.

A hash $h = \mathcal{H}(b_i^k)$ names both the block and, through the predecessor-hash chain, its valid lane prefix $b_i^1, \ldots, b_i^k$. A reference $(i, k, h)$ in a well-formed fixed proposal, a local completion or seal, or a consumed control anchor authorizes repair of its root block. After obtaining a block whose hash, session, encoded author, height, and BlockOK check match an authorized reference, the requester also authorizes its predecessor reference; this process continues recursively to genesis. For each authorized missing hash, the party requests the block from all parties. A holder answers with a distinctly typed $\langle \textsc{serve}, h, b \rangle$ repair message. A canonical, correct-session, size-bounded, BlockOK body whose hash is $h$ is cached even if its encoded author or height does not match the currently authorized coordinate; only an exact coordinate match advances the predecessor walk. This prevents a false coordinate from consuming the sole useful response for a later genuine reference. A reply that fails the hash or body checks is ignored and never marks the hash as obtained. No repair message establishes publication provenance, even when its sender equals the encoded author; if the corresponding publish message arrives later, it may upgrade the cached bytes.

A correct party records the first pending request for each requester–hash pair even when it does not yet hold the block, and answers once if it later retains that block. A correct requester sends at most one request for a given authorized hash to each peer, and a correct holder answers at most once for each requester–hash pair. Reliable delivery and retained pending requests make retransmission unnecessary; the rule also bounds correct-party traffic induced by repeated Byzantine requests.

For $h = h_a^k$, let $\textsc{DirectPub}_j(a, k, h)$ mean that $p_j$ holds the valid lane prefix through $h$ and received every block in it through publish messages directly from $p_a$. When this predicate first holds, $p_j$ broadcasts $\langle \textsc{ack}, a, k, h \rangle$. Thus a correct acknowledgment asserts validity, possession, and direct receipt from the encoded author; like every statement below, it is counted only first-hand from its own sender.

*Definition B.1 (Availability).* A lane reference $(a, k, h)$ is *$q$-available* at a party if it has received $\langle \textsc{ack}, a, k, h \rangle$ first-hand from at least $q$ distinct parties.

A valid lane prefix obtained only through repair is held and served but never acknowledged. Hash links prove content, not authorship: anyone can construct a well-linked block carrying another party's index. Authenticated publish receipt establishes authorship locally, while $f{+}1$ author-bound acknowledgments establish that some correct recipient observed such a publication. Consequently, a block that $p_a$ never publishes to a correct party gathers at most $f$ acknowledgments for $(a, k, h)$.

A reference is *locally available* at a party when its valid lane prefix is held by that party or it is $(f{+}1)$-available there; a manifest is locally available when every entry in it is. Define the separate provenance gate

$$\begin{aligned} &\textsc{AuthorOK}_j(a, k, h) \\ \iff\ &\textsc{DirectPub}_j(a, k, h) \\ \vee\ &\big((a, k, h) \text{ is } (f{+}1)\text{-available at } p_j\big). \end{aligned}$$

For a manifest, AuthorOK$_j$ means that the predicate holds for every entry at its encoded author coordinate. AuthorOK implies local availability; the converse is false for a fetched lane prefix. *Retention rule:* a correct party retains every valid lane prefix it acknowledges; every valid lane prefix whose local holding it relies on when a protocol rule checks local availability, AuthorOK, or tip anchoring; and every valid lane prefix it obtains by request for such a check — even when the prompting window has closed by the time the lane prefix arrives. Hash-correct bodies received during an authorized repair are also cached so that a later exact coordinate can consume them. Retained lane prefixes are held and served indefinitely; local sealing or output of the naming views is no discard signal, since another correct party's seal and output may lag arbitrarily. As for control bodies (Section 6.1), checkpoint garbage collection is outside the model. We call a reference *retrievable* when some correct party permanently retains its valid lane prefix. In a fair execution, the repair rule below makes every correct party that authorizes the exact reference eventually obtain it.

Algorithm 2 gives the corresponding event handlers; the prose above is normative.

Lemma B.2 (Availability witnesses). *For every reference $(a, k, h)$:*

(i) *if a correct party relies on it in a local-availability, AuthorOK, or tip-anchoring check, then the reference is retrievable;*

(ii) *in every fair execution, every correct party that authorizes a retrievable reference eventually obtains its valid lane prefix;*

(iii) *if it is $(n-f)$-available at one correct party, then it eventually becomes $(f+1)$-available at every correct party.*

Proof. For (i), if the checking party holds the lane prefix, the retention rule applies; otherwise it counted $f+1$ acknowledgments, including one from a correct party that holds and retains it. For (ii), requests go to every party and a correct holder answers. Starting at $h$, each verified block authorizes its predecessor, so induction on height retrieves the lane prefix through genesis; collision resistance makes each requested hash unambiguous. For (iii), at least $n - 2f \geq f+1$ counted acknowledgments are from correct parties, and their broadcasts eventually reach every correct party. □

Lemma B.3 (Author witness). *If a correct party satisfies AuthorOK$_j(a, k, h)$, then some correct party permanently holds the valid lane prefix ending at $h$ and received every block in that lane prefix through publish messages directly from $p_a$. In particular, every block in the lane prefix satisfies BlockOK and $p_a$ published every such block to a correct party.*

Proof. The claim is immediate in the DirectPub case. Otherwise, among the $f+1$ acknowledgers counted by $p_j$ at least one is correct. A correct party sends that exact author-bound acknowledgment only after its own DirectPub predicate holds, and the retention rule is permanent. □

A count of $n-f$ acknowledgments is therefore local evidence that at least $n - 2f \geq f+1$ correct parties hold the directly published valid lane prefix and, by Lemma B.2(iii), that both the $(f+1)$-availability and AuthorOK conditions used for a core entry below eventually hold at every correct party.

### B.2 Interface

The direct interface has one sender input, two idempotent environment events, one completion event that makes the core irrevocable, and an optional direct-result event:

- propose$(v, C, T, M)$ — invoked once by Proposer$(v)$. A correct Direct AGB caller supplies a value satisfying Formed$_v(C, T, M)$. In Vantage, a correct proposer places in $C$ only references $(a, k, h)$ for which DirectPub$_i(a, k, h)$ holds and it has counted $n-f$ matching acknowledgments, and in $T$ only references for which DirectPub$_i(a, k, h)$ holds. Concretely, it lists in $C$, for each author, the newest such reference with $n-f$ counted acknowledgments, and

**Algorithm 2:** Authenticated data publication and repair at party $p_i$

```
/* A stored block may carry a direct-publication mark, a repair mark, or both; only the first can establish provenance or trigger an
   ACK. */
Global variables and constants:
    maps of cached blocks and direct-publication marks; sets authorized, requested, pendingReq, retained, acked, and answered
upon the local application creates the next valid block b_i^k do
    store b_i^k; process it as a direct publication from p_i; broadcast ⟨PUBLISH, b_i^k⟩
upon a direct ⟨PUBLISH, b⟩ from p_a do
    if b = (sid, a, k, h', x) is canonical, |x| ≤ ℓ_max, and BLOCKOK(b) then
        h ← H(b); store b and mark (a, k, h) directly published by p_a          /* may upgrade repaired bytes */
upon DIRECTPUB_i(a, k, h) first becomes true do
    if (a, k, h) ∉ acked then
        retain the valid lane prefix through h; add (a, k, h) to acked; broadcast ⟨ACK, a, k, h⟩
upon a first-hand ⟨ACK, a, k, h⟩ do
    count the sender once for the exact tuple (a, k, h)
procedure AUTHORIZE(a, k, h)
    add (a, k, h) to authorized if absent
    if a cached b = (sid, a, k, h', x) satisfies H(b) = h, BLOCKOK(b), the size bound, and either k > 1 or h' is the session genesis hash then
        if k > 1 then
            AUTHORIZE(a, k − 1, h')
    else
        foreach p_j ∈ Π with (j, h) ∉ requested do
            add (j, h) to requested; send ⟨REQUEST, h⟩ to p_j
upon ⟨SERVE, h, b⟩ arrives for a requested hash h do
    if b = (sid, a, k, h', x) is canonical, H(b) = h, |x| ≤ ℓ_max, and BLOCKOK(b) then
        cache b as repaired data, without a direct-publication mark          /* coordinate-independent cache */
                                                                  /* a failed hash/body check changes no state */
upon a cached b = (sid, a, k, h', x) matches an authorized (a, k, H(b)) and either k > 1 or h' is the session genesis hash do
    AUTHORIZE(a, k, H(b))                                          /* fires after either PUBLISH or SERVE */
upon an authorized predecessor walk verifies a valid lane prefix through genesis do
    retain every block in that lane prefix; invoke TRYSERVE on their hashes
upon a direct ⟨REQUEST, h⟩ from p_j do
    add (j, h) to pendingReq if (j, h) ∉ answered; TRYSERVE(h)
upon a cached block with hash h becomes retained do
    TRYSERVE(h)
procedure TRYSERVE(h)
    if a retained block b satisfies H(b) = h then
        foreach (j, h) ∈ pendingReq \ answered do
            add (j, h) to answered; remove it from pendingReq; send ⟨SERVE, h, b⟩ to p_j
function AUTHOROK_i(a, k, h):
    return DIRECTPUB_i(a, k, h) or at least f + 1 counted first-hand ⟨ACK, a, k, h⟩ messages
```

in $T$ the newest directly published lane tip whose valid lane prefix strictly contains the author's $C$ entry — an author with no $C$ entry is anchored at genesis, so any directly published tip qualifies — at most one $C$ and one $T$ entry per author. Inclusion is thus deterministic in the proposer's first-hand state. Here *newest* means greatest verified lane height, with the lexicographically smallest hash breaking an equal-height tie. If an author forked its lane, the $C$ entry pins the kept branch below itself (the same height/hash order resolves locally visible forks; a forking author is Byzantine), and a directly published lane tip on any other branch is never included. Authors are processed in a canonical order, skipping any hash already listed under an earlier index, so all entries are globally distinct and no hash appears in both manifests. A correct Vantage proposer also keeps one local quarantine witness per author. Ordinary inclusion does not fill this slot. When the party emits an initial READY-mix for a proposal carrying a tip reference of author $a$ that is not $n-f$-available locally, it stores that reference if the slot is empty. It omits $a$ from

later $T$ manifests until the witness becomes $n-f$-available or a containing prefix becomes its current $C$ candidate, then clears the slot. This caller-side discipline is not part of FORMED and cannot constrain a Byzantine proposer. It never activates on the all-grade-1 common path, and acknowledgment broadcasts eventually clear any slot whose witness its author actually published. A fabricated witness never clears and suppresses only that author's tip entries at this proposer (Section 8). Over-inclusion elsewhere is harmless since sequencing outputs every data block once (Section 6). Each manifest thus has at most $n$ entries. Vantage instantiates FORMED by requiring that each of $C$ and $T$ list at most one entry per author, all hashes across $C$ and $T$ are distinct, every reference has a canonical positive height and hash, and the auxiliary field is either empty, one resolution entry, or a vector of at most $f$ skip entries with strictly increasing, distinct target views $u \le v - 3$. The single entry's payload manifests, if present, each list at most one canonical reference per author. Correct parties never echo a malformed proposal (R2), so the per-author entry bound — and with it the $n$-entry bound — holds for every echoed proposal, Byzantine ones included. These are syntactic checks: when bytes or acknowledgments are available, $\text{AUTHOROK}_i(a, k, h)$ can hold only for a valid lane prefix whose encoded author and height are exactly the reference coordinate $(a, k)$.

- ACTIVATE$(v)$ — permits the fixed proposal's positive echo gate to fire. In Vantage this local event occurs when authenticated receipt advances the contiguous responsive proposal frontier through $v$; formal entry activates separately through ENTER$(v)$.
- ENTER$(v)$ — records the local entry time, activates the fixed proposal if present, and arms the echo and ready fallback deadlines. Vantage generates this local event with the WISH pacemaker of Appendix B.3. Neither environment event is a counted statement or transferable evidence.
- COMPLETE$(v) \to B_v$ — emitted when the first proposal-ready quorum is counted; it fixes $B_v = (C, T, M)$ and makes $C$ an irrevocable prefix of the view's eventual non-skip output. A completed but unsealed view is open-tip$(C, T)$.
- DIRECT-SEAL$(v) \to X$ — the optional direct output, where $X$ is commit-full$(C, T)$ or commit-core$(C)$. It fires on the first homogeneous ready quorum of the corresponding grade. The mixed grade is nonterminal and skip is not a Direct AGB output.

Direct AGB never reads resolver state. Vantage connects its DIRECT-SEAL output to a caller-owned TRY-SEAL$(v, X)$ arbiter. Every direct result, Vantage fast or skip result, and resolver outcome is submitted to it. While empty, the arbiter accepts the first submission, stores $X$, and emits the terminal SEAL$(v) \to X$ event; afterward it retains that outcome and ignores later submissions. Compatibility ensures that two submissions, if both occur, name the same outcome. This output multiplexer belongs to the composition, not to Direct AGB. The lifecycle vocabulary of Section 2 applies throughout: the arbiter is what turns a direct, fast, skip, or resolver result into the terminal *seal* event.

### B.3 Vantage Pacemaker and Activation

The following service is Vantage's concrete implementation of Direct AGB's ENTER and ACTIVATE environment events. It is not used by the asynchronous safety proof of Theorem B.7.

The pacemaker uses first-hand $\langle \text{WISH}, x \rangle$ messages, following the bounded-space high-watermark construction of Bravo, Chockler, and Gotsman [10]. Party $p_i$ stores, for every author $p_j$, the largest directly received wish $\omega_i[j]$. Let $\omega_i^+$ and $\omega_i^Q$ be the $(f+1)$st and $(n-f)$th largest entries, respectively. When $\omega_i^+$ exceeds $p_i$'s own largest wish, it broadcasts a wish for $\omega_i^+$; independently, whenever $\omega_i^Q$ increases it raises its entry target to $\omega_i^Q$. Wishes are high-watermarks, and the two updates are independent: a wish for $x$ supports every view at most $x$, and target advancement never waits for $\omega_i^Q = \omega_i^+$. On a target $x$, the party records entry to every missing view through $x$ immediately and in order.

Initially $\omega_i[j] = 0$ for every $j$, the largest entered view is 0, and the own-wish high-watermark is 0. Genesis enters view 1, after which $p_i$ sets $\omega_i[i] = 2$, records own wish 2, and broadcasts $\langle \textsc{wish}, 2\rangle$ (self-delivery is immediate). Upon receiving $\langle \textsc{wish}, x\rangle$ first-hand from $p_j$, including as a piggybacked high-watermark, $p_i$ sets $\omega_i[j] \leftarrow \max\{\omega_i[j], x\}$ and recomputes both order statistics. It first performs any enabled $f$+1 amplification, updating $\omega_i[i]$ and its own-wish high-watermark when it broadcasts and recomputing both order statistics after that update. It then records entry through any newly supported quorum target. Stale wishes cause no transition.

Write $E_i(u)$ for $p_i$'s unique echo-stage response and $R_i(u)$ for its initial ready-stage response in view $u$. The local trigger for wishing follows Starfish's two-response progress condition [30]: before emitting whichever of $E_i(v-1)$ and $R_i(v-2)$ completes the pair, $p_i$ raises its own wish to at least $v+1$. Every response piggybacks the sender's current own-wish high-watermark outside its counted response identity. An optional READY-mix refinement carries the same field but does not run this progress trigger a second time. Amplification may send a wish standalone. A wish only schedules views; it is never an echo, a ready, an availability acknowledgment, an origin bit, or a resolution justification.

Let $e_i(v)$ be the time $p_i$ enters view $v$. Entry is strictly increasing locally. Recording entry to $v$ arms both fallback schedulers for $v$; positive responses may already have fired through the responsive path, and pending responses may finish after later entries. R2 and R3 close them by $e_i(v) + \theta_E$ and $e_i(v) + \theta_R$, respectively, in post-GST drift-free time, unless their positive gates fire earlier. Thus entry never waits for an older response window.

In parallel, $p_i$ maintains a *responsive proposal frontier* $a_i$, initially 0. Formal entry to view $v$ raises $a_i$ to at least $v-1$. For each view, $p_i$ fixes the first proposal received directly from its designated proposer and ignores later versions. When the fixed proposal for $a_i + 1$ is well formed, it advances $a_i$ by one; buffered consecutive fixed proposals are processed in increasing view order. A proposal that advances the frontier may activate its positive response path before formal entry, but it never causes formal entry and is not evidence for any quorum. Thus WISH remains the only recovery and synchronization mechanism, while a contiguous stream of first-hand proposals supplies a one-hop responsive path. As in the rest of the unbounded-execution model, parties retain future per-view messages; the non-Zeno convention makes the retained set finite at every finite time.

The designated proposer also uses WISH evidence directly: R1 invokes $\textsc{propose}(v, \cdot)$ already when $\omega_i^+$ first supports $v$ at $p_i = \mathsf{Proposer}(v)$, one amplification hop before the entry quorum threshold $\omega_i^Q$. The early proposal is passive — receivers fix it and authorize repair of its references, but no response fires before activation — and it cannot outrun genuine progress. For $v \geq 3$, by induction over raise events, the earliest correct wish supporting $v$ is an own raise of the two-response trigger, by whose local step the sender has emitted its echo-stage response for a view at least $v-2$; an amplification raise to $\omega^+ \geq v$ counts a wish of at least one correct author with a strictly earlier send and inherits the same provenance. Genesis wishes support only views up to 2. Since $\omega_i^+ \geq v$ counts $f$+1 authors, at least one correct, the trigger certifies that progress. Its payoff appears in the fallback analysis: the wish quorum behind the first correct entry to $v$ contains $f$+1 correct wishes supporting $v$, which reach $\mathsf{Proposer}(v)$ within one delay of that entry, so the proposer invokes within $\delta$ of the first entry rather than within the $2\delta$ entry spread (Theorem B.8). A smaller entry spread would require a smaller entry threshold: the $f$+1 correct wishes guaranteed behind the first entry reach every correct party within one delay, but completing a $n-f$ quorum from them costs the amplification hop, since the up to $f$ Byzantine wishes inside the first quorum reach only their chosen recipient. Entering on fewer than $2f$+1 wishes would instead let the adversary drive a single correct party's deadlines arbitrarily far ahead, spending its one-shot refusals. Signed and certified pacemakers escape this dilemma by relaying transferable evidence; the proposer-side trigger recovers the lost delay exactly where the liveness analysis needs it, because proposing early is harmless where entering early is not.

Lemma B.4 (Pacemaker progress and synchronization).

(a) *Every correct party eventually enters every view and emits one echo-stage and one initial ready-stage response for every view; a provisional READY-mix may additionally refine once.*

(b) *There is a finite view $V$ such that, for every $v \geq V$, if the first correct entry to $v$ occurs at time $s_v$, then every correct party enters $v$ by $s_v + 2\delta$.*

(c) *If every party is correct, the same cutoff can be chosen so that every correct party enters each $v \geq V$ by $s_v + \delta$.*

Proof. For (a), all correct parties enter view 1 from genesis; responses for nonpositive views are fixed, and every correct party initially broadcasts $\langle \text{WISH}, 2 \rangle$. Suppose all correct parties eventually enter $v$. The deadlines for views $v-1$ and $v-2$ force every correct party to emit both responses, thereby broadcasting a wish of at least $v+1$. Every correct party therefore receives a wish quorum for $v+1$ and enters every missing view through $v+1$. Induction proves entry to all views; the stage deadlines then prove emission of every response.

For (b), assume the standard non-Zeno execution convention and choose $V$ larger than every view entered, proposed, responded to, or wished for by a correct party before GST; this is a finite prefix. Fix $v \geq V$ and its first correct entry time $s = s_v \geq \text{GST}$. Let $x \geq v$ be the target whose traversal caused that entry. The entering party counted a wish quorum supporting $x$, including at least $f+1$ correct authors. Their wishes reach every correct party by $s + \delta$, causing every correct party to broadcast a wish supporting $x$ if it has not already done so. These wishes arrive everywhere by $s + 2\delta$, making every $\omega_i^Q$ support $x$. Sequential traversal therefore enters $v$ by that time, even if the target is higher.

For (c), every author in the wish quorum counted at time $s_v$ is correct. Those same wishes reach every correct party by $s_v + \delta$, directly giving each of them a wish quorum for the same target. No amplification hop is needed. □

The synchronizer also bounds how long a view can hold every correct party.

Lemma B.5 (Eventual worst-case view duration). *Let $v$ be a view whose $t_1$ — the earliest time at which every correct party has entered view $v$ or higher — satisfies $t_1 \geq \text{GST}$. Then every correct party enters a view greater than $v$ by $t_1 + \theta_R + \delta = t_1 + 4\Delta + \delta$. Vantage's eventual worst-case view duration, in the sense of [42], is therefore $4\Delta + \delta$.*

Proof. Fix a correct party $p_i$. By $t_1$ it has recorded entry, in order, to every view through $v$, so $e_i(v-2) \leq e_i(v-1) \leq e_i(v) \leq t_1$; the entries themselves may precede GST. The fallback rules close its echo-stage response for $v-1$ by $\max\{e_i(v-1), \text{GST}\} + \theta_E$ and its initial ready-stage response for $v-2$ by $\max\{e_i(v-2), \text{GST}\} + \theta_R$: a timer still armed at GST has at most its full duration remaining once clocks are drift-free. Hence the progress pair for $v-1$ completes by $t_1 + \theta_R$; for $v = 2$ the initial ready-stage member of the pair is a fixed genesis response, so the pair completes with the echo-stage emission alone, and for $v = 1$ both members are fixed and the genesis wish already supports 2. Before emitting the completing response, $p_i$ raises its own wish to at least $v+1$ unless its own wish already supports $v+1$; in either case $p_i$ has broadcast a wish supporting $v+1$ by $t_1 + \theta_R$. The correct parties' wishes supporting $v+1$ — at least $n-f$ of them — reach every correct party by $t_1 + \theta_R + \delta$ under the in-flight convention, make $\omega^Q$ support $v+1$ there without an amplification hop, and sequential traversal enters every view through $v+1$. □

### B.4 Protocol

All statements below are broadcast to all parties over authenticated channels. A party sends at most one echo-stage statement and one *initial* ready-stage statement for a view. Echo-stage statements are either proposal echoes or echo-skip and are immutable. The initial ready-stage statement is proposal-ready or no-ready; no-ready and homogeneous

proposal-ready are final, whereas an initial READY-mix may be followed by at most one same-proposal homogeneous refinement. A party counts a statement only after receiving it directly from its author. It counts the first ready-stage statement from an author once toward the stage-response and proposal-ready censuses. If that first statement is READY-mix for $B$, it additionally accepts one later READY-$g$ for the same $B$, $g \in \{0, 1\}$, into the grade-$g$ tally without counting the author again; every other later or conflicting statement is ignored. If the homogeneous statement is the first countable version from that author — whether due to network reordering or because digest-named statements awaited body validation — it is the final initial statement and a later READY-mix is ignored. Where READY-mix was counted first, it remains historical non-grade-1 evidence for resolver candidate selection even after refinement. The sender's own responses are delivered and counted immediately under the self-delivery convention of Section 4.

In the Vantage wrapper, responses for views $v' \leq 0$ are fixed genesis responses treated as received from every party. Genesis enters every correct party into view 1, raises its responsive proposal frontier to 0, and makes it broadcast $\langle \textsc{wish}, 2 \rangle$. Entry, frontier state, and WISH messages are not evidence for a broadcast instance; safety holds for arbitrary entry and timeout timings.

**R1. Propose.** On its first sender input $\textsc{propose}(v, C, T, M)$, party $p_i = \mathsf{Proposer}(v)$ sends $\langle \textsc{propose}, v, C, T, M \rangle$ once. Vantage invokes this input when its responsive frontier first satisfies $a_i \geq v - 1$ or its wish statistic first satisfies $\omega_i^+ \geq v$ (Appendix B.3), and constructs $(C, T, M)$ as in Appendix B.2. In particular, formal entry to $v$ raises that frontier to $v - 1$, so a correct Vantage proposer invokes the input no later than its own entry; receipt of the preceding direct proposal or early wish evidence may make it invoke earlier.

**R2. Echo stage.** Proposal messages may arrive before or after a party enters the proposal's view. The party fixes the first view-$v$ proposal received directly from $\mathsf{Proposer}(v)$ and ignores every later version. It checks $\textsc{Formed}_v(C, T, M)$; a malformed fixed proposal is not echoable. Otherwise the party stores $B_v = (C, T, M)$ and invokes the value domain's authorization and repair hooks for $C$, $T$, and $\textsc{AuxRefs}(M)$. In Vantage, a malformed proposal also cannot advance the responsive frontier, and authorization requests from all parties every missing block of each named lane prefix.

A stored proposal becomes *active* after the local environment invokes $\textsc{activate}(v)$ or $\textsc{enter}(v)$. In Vantage, processing the fixed proposal invokes $\textsc{activate}(v)$ exactly when it advances the responsive frontier to $v$; a buffered proposal can therefore activate when the missing contiguous prefix arrives. From activation onward, while the echo-stage response is pending, the party continuously evaluates the positive gate: $\textsc{CoreOK}_i(C)$, $\textsc{TipOK}_i(C, T)$, and $\textsc{AuxOK}_i(v, M)$ all hold. In Vantage, the tip predicate means $\textsc{AuthorOK}_i$ holds for every entry of $T$ and the *tip-anchoring check* holds: for every author index present in both manifests, the party holds the tip entry's full lane prefix and that lane prefix strictly contains the author's $C$ entry. This is a pure local hash walk, so counted acknowledgments alone never substitute for a paired tip. When the gate first holds, the party broadcasts $\langle \textsc{echo}, v, C, T, M, 1, o \rangle$, even if formal entry has not yet occurred. Concretely, once the preceding data gates hold, Vantage's atomic R2 transition invokes $\textsc{TryMetaOK}$; success sets the persistent $\textsc{AuxOK}$ latch immediately before that ECHO.

Formal entry to $v$ additionally arms the fallback scheduler. If the response is still pending and a well-formed proposal is fixed, let $t$ be the later of entry and its direct receipt. The fallback window is $[t,\ \min\{t+\Delta, e_i(v)+\theta_E\}]$. If no grade-1 echo has fired by its end, the party broadcasts $\langle \textsc{echo}, v, C, T, M, 0, o \rangle$ if $\textsc{CoreOK}_i(C)$ and $\textsc{AuxOK}_i(v, M)$ hold, and $\langle \textsc{echo-skip}, v \rangle$ otherwise. Here too Vantage attempts the metadata latch after the core gate and before the grade-0 ECHO. If no echoable proposal becomes active by the absolute fallback deadline $e_i(v) + \theta_E$, the party

instead broadcasts $\langle \textsc{echo-skip}, v \rangle$. A well-formed proposal delivered after that deadline can no longer be echoed; it is still fixed and its references are authorized for repair, and in Vantage the responsive frontier processes it. In every proposal echo, the field $o = \textsc{Ann}_i(v, M)$ is the auxiliary annotation; it is empty for empty $M$. No later echo-stage statement for $v$ is allowed; the pre-entry and post-entry routes share this single immutable response state.

**R3. Ready stage.** Entry to view $v$ arms its ready-stage refusal deadline. Independently of entry, while the initial response is pending the party continuously evaluates the first-hand echoes it has counted. As soon as a quorum of proposal echoes names a common view proposal $B_v = (C, T, M)$ and the ready guard holds, it broadcasts the proposal-ready statement $\langle \textsc{ready}, v, C, T, M, g \rangle$, where, over all echoes counted at emission, the grade $g$ is 1 if it has counted $n-f$ grade-1 proposal echoes for $B_v$, 0 if it has counted $n-f$ grade-0 proposal echoes for $B_v$, and the mixed grade mix otherwise. Grades 0 and 1 finalize the sender's ready stage. A mixed grade is provisional: while fewer than all $n$ echo-stage responses have been counted and the ready deadline has not fired, the sender continues evaluating echoes for the same $B_v$. If one grade reaches $n-f$, it broadcasts one READY-$g$ refinement for $B_v$ and finalizes. Counting all $n$ echo-stage responses without such a quorum, or reaching $e_i(v) + \theta_R$, instead finalizes the mixed response without another statement; deadline ties process delivered messages first. The refinement is not a second initial response and does not invoke the WISH progress hook. In the Vantage wrapper, before broadcasting an initial READY-mix, the party applies the tip-quarantine update of Appendix B.2. If formal entry has occurred and no initial proposal-ready has fired by $e_i(v) + \theta_R$, it broadcasts the refusal $\langle \textsc{no-ready}, v \rangle$. Proposal-ready is additionally gated on the ready guard: $\textsc{ReadyOK}(M, \mathcal{E})$ must hold over the counted annotated proposal echoes $\mathcal{E}$ for $B_v$. Wishes and entry notifications never satisfy either gate.

**R4. Completion and direct result.** Upon counting a ready quorum for a common $B_v = (C, T, M)$, disregarding grades and counting each author once, record $\textsc{complete}(v) \to B_v$. On the first homogeneous ready quorum, emit $\textsc{direct-seal}(v) \to \mathsf{commit\text{-}full}(C, T)$ for grade 1 or $\textsc{direct-seal}(v) \to \mathsf{commit\text{-}core}(C)$ for grade 0. A completed view with no homogeneous ready quorum remains $\mathsf{open\text{-}tip}(C, T)$ at the Direct AGB boundary. Vantage may close it through the resolver adapter of Section 6. Ready counting continues after completion, so a late-arriving ready that closes a homogeneous quorum produces a direct result. Vantage submits every such result to its caller-owned TRY-SEAL arbiter; a compatible result arriving after a terminal Vantage seal is ignored by that arbiter. Neither completion nor the direct result waits for formal entry. When a homogeneous quorum is the first completing quorum, completion and the corresponding DIRECT-SEAL event occur in the same local step.

Vantage adds the following caller-side rule without changing Direct AGB. For an exact data-only proposal $B = (C, T, \emptyset)$, a *matching response* is a grade-1 proposal echo for exactly $B$; an origin bit, if the rule is later generalized, is not part of this comparison. Immediately before a correct party sends a matching response, it records an optimistic lock $L_i(v, B)$. The lock is *active* exactly while the party has counted fewer than $f+1$ first-hand echo-stage responses for $v$ that are not matching responses for $B$. Thus a lock may be born inactive, and once inactive it never becomes active again. Upon counting matching responses from all $n$ parties, the wrapper emits

$$\textsc{fast-seal}(v) \to \mathsf{commit\text{-}full}(C, T)$$

once and submits that result to TRY-SEAL. It does not fire COMPLETE or DIRECT-SEAL, expose a core separately, or create a completion report; R3 and R4 continue normally. The resolver-side use of $L_i$ is defined in Section 6.

The interface and R1–R4 above define Direct AGB. Algorithm 3 gives Vantage's concrete realization in event form; its frontier, WISH, manifest construction, and TRY-SEAL lines are caller-side wrappers rather than additional Direct AGB requirements. Algorithm 4 separately gives the Vantage-only fast- and skip-seal, lock-release, and TRYMETAOK/accepted-latch hooks so they are not mistaken for Direct AGB rules.

Grades, including a permitted homogeneous refinement, are outside the counted view-proposal identity: mixed grades can never prevent completion. A residual mixed grade leaves only the tip open; a refinement may instead close a later homogeneous quorum. Grade 0 is positive support for a terminal core-only outcome, not merely the absence of grade 1. A ready carries no transferable echo certificate; its support is checked only by its sender.

LEMMA B.6 (RESPONSIVE PROPOSAL PACING). *Suppose every party is correct and $v$ is past the cutoff of Lemma B.4(c), enlarged past clearance of every earlier mixed-tip quarantine. Let $\beta_v$ be the time $B_v$ is sent. Then*

$$\beta_{v+1} \le \max_{u \le v} \beta_u + \delta,$$

*so the least not-yet-proposed view always follows within $\delta$ of the latest send. Call $B_v$* prefix-extending *when every lower view was sent by $\beta_v$. Suppose $B_v$ is prefix-extending, the AUTHOROK, tip-anchoring, and local metadata checks for $B_v$ pass everywhere by $\beta_v + \delta$, and, when $M_v$ carries a* commit-full *or* commit-core *entry, at least $f+1$ correct parties satisfy its origin-bit condition (Section 6) when echoing. Then every correct party obtains the ordinary Direct AGB full result by $\beta_v + 3\delta$; if $M_v = \emptyset$, every correct party also emits and submits the Vantage fast result* commit-full$(C_v, T_v)$ *by $\beta_v + 2\delta$. The send times need not be monotone in the view number: the early wish trigger of R1 may send a proposal before an older one, and receivers buffer it until its contiguous prefix arrives or formal entry activates it.*

PROOF. Because $v$ is past the cutoff, $\beta_v \ge$ GST. Let $z = \max_{u \le v} \beta_u$. In an all-correct execution every view $u \le v$ is proposed, and by $z$ every such $B_u$ has been sent; the in-flight convention delivers all of them everywhere by $z + \delta$, and in-order processing of buffered fixed proposals from genesis raises every correct frontier to $v$ by $z + \delta$ — a proposal delivered after its echo deadline is still fixed and processed by the frontier (R2). Hence Proposer$(v+1)$ sends $B_{v+1}$ by $z + \delta$ unless it already did, proving the first claim.

For a prefix-extending $B_v$, the same delivery argument with $z = \beta_v$ activates $B_v$ at every correct party by $\beta_v + \delta$, so under the stated checks every positive echo gate holds by $\beta_v + \delta$. No fallback response can preempt it: fallback timers exist only after formal entry, and the first correct formal entry satisfies $s_v \ge \beta_v - \delta$, since the all-correct spread of Lemma B.4(c) makes the proposer enter and send by $s_v + \delta$. Party $p_i$'s echo fallback window starts at $t \ge$ its direct receipt of $B_v \ge \beta_v$ and therefore ends at $\min\{t + \Delta,\ e_i(v) + \theta_E\} \ge \beta_v + \delta$, using $e_i(v) + \theta_E \ge \beta_v + 3\Delta - \delta$ from $e_i(v) \ge s_v$; the absolute ECHO-SKIP deadline is the same bound, and the NO-READY deadline $e_i(v) + \theta_R \ge \beta_v + 4\Delta - \delta$ exceeds the ready gate time $\beta_v + 2\delta$ below. Messages delivered by a deadline are processed before its timeout branch, so every echo is grade 1 and fires by $\beta_v + \delta$, with the required origin bits present when $M_v$ carries a non-skip resolution. The echoes arrive everywhere by $\beta_v + 2\delta$. For data-only $B_v$, these are all $n$ exact grade-1 responses, so the Vantage wrapper emits its fast full result then. Independently, positive proposal-ready fires even before formal entry, and those statements arrive everywhere by $\beta_v + 3\delta$, producing the ordinary Direct AGB full result. All proposal echoes in this suffix have grade 1, so no new quarantine is created. Enlarging the finite cutoff as stated is valid because a previously quarantined correct-author witness eventually receives $n-f$ acknowledgments everywhere and clears. □

**Algorithm 3:** Direct Availability-Graded Broadcast as instantiated by Vantage for view $v$ at party $p_i$ (R1–R4; $Q = n - f$)

**Global variables and constants:**
- $fixed_v \leftarrow \perp$; $echo_v \leftarrow$ pending; $ready_v \leftarrow$ pending — /* later provisional($B$) or final */
- $completed_v$, $directed_v \leftarrow \perp$
- $active_v$, $activated_v$, $proposed_v \leftarrow$ false; $quarantine_i[a] \leftarrow \perp$ for every author $a$; $e_i(v), \rho_i(v) \leftarrow \perp$; $\theta_E = 3\Delta$; $\theta_R = 4\Delta$ — /* $\rho_i(v)$ is direct proposal-receipt time */

**upon** $p_i$ = Proposer($v$), *not* $proposed_v$, *and either* $a_i \geq v - 1$ *or* $\omega_i^+ \geq v$ **do**
- $proposed_v \leftarrow$ true; clear $quarantine_i[a]$ when its witnessed prefix is $Q$-available or is contained in the current core prefix for $a$; construct $(C, T)$ by R1: $C$ uses DirectPub plus $Q$ matching author-bound ACKs; $T$ uses DirectPub and omits authors whose quarantine is nonempty
- choose $M$ as in Section 6, or $M \leftarrow \emptyset$; **broadcast** $\langle$PROPOSE, $v, C, T, M\rangle$ — /* R1 */

**upon** *local* ENTER($v$) *or* ACTIVATE($v$) *from the Vantage pacemaker/frontier wrapper* **do**
- **if** *this is* ENTER($v$) *and* $e_i(v) = \perp$ **then**
  - $e_i(v) \leftarrow$ now; $activated_v \leftarrow$ true; raise $a_i$ to at least $v - 1$ and process buffered fixed proposals in view order; arm deadlines $e_i(v) + \theta_E$ and $e_i(v) + \theta_R$
- **if** *this is* ACTIVATE($v$) **then**
  - $activated_v \leftarrow$ true
- **if** $activated_v$ *and* $fixed_v = (C, T, M)$ **then**
  - $active_v \leftarrow$ true

**upon** *the first direct* $\langle$PROPOSE, $v, C, T, M\rangle$ *from* Proposer($v$) *while* $fixed_v = \perp$ **do**
- $\rho_i(v) \leftarrow$ now
- **if** $(C, T, M)$ *is malformed* **then**
  - $fixed_v \leftarrow$ reject — /* later proposer versions remain ignored */
- **else**
  - $fixed_v \leftarrow (C, T, M)$; store $B_v$; invoke AUTHORIZE for each lane prefix named by $C$, $T$, or AuxRefs($M$) — /* typed repair; never provenance */
  - process the proposal in the responsive frontier; **if** $e_i(v) \neq \perp$ *or* $a_i \geq v$ *after processing* **then**
    - $activated_v \leftarrow$ true
  - **if** $activated_v$ **then** $active_v \leftarrow$ true

**upon** $echo_v$ *is pending and* $active_v$*, evaluated whenever local state changes* **do**
- let $(C, T, M) \leftarrow fixed_v$
- **if** CoreOK$_i(C)$, TipOK$_i(C, T)$, *and* TryMetaOK$_i(v, M)$ **then**
  - $echo_v \leftarrow$ sent — /* persist stances before the carrier ECHO */
  - $o \leftarrow$ Ann$_i(v, M)$: the origin bit for a non-skip resolution, else empty; RESPOND($\langle$ECHO, $v, C, T, M, 1, o\rangle$) — /* R2 positive path */

**upon** $e_i(v) \neq \perp$, $d_i^E(v) = \min\{\max\{e_i(v), \rho_i(v)\} + \Delta, e_i(v) + \theta_E\}$, $echo_v$ *is pending, and* $fixed_v = (C, T, M)$ **do**
- $echo_v \leftarrow$ sent; $o \leftarrow$ Ann$_i(v, M)$: the origin bit for a non-skip resolution, else empty
- **if** CoreOK$_i(C)$ *and* TryMetaOK$_i(v, M)$ **then**
  - RESPOND($\langle$ECHO, $v, C, T, M, 0, o\rangle$)
- **else**
  - RESPOND($\langle$ECHO-SKIP, $v\rangle$)

**upon** $e_i(v) \neq \perp$, $e_i(v) + \theta_E$, $echo_v$ *is pending, and no active well-formed fixed proposal exists* **do**
- $echo_v \leftarrow$ sent; RESPOND($\langle$ECHO-SKIP, $v\rangle$) — /* no active echoable proposal */

**upon** *the counted echo set changes and* $ready_v$ *is pending* **do**
- **if** *some* $B = (C, T, M)$ *has* $Q$ *proposal echoes and the ready guard* READYOK *holds:* $M$ *carries no non-skip resolution or at least* $f + 1$ *counted proposal echoes* for $B$ *carry origin value* 1 **then**
  - $g \leftarrow 1$ if $Q$ counted echoes for $B$ have grade 1; 0 if $Q$ have grade 0; otherwise mix
  - $ready_v \leftarrow$ provisional($B$) if $g =$ mix, else final; **if** $g =$ mix **then** store each non-$Q$-available $(a, k, h) \in T$ in empty $quarantine_i[a]$ slots
  - RESPOND($\langle$READY, $v, C, T, M, g\rangle$) — /* R3 initial; no entry, fixed-proposal, or own-echo guard */

**upon** *the counted echo set changes and* $ready_v = provisional(B)$ **do**
- **if** $Q$ *counted echoes for* $B$ *have one grade* $g \in \{0, 1\}$ **then**
  - $ready_v \leftarrow$ final; BROADCAST($\langle$READY, $v, C, T, M, g\rangle$) — /* same-value homogeneous refinement; no second WISH hook */
- **if** $ready_v$ *is still provisional and all* $n$ *echo-stage responses are counted* **then** $ready_v \leftarrow$ final

**upon** $e_i(v) \neq \perp$ *and* $e_i(v) + \theta_R$ **do**
- **if** $ready_v = provisional(B)$ **then**
  - $ready_v \leftarrow$ final — /* finalize MIX locally; no second statement */
- **if** $ready_v = pending$ **then**
  - $ready_v \leftarrow$ final; RESPOND($\langle$NO-READY, $v\rangle$)

**upon** *the counted ready set changes* **do**
- count each author once; a same-$B$ refinement after READY-mix changes only that author's grade tally
- **foreach** $B = (C, T, M)$ *named by counted proposal-ready statements* **do**
  - **if** $completed_v = \perp$ *and at least* $Q$ *name* $B$ **then**
    - $completed_v \leftarrow B$; store $B$; authorize lane prefixes named by $C$ or AuxRefs($M$); fire COMPLETE($v$) $\rightarrow B$ — /* R4; makes the core irrevocable; does not run the cursor */
  - **if** $directed_v = \perp$ *and at least* $Q$ *grade-*1 *readies name* $B$ **then**
    - $directed_v \leftarrow$ commit-full($C, T$); fire DIRECT-SEAL($v$) $\rightarrow$ commit-full($C, T$) and submit it to caller-side TRYSEAL
  - **else if** $directed_v = \perp$ *and at least* $Q$ *grade-*0 *readies name* $B$ **then**
    - $directed_v \leftarrow$ commit-core($C$); fire DIRECT-SEAL($v$) $\rightarrow$ commit-core($C$) and submit it to caller-side TRYSEAL

**function** RESPOND($m$):
- apply Appendix B.3's WISH hook and piggyback own-WISH outside $m$'s identity; **broadcast** $m$

**Algorithm 4:** Vantage's fast- and skip-seal wrappers and resolution stances at $p_i$

```
/* Hooks into Direct AGB; Q = n − f; count only the first direct response per author; exact fast matching ignores any origin bit */
Global variables and constants:
    fastLock_v ← ⊥; fastReleased_v, fasted_v ← false; auxAccepted_v ← false
    for every target u: stance_u ← free and skipSent_u, skipSubmitted_u ← false
upon about to emit grade-1 ECHO for B = (C, T, ∅) in view v do
    fastLock_v ← (C, T)
    fastReleased_v ← true iff already f + 1 counted echo-stage responses are not exact grade-1 echoes for B
    emit the Direct AGB ECHO                                   /* the lock is recorded first */
upon the counted view-v echo set changes do
    if fastLock_v = (C, T) and f + 1 counted responses are not exact grade-1 echoes for (C, T, ∅) then
        fastReleased_v ← true                                  /* perform before R3 on this update */
    if not fasted_v and all n responses are exact grade-1 echoes for some (C, T, ∅) then
        fasted_v ← true; fire FAST-SEAL(v) → commit-full(C, T); submit to TrySeal   /* no completion or report */
upon R_i(u) = NO-READY, at least Q counted target-view responses are ECHO-SKIP, stance_u = free, the known terminal outcome for u is absent or
 skip, and not skipSent_u do
    process the lock-release rule above before this event
    stance_u ← skip-voted; skipSent_u ← true
    broadcast ⟨SKIP-VOTE, u⟩                                  /* persist the stance first; no f + 1 amplification */
upon at least Q direct ⟨SKIP-VOTE, u⟩ statements are counted and not skipSubmitted_u do
    skipSubmitted_u ← true; fire SKIP-SEAL(u) → skip; submit to TrySeal   /* no completion or report */
function TryMetaOK_i(w, M):
    if M = ∅ then return true
    if auxAccepted_w then return true
    foreach well-formed entry in M resolving target u do
        if either E_i(u) or R_i(u) is pending then
            return false
        if the entry is non-skip and either stance_u = skip-voted or the known terminal outcome for u is skip then
            return false
        if fastLock_u = (C, T), ¬fastReleased_u, and the entry is not RESOLVE(u, commit-full, C, T) then
            return false
        if the entry fails its outcome-specific response, data, or tip checks of Section 6 then
            return false
    atomically set the free non-skip target stance in M, if any, to non-skip and auxAccepted_w ← true
    return true                                                /* R2 emits the carrier ECHO in the same event */
```

### B.5 Guarantees

Every quorum below is first-hand and counts distinct authors. Manifests travel by value; replacing them by hashes is an optimization outside these proofs. Every decisive threshold is a quorum or $f+1$, and WISH never contributes to one. R2 treats the eligibility and auxiliary hooks as opaque local predicates; only their witness contract and the composition results depend on their content.

Every correct proposal-ready sender counted a proposal-echo quorum for the named proposal; a correct grade-$g$ sender, $g \in \{0, 1\}$, counted a grade-$g$ echo quorum. Hence any two correct proposal-readies name the same proposal, and correct grade-1 and grade-0 proposal-readies cannot coexist: their grounding echo quorums would intersect in a correct immutable echoer.

The direct safety theorem below treats the eligibility and auxiliary hooks opaquely. It concerns only completions, the initial-response and refinement discipline, and direct R4 seals. Cross-party validity and agreement of resolver-supplied terminal seals are separate composition properties proved in Lemmas D.6 and D.7 and theorem C.1.

THEOREM B.7 (SAFETY). *In every execution, regardless of timing:*

(a) (Local uniqueness and sender integrity) *Each correct party completes $v$ at most once and emits at most one* DIRECT-SEAL *result. Every completion names a proposal received directly from* Proposer$(v)$ *by a correct echoer. If* Proposer$(v)$ *is*

*correct and proposes $B_v = (C, T, M)$, every completion names that proposal and every direct result for view $v$ names the payload $(C, T)$.*

(b) (Value agreement) *All correct proposal-ready statements, proposal-ready quorums, and completions for $v$ derive from one view proposal $B_v = (C, T, M)$ and therefore from unique payload manifests $(C, T)$.*

(c) (Grade compatibility) *A grade-*1 *proposal-ready quorum and a grade-*0 *proposal-ready quorum cannot both exist. Hence correct parties never emit conflicting direct outcomes* commit-full *and* commit-core*; either may coexist with the same* open-tip$(C, T)$*. Parties that have not counted a completion quorum may still remain unknown.*

(d) (Eligibility witness) *Completion implies* Witnessed$(C)$*; a grade-*1 *proposal-ready quorum also implies* Witnessed$(T)$*. In Vantage these properties mean that every named valid lane prefix is author-backed and retrievable.*

(e) (Ready–refusal exclusion) *A proposal-ready quorum and a no-ready quorum for view $v$ cannot both exist.*

Proof. *(a)* A correct party acts only on its first completion and first direct result. Every triggering proposal-ready quorum contains a correct sender, whose grounding echo quorum contains a correct echo. That echo was sent only for a proposal received directly from Proposer$(v)$, proving sender integrity. Against a correct proposer it names the sender's unique $B_v$.

*(b)* The grounding fact gives one proposal for all correct proposal-readies. Every proposal-ready quorum contains a correct sender, so all such quorums and the completions they trigger use that proposal.

*(c)* Quorums of grade-1 and grade-0 proposal-readies would contain correct senders of both grades, which the grounding fact excludes.

*(d)* A completing quorum contains a correct ready sender, whose grounding echo quorum contains a correct party that passed CoreOK; the eligibility contract gives Witnessed$(C)$. For a grade-1 ready quorum, choose a correct grade-1 sender and apply the same argument to its grade-1 echo quorum and TipOK. The concrete Vantage consequence follows from Lemmas B.2 and B.3.

*(e)* The two quorums count $n-f$ distinct initial ready-stage authors each, so they share at least one correct party. That party's initial choice cannot be both proposal-ready and no-ready; a READY-mix refinement does not add another author to either quorum. □

*Graded Complete–Adopt relation.* Reading a correct party's initial proposal-ready response as adopt$(v, B, g)$, $g \in \{0, \text{mix}, 1\}$, with a possible monotone update from mix to a homogeneous grade, and reading its no-ready response as no-adopt$(v)$ gives a graded analogue of BBCA's Complete–Adopt interface [26], directly from Theorem B.7 and the grounding argument above: all correct adopters name one proposal, and grade-1 and grade-0 adopters cannot coexist; a completion implies at least $f+1$ correct adopters of its proposal, while a quorum of no-adopt responses excludes every completion; and each correct adopter's grounding echo quorum establishes Witnessed$(C)$ through the eligibility contract, a grade-1 adopter's likewise Witnessed$(T)$. These are names for already emitted responses, not a probe operation, and no totality is added: without the Vantage resolver, some correct parties may never complete or obtain a direct result for a Byzantine-proposer instance.

Theorem B.8 (Conditional termination). *Let $s \geq$ GST be the first correct enter$(v)$ event, assume every correct party enters $v$ by $s + 2\delta$, and assume the view proposal is broadcast no earlier than GST. Suppose* Proposer$(v)$ *is correct and invokes propose$(v, C, T, M)$ with Formed$_v(C, T, M)$ no later than time $s + \delta$. Assume that at every correct party the conjunction CoreOK$_i(C) \wedge$ AuxOK$_i(v, M)$ holds at some instant no later than its echo-window closure; persistence then keeps it true through the fallback action. Also assume that the annotated proposal echoes of the correct parties alone*

*satisfy the ready guard* ReadyOK$(M, \cdot)$*; monotonicity extends this to every counted superset. Then every correct party fires* complete$(v)$ *by* $s + 4\delta + \Delta \le s + 5\Delta$*, returning* $B_v$*.*

Proof. By premise, all correct parties formally enter by $s + 2\delta$ and the sender invokes its well-formed input by $s + \delta$. No correct receiver rejects it, so the proposal is received everywhere by $s + 2\delta$ and, since formal entry activates any fixed proposal, active everywhere by $s + 2\delta$. For each correct party $p_i$, both $e_i(v) \le s + 2\delta$ and its direct proposal receipt occur by $s + 2\delta$. Thus the fallback-window start $t$ is at most $s + 2\delta$, while $e_i(v) + \theta_E \ge s + 3\Delta \ge t + \Delta$; its window therefore ends at $t + \Delta$, at least $\Delta$ after direct receipt. The assumed core and auxiliary checks pass within every echo window, so every correct party sends a proposal echo by $s + 2\delta + \Delta$. This is no later than the earliest absolute echo deadline $s + 3\Delta$; R2's absolute echo-skip branch requires that no echoable proposal be active by that deadline, so under the checks premise the window branch emits a proposal echo instead.

The annotated echoes arrive everywhere by $s + 3\delta + \Delta \le s + 4\Delta$, no later than the earliest ready deadline $s + 4\Delta$; messages delivered by a deadline are processed before its timeout branch, and the guard premise makes the ready gate pass at every correct party. Every correct party therefore sends proposal-ready, and those statements arrive everywhere by $s + 4\delta + \Delta$, proving completion. □

For every view beyond the cutoff of Lemma B.4(b), Vantage's WISH service supplies the entry-spread premise of Theorem B.8, and the cutoff's definition excludes a pre-GST correct proposal for that view. The R1 construction supplies a formed value, and its early wish trigger supplies the invocation premise: the wish quorum counted at the first correct entry contains $f$+1 correct wishes supporting $v$, sent by $s$ and delivered to Proposer$(v)$ by $s + \delta$ under the in-flight convention, so $\omega^+$ supports $v$ there and the proposer invokes by $s + \delta$ unless it already has. For the Vantage eligibility hooks, CoreOK passes within every echo window by a bounded-delivery argument: the sender counted $n-f$ acknowledgments per core entry before the send time $\beta \ge$ GST, so at least $f$+1 correct acknowledgments among them reach every correct party by $\beta + \delta$ under the in-flight convention, establishing $(f+1)$-availability there. Every correct echo window closes no earlier: its fallback end $\min\{t + \Delta,\ e_i(v) + \theta_E\}$ satisfies $t \ge \beta$ from direct receipt and $e_i(v) + \theta_E \ge s + 3\Delta \ge \beta + \delta$ from $\beta \le s + \delta$, and deadline ties favor delivered messages. The theorem otherwise depends only on Direct AGB's environment events, message rules, and the stated auxiliary-field premises.

For a data-only view with a correct proposer beyond this cutoff, none of these premises requires TipOK. If a correct party cannot verify the tip, R2 emits a grade-0 proposal echo once its fallback window closes, while R3 and R4 count proposal echoes and readies regardless of grade. Thus every correct party still completes by $s + 4\delta + \Delta$, regardless of which correct parties fail which tip checks. Completion fixes the witnessed core, and once the output cursor reaches the view and obtains its lane prefixes, it emits $\mathrm{Core}_D(C_v)$ without waiting for the tip; only the tip may remain open (Theorem B.7 and section 6).

Corollary B.9 (Transaction latency). *Suppose every party is correct, each party places arriving transactions in a new valid data block and publishes it at once. Suppose the execution is in a non-Zeno responsive suffix beginning at a view* $r$ *past the cutoff of Lemma B.4(c): every view below* $r$ *has already been proposed, and sealed and fully output at every correct party, by* $\beta_r$*, and every proposal* $B_v$*,* $v \ge r$*, is data-only (*$M_v = \emptyset$*). Let* $\beta_v$ *be the send time of* $B_v$*. A transaction arriving at time* $t \ge \beta_r$ *is sequenced within*

$$L_{\mathrm{tip}} \le \delta + \delta + 2\delta = 4\delta,$$

*and, ignoring tips,*

$$L_{\mathrm{core}} \le 2\delta + \delta + 2\delta = 5\delta.$$

*If the next prefix-extending proposal opportunity is sent at time $t+\delta$ after processing the new block, the tip path is aligned and sequences it by $t+3\delta$. If a prefix-extending opportunity is instead sent at $t+2\delta$ after processing the block's acknowledgment quorum, the core path is aligned and sequences it by $t+4\delta$. Over any finite union of complete responsive-opportunity intervals, if the eligibility time is wall-clock-uniform and independent of the fixed proposal schedule, the expected wait for the next prefix-extending opportunity is at most $\delta/2$. Under this additional convention,*

$$\mathbf{E}[L_{\text{tip}}] \le \delta + \frac{\delta}{2} + 2\delta = 3.5\delta, \qquad \mathbf{E}[L_{\text{core}}] \le 2\delta + \frac{\delta}{2} + 2\delta = 4.5\delta.$$

Proof. We first factor out the proposal wait. Fix either path and let $x \ge \beta_r$ be a time by which every proposer satisfies DirectPub for a correct-author reference covering the block; on the core path, require in addition that every proposer has counted $n-f$ acknowledgments for that reference. Every view is eventually proposed by Lemma B.4(a) and R1, while non-Zeno execution permits only finitely many proposal sends by $x$. Hence there is a least $k \ge r$ with $\beta_k > x$. Since $x \ge \beta_r$, we have $k > r$. The suffix premise bounds $\beta_u \le \beta_r \le x$ for $u < r$, minimality bounds $\beta_u \le x$ for $r \le u < k$, and Lemma B.6 gives

$$x < \beta_k \le \max_{u<k} \beta_u + \delta \le x + \delta.$$

The inclusion rule makes $B_k$ cover the block. On the core path its newest quorum-available core does so; on the tip path, either the core already covers it or the newest directly published lane tip does. The author's lane is unforked.

Every data-only proposal in the all-correct suffix passes its AuthorOK and tip-anchoring checks everywhere within one delay of its send: every named lane prefix was directly published by its correct author before the send and reaches every party within one further delay. $B_k$ is prefix-extending by the send bounds above, so Lemma B.6 fast-seals it commit-full within $2\delta$ of its send; thus $B_k$ seals by $x+3\delta$. Every lower view was sent by $x$ — below $r$ by the suffix premise, in $[r,k)$ by those bounds — so, by the delivery argument of that lemma, every proposal $B_u$ with $u < k$ is fixed and, through in-order frontier processing, active at every correct party by $x+\delta$, without waiting for formal entry. For a suffix view $u \in [r,k)$ the checks pass by $\beta_u + \delta \le x + \delta$, so every correct party's grade-1 gate holds by $x+\delta$. No fallback response preempts it: a fallback window exists only after entry, starts at $t = \max\{e_i(u), \rho_i(u)\}$, and ends no earlier than the gate time $\max\{t, \beta_u + \delta\}$ on both arms. Indeed, $t + \Delta$ dominates both $t$ and $\beta_u + \Delta$ from $\rho_i(u) \ge \beta_u$; and $e_i(u) + \theta_E \ge \beta_u + 3\Delta - \delta$ dominates $\beta_u + \delta$ as well as $t \le \max\{e_i(u),\ \beta_u + \delta\}$, using $e_i(u) \ge s_u \ge \beta_u - \delta$ — valid for every suffix view, since a correct proposer invokes no later than its own entry and the all-correct entry spread past the cutoff is $\delta$ — and $\rho_i(u) \le \beta_u + \delta$. Hence all $n$ exact grade-1 echoes for each suffix view in $[r,k)$ are sent by $x+\delta$, counted by $x+2\delta$, and fast-seal it. The drained earlier prefix therefore makes the block sequenced by $x+3\delta$.

For the tip path, direct publication makes the block eligible at $x = t+\delta$, giving $4\delta$. If a prefix-extending proposal is sent exactly at that time after processing the block, it covers the block and all lower views have already been proposed, so sequencing occurs by $t+3\delta$. For the core path, every party acknowledges by $t+\delta$ and every proposer counts the acknowledgment quorum by $x = t+2\delta$, giving $5\delta$. If the next prefix-extending proposal is sent exactly at that time after processing the quorum, the same argument gives the aligned $t+4\delta$ endpoint.

For the expected bounds, let $S_j$ be the chronological proposal-send events that fill the least not-yet-proposed numerical view and hence extend the fully proposed prefix. Already-sent higher-view proposals may enlarge the resulting jump, but are not fresh opportunities. If the prefix reaches view $q$ at $S_j$, then the responsive-pacing rule gives $\beta_{q+1} \le \max_{u<q+1} \beta_u + \delta \le S_j + \delta$; hence the next fresh event satisfies $g_j = S_{j+1} - S_j \le \delta$. Conditional on the fixed

schedule, a wall-clock-uniform eligibility time over any finite union of complete intervals $[S_j, S_{j+1})$ therefore gives

$$\mathbf{E}[W] = \frac{\sum_j g_j^2}{2\sum_j g_j} \le \frac{\delta}{2}.$$

Adding the respective admission delay and the $2\delta$ fast-sealing time gives the two expected bounds. Any limiting time average, when it exists, inherits the same inequality. □

*Communication.* Per instance, Direct AGB exchanges $O(n^2)$ messages: one proposal broadcast ($n$ messages), one echo-stage and one initial ready-stage broadcast per party ($n^2$ messages each), and at most one homogeneous READY refinement per party after READY-mix (at most another $n^2$ messages). Thus the explicit worst-case count is $3n^2 + O(n)$, while the all-grade-1 common path has no refinement and retains $2n^2 + O(n)$. ECHO-SKIP and NO-READY replace initial statements and never add any. In Vantage, responses piggyback the sender's current WISH; threshold amplification adds at most one own WISH broadcast per raised view, within the same $O(n^2)$ per-view message bound. Fast sealing only reads the existing ECHO set and adds no message; its lock is one payload identifier plus the already maintained first-response counts. Mixed-tip quarantine adds one reference slot per author and no message. The grounded skip shortcut adds at most one short SKIP-VOTE broadcast per party on an eligible failed view, another $O(n^2)$ messages in that view and none on the all-correct favorable path. Its per-target stance is one persistent trit. Thus Vantage's complete per-view statement layer remains $O(n^2)$ messages. In Vantage's proved by-value instantiation, manifests are carried by value and well-formedness bounds each at $n$ entries — for Byzantine proposals too, since correct parties never echo a malformed proposal — so every statement is $O(n\lambda)$ bits and a view costs $O(n^3\lambda)$ bits.[1] The default implementation names a single proposal by hash in later statements and puts an $O(n)$-bit availability bitmap on each ECHO. Its aligned common case is therefore $O(n^2\lambda + n^3)$ bits per view; sparse digest-bearing exceptions or by-value batch recovery restore the $O(n^3\lambda)$ worst case. Appendix F.3 gives the accounting. This wire encoding is not part of the by-value protocol proved here. A single general Vantage resolution entry adds at most two payload manifests; a skip-only batch adds at most $f = O(n)$ scalar entries and no manifest. Both remain within the same statement bound. Data dissemination is accounted separately: a data block of $\ell_b \le \ell_{\max}$ payload bits occupies $\ell_b + O(\lambda)$ bits on the wire and costs $O(n(\ell_b + \lambda))$ for its author's typed publication and $O(n^2\lambda)$ for the author-bound acknowledgment broadcasts, so blocks of size $\ell_b = \Omega(n\lambda)$ cost $O(n)$ bits of communication per payload bit — the cost of $n$-fold replication itself. Repair is off the common path and absent with correct authors: a repairing party spends $O(n\lambda)$ on requests per distinct authorized missing block, including ancestors discovered while walking a named root, and, under the deduplicated typed-repair response rule, may receive up to $O(n(\ell_b + \lambda))$ per block — $O(n^2(\ell_b + \lambda))$ across all parties in the worst case. Repairing a lane prefix sums these per-block costs over its missing ancestors. Coded dissemination would reduce this and is orthogonal.

The control plane of Section 6.1 is accounted separately. For a resolution-bearing completed proposal, at most one completion report per party costs $O(n^2)$ short messages in total. Each non-speculative Simple-IT control round, including its validated Bracha broadcast, commit messages, and timeout notification, costs $O(n^2)$ messages. ECHO, READY, and the log carry only the short control proposal; for a nonempty value, INIT also sends the $O(n\lambda)$ manifest $B_w$ once to each recipient. A correct-leader control round therefore costs $O(n^2\lambda)$ bits. After a Byzantine-leader round, body repair remains $O(n^2)$ messages but can cost $O(n^2|B_w|) = O(n^3\lambda)$ correct-party bits in the worst case because

[1] Scalar fields — view numbers, heights, lengths — cost $O(\log \nu)$ bits, where $\nu$ bounds those quantities in the execution prefix. The displayed bounds use the deployment regime $\log \nu = O(\lambda)$; otherwise each $O(\lambda)$ scalar term reads $O(\lambda + \log \nu)$. This accounting convention does not bound the unbounded-execution liveness model.

every correct holder may answer every missing consumer. Thus the control plane preserves the quadratic message asymptotic, but adds a real background constant and may serialize queued fallback anchors. It is absent from the logical dependency chain of direct completion and sealing, not from network resource usage.

## C RESOLUTION AND OUTPUT: FULL SPECIFICATION AND PROOFS

*Resolver adapter.* Resolution is outside Direct AGB. Contiguous control-log processing invokes APPLY-ANCHOR$(v, X)$ only at the first applicable anchor $A_v$ of Section 6.1. Its echo-accepted entry determines $X$, which is commit-full$(C, T)$, commit-core$(C)$, or skip; a non-skip call also supplies the backing $(C, T)$. The adapter submits $X$ to the caller-owned TRY-SEAL arbiter, stores and authorizes the accepted manifests, and fires SEAL$(v) \to X$. The call may precede local completion; whichever compatible fast, skip, direct, or anchor submission arrives first wins, and later submissions are idempotent. These compatibility properties are proved in Lemmas D.4 to D.7 and theorem C.1; arbitrary calls are outside the Vantage execution.

Throughout this section, $u$ is the view being sealed and $w \geq u + 3$ a carrying view. Once a proposer has a quorum of initial ready-stage responses for $u$, its view-$w$ proposal may include a resolution entry for $u$ in its bounded metadata vector; if that proposal is not anchored, a later carrying view inherits the obligation. The initial ready-stage response quorum is a common prerequisite for every outcome; the outcome-specific evidence below is additional.

Each correct party maintains a persistent next-turn bit, initially *data-only*. At each of its proposer turns it scans views $u \leq w - 3$ in increasing order, skipping views already sealed or resolved by an observed anchor and also skipping a view for which it does not yet justify any outcome. Thus an older no-evidence view does not prevent an attempt for a later target. If no target qualifies, it sends a data-only proposal ($M_w = \emptyset$) and leaves the bit unchanged. If a target qualifies, the bit selects a data-only or recovery proposal and then flips; a recovery proposal targets the first qualifying view and may use the skip-only batch specified in Appendix C.1. The alternation bit and the per-target candidate pointer below persist across proposer turns. Thus recovery metadata can delay neither every correct proposal nor correct-author data indefinitely, while every fixed oldest qualifying target still receives infinitely many attempts. Resolving the oldest unsealed view, once it qualifies, unblocks sequencing; the prescribed recurring single-target attempt keeps distinct candidate cycles from constraining one another (Theorems D.9 and E.1). Figure 4 contrasts the grounded crash shortcut with the retained anchor fallback.

A resolution entry in $M_w$ is one of

$$\text{RESOLVE}(u, \text{commit-full}, C_u, T_u),$$

$$\text{RESOLVE}(u, \text{commit-core}, C_u, T_u),$$

$$\text{RESOLVE}(u, \text{skip}).$$

Because $M_w$ is part of the carrying proposal, its echo and ready statements agree on the entry.

These entries instantiate Direct AGB's auxiliary hooks. Vantage's FORMED accepts as metadata the empty field, one entry of the displayed form, or the skip-only bounded vector defined below, always targeting distinct $u \leq w - 3$ and with syntactically bounded manifests in the single general entry; AUXREFS$(M)$ contains the manifests named by that non-skip entry; and AUXOK$_i(w, M)$ is the persistent acceptance latch set by the response-based TRYMETAOK transition below. The annotation is the origin bit defined after that transition, and the ready guard requires, for the single full or core entry, at least $f+1$ counted proposal echoes *for the carrying proposal* that carry origin value 1; for skip-only metadata or an empty field it always passes. The guard is deterministic, and it is monotone in the required subset sense: a lower-bound count over annotated echoes never decreases when the set grows.

For a view $u$ whose outcome is not already directly final, a correct proposer first counts the common initial ready-stage response quorum and then uses only first-hand evidence to choose the resolution it carries. For payload $P = (C, T)$, write $R_u^{\text{full}}(P) = \textsc{resolve}(u, \text{commit-full}, C, T)$, $R_u^{\text{core}}(P) = \textsc{resolve}(u, \text{commit-core}, C, T)$, and $R_u^{\text{skip}} = \textsc{resolve}(u, \text{skip})$. These symbols denote carried entries, not their resulting terminal seals. With $P_u = (C_u, T_u)$, the justification rules are:

- $R_u^{\text{full}}(P_u)$ is justified by $f$+1 grade-1 echoes for view-$u$ proposals whose payload is $(C_u, T_u)$.
- $R_u^{\text{core}}(P_u)$ is justified by some $n-f$ counted initial ready-stage statements for $u$ of which none was initially a grade-1 proposal-ready — a counted READY-mix remains historical non-grade-1 evidence after refinement, and the proposer may select this sub-quorum from a larger counted set — together with $f$+1 echoes for payload $(C_u, T_u)$ to fix the payload.
- $R_u^{\text{skip}}$ is justified by a quorum of NO-READY responses for $u$.

These conditions guide a correct proposer; they are not transferable certificates. In particular, partial commit-full or commit-core evidence is not a lock against skip: skip is excluded by a proposal-ready quorum, but it is compatible with $f$+1 echoes or isolated proposal-ready statements. Safety is enforced by the echo rule for the view proposal that carries the resolution. Byzantine parties can pad justification for a candidate that correct parties will refuse, so no fixed choice among justified candidates is live. At one correct proposer, a justified $R_u^{\text{full}}$ or $R_u^{\text{core}}$ candidate names a payload with $f$+1 counted echoes; each author is counted once in that local count and the payloads' author sets are disjoint, so at most $\lfloor n/(f{+}1) \rfloor$ payloads appear in its justified set. The justified candidates for a target therefore form a subset of

$$\{\, R_u^{\text{full}}(P),\ R_u^{\text{core}}(P) : P \text{ a justified payload} \,\} \cup \{R_u^{\text{skip}}\},$$

so there are at most $2\lfloor n/(f{+}1) \rfloor + 1$. The canonical order sorts the distinct payloads $P = (C, T)$ lexicographically by $\text{enc}(C, T)$, lists $R_u^{\text{full}}(P)$ before $R_u^{\text{core}}(P)$ for each payload, and places $R_u^{\text{skip}}$ last. The proposer therefore keeps, per target, a persistent local pointer naming a candidate — not a list position, so growth of the justified set never skips one. It is unset while the justified set is empty, initializes to the first candidate in this canonical order, and advances to the cyclically next candidate of the current canonical justified list after every recovery attempt for that target. Justification is monotone — counted echoes, initial ready-stage responses, historical non-grade-1 marks, and homogeneous refinements only accumulate — so the candidate set stabilizes and every persistently justified candidate is carried infinitely often (Theorem D.9).

Vantage also maintains a persistent per-target *resolution stance* $z_i(u) \in \{\text{free}, \text{non-skip}, \text{skip-voted}\}$, initially free. Immediately before a correct party emits a proposal ECHO for any carrying proposal whose metadata contains a full or core entry for $u$, it requires $z_i(u) \neq$ skip-voted and no known terminal skip, then persistently sets $z_i(u) \leftarrow$ non-skip before the ECHO. A later non-skip carrier may reuse that stance. A skip entry does not change it. Conversely, the post-ready skip rule below can change only a free stance to skip-voted. These persist-before-send transitions are atomic with the corresponding message emission. Concretely, each carrying view has an initially false accepted-metadata latch. After the relevant core and tip gate holds, R2 invokes TryMetaOK: if all checks below pass, one atomic transition claims the free non-skip coordinate, if any, sets the latch, and emits the one carrying ECHO. A successful latch is never cleared. The stance restricts what a party may endorse, not which common outcome it may later learn from the arbiter or control log.

A correct party echoes a view proposal whose metadata field contains a resolution for view $u$ only if its echo-stage and initial ready-stage responses for $u$ have already been durably emitted and its ready state is closed. An initial READY-mix is not closed while it may still refine; after a homogeneous refinement, all-$n$ echo-stage closure, or the

ready deadline, the effective local ready value is respectively that homogeneous grade or the finalized mixed grade. The following local conditions must hold at the echo decision time (R2). Successful TryMetaOK sets the persistent AuxOK$_i(w, M)$ latch for Vantage. The carrying ECHO is immutable; a still-pending target response or provisional READY-mix makes the entry fail its local check for that carrying attempt. In addition, if $L_i(u, B)$ is active for a data-only $B = (C_u, T_u, \emptyset)$, only the exact entry resolve$(u, \text{commit-full}, C_u, T_u)$ can pass; every core, skip, or different-payload entry fails. Every non-skip entry also fails when $z_i(u) =$ skip-voted or the party has already learned terminal skip.

- For resolve$(u, \text{commit-full}, C_u, T_u)$: the party's effective ready value for $u$ is neither a grade-0 proposal-ready nor a proposal-ready naming a payload different from $(C_u, T_u)$; every entry of $C_u$ and $T_u$ is locally available at the party; and the tip-anchoring check of R2 holds for $(C_u, T_u)$ — the party holds each paired tip entry's full lane prefix, which strictly contains the author's $C_u$ entry.
- For resolve$(u, \text{commit-core}, C_u, T_u)$: the party's effective ready value for $u$ is neither a grade-1 proposal-ready nor a proposal-ready naming a payload different from $(C_u, T_u)$; and every entry of $C_u$ is locally available at the party.
- For resolve$(u, \text{skip})$: the party's effective ready value for $u$ is no-ready.

If the resolution entry still fails its checks at the end of the echo window, the party's echo-stage response for the carrying view is echo-skip (R2). We say that a resolution entry is *echo-accepted* in the carrying view if its proposal has a carrying-view proposal-echo quorum; by R2, every correct member of that quorum passed the entry's checks.

This latched AuxOK remains persistent. TryMetaOK cannot succeed before the target echo is emitted; a matching grade-1 target echo records its lock before emission. A free stance that could later change is not itself the abstract predicate: successful acceptance first claims every non-skip stance, then sets the latch. Thereafter the target ECHO and closed effective ready value are immutable, local availability only grows, and an active lock can only become inactive. If commit-full$(C_u, T_u)$ passes while its lock is active, releasing that lock cannot invalidate it. The threshold update is evaluated before R3 on the same newly counted echo-stage responses, so a grade-0 or different-payload ready cannot be emitted while a contradictory lock remains active. This ordering is a local coherence convention rather than load-bearing: any ECHO set supporting such a ready already contains the $f+1$ nonmatching responses that release the lock, and Lemma D.4 does not rely on the ordering.

*Grounded post-ready skip.* After durably emitting no-ready for target $u$, a correct party that has directly counted a quorum of echo-skip responses for $u$, has learned no non-skip terminal seal for $u$, and still has $z_i(u) =$ free first processes every enabled optimistic-lock release, then persistently sets $z_i(u) \leftarrow$ skip-voted and broadcasts the one-shot statement $\langle \text{skip-vote}, u \rangle$. A correct party sends at most one such statement per target and counts only the first directly received statement per author. Learning the compatible outcome skip early does not suppress this vote; the eventual duplicate submission is idempotent at the arbiter. Upon counting a quorum of them, the Vantage wrapper emits

$$\text{skip-seal}(u) \longrightarrow \text{skip}$$

and submits it to try-seal. This caller-side shortcut does not fire complete or direct-seal, create a completion report, or alter R1–R4. There is deliberately no "send on $f+1$ skip votes" amplification: the resolver remains the total route when a Byzantine sender selectively supplies votes.

The carrier's local availability checks deliberately do not rerun AuthorOK: repair makes data common, whereas selectively sent Byzantine acknowledgments need not make provenance evidence common. Instead, the *origin bit* — Vantage's echo annotation Ann — transports an existential target-view witness through first-hand counting. For an

exact payload $P = (C_u, T_u)$,

$$\begin{aligned} &\textsc{Origin}_i(R_u^{\text{full}}(P)) = 1 \\ \iff\ & p_i \text{ echoed } P \text{ with grade 1 in } u, \end{aligned}$$

$$\begin{aligned} &\textsc{Origin}_i(R_u^{\text{core}}(P)) = 1 \\ \iff\ & p_i \text{ echoed } P \text{ with grade 0 or 1 in } u. \end{aligned}$$

It is empty for a skip or empty entry. These are immutable local-history facts. The ready guard requires $f$+1 carrying-proposal echoes with origin value 1 before proposal-ready, so at least one such echo is from a correct sender. Its target-view echo passed AuthorOK for exactly the data the resolution would output — $C_u \cup T_u$ for full and $C_u$ for core — and was based on a proposal received directly from $\mathsf{Proposer}(u)$. Thus the same gate preserves both data-author provenance and the target proposer's payload association without a transferable certificate.

Vantage instantiates the abstract expansion contract as follows. Per-view sealing and ordered output are distinct. A manifest is traversed in its encoded vector order (increasing party index for every well-formed proposal), and each named lane prefix is traversed from genesis toward its named frontier. Genesis is not output. Let $D$ be the set of block hashes already output when the cursor reaches view $v$, initialized with the genesis hash, and let $\mathsf{Expand}_D(X)$ expand the manifest in that order and return the resulting block sequence after omitting hashes in $D$ and earlier occurrences in the same traversal. Deduplication at this layer is by block hash. Define $D + K$ as $D$ augmented by the hashes of all blocks in sequence $K$. Then

$$\begin{aligned} K_{v,D} &= \mathsf{Expand}_D(C_v), \\ \widehat{T}_{v,D} &= \mathsf{Expand}_{D+K_{v,D}}(T_v), \\ \mathsf{Full}_D(C_v, T_v) &= K_{v,D} \parallel \widehat{T}_{v,D}, \\ \mathsf{Core}_D(C_v) &= K_{v,D}. \end{aligned}$$

Thus the completed core is literally a prefix of the full view expansion, possibly equal to it after duplicate removal.

The ordering contract produces data blocks that are author-backed and externally valid. BlockOK is deliberately order-independent. After ordering, an application applies a fixed deterministic transition function to the common block sequence; state-dependent transaction validation, replay handling, and transaction-level deduplication belong to that function. An invalid transaction at the resulting state may, for example, be a deterministic no-op. Client authorization is orthogonal to the validator-block authorship proved here.

The output cursor processes views in increasing order. At a locally completed but open cursor view, the party requests any missing core lane prefixes, emits $K_{v,D}$ only after obtaining and verifying their hashes, author coordinates, heights, predecessor-hash links, and BlockOK predicates, records that the core was emitted, and stops before $T_v$. A commit-full seal first emits $K_{v,D}$ if needed, then requests any missing tip lane prefixes and appends $\widehat{T}_{v,D}$ only after obtaining and verifying them, before advancing the cursor; a commit-core seal first emits $K_{v,D}$ if needed and advances without appending $T_v$; a skip seal emits nothing and advances. A non-skip anchor may be observed before local completion and supplies the manifests for the same procedure; a fast full seal already names both manifests, which the cursor uses directly. A late compatible completion or seal is idempotent and never reopens the view or duplicates output. Parties continue later AGB instances and buffer their results while a tip is open, but no payload from a later view crosses that cursor position.

Every route to commit-full includes correct parties that check tip anchoring: grade-1 echoers on the direct path and carrying-view echoers for an anchored full resolution. Paired core and tip entries therefore expand along one lane

branch. Byzantine authors may still fork across views; canonical expansion and first-occurrence deduplication remain deterministic, while a correct proposer's paired manifests are linear by construction.

### C.1 Resolved AGB Contract

Direct AGB intentionally leaves residual mixed-grade and silent instances unsealed. A *resolver* is a composition service that may call APPLY-ANCHOR$(u, X)$ through the adapter above. It need not globally order resolutions for different target views. The reusable semantic contract is per target. Every non-skip call also supplies a backing payload $P_X = (C, T)$ from its resolution record; even when $X = \mathsf{commit\text{-}core}(C)$ omits $T$, the backing payload retains it for identity and compatibility checks.

**RS1. Single choice.** All resolver calls at correct parties for target $u$ supply one exact outcome $X_u$ and, for non-skip, one exact backing payload $P_u$.

**RS2. Future-closed direct compatibility.** If $X_u = \mathsf{commit\text{-}full}(C, T)$ with $P_u = (C, T)$, no grade-0 ready quorum and no ready quorum for a payload other than $P_u$ can ever exist. If $X_u = \mathsf{commit\text{-}core}(C)$ with $P_u = (C, T)$, no grade-1 ready quorum and no ready quorum for a payload other than $P_u$ can ever exist. If $X_u = \mathsf{skip}$, no ready quorum can ever exist.

**RS3. Output validity.** A non-skip outcome satisfies $\mathsf{Witnessed}(C)$ for commit-core and both $\mathsf{Witnessed}(C)$ and $\mathsf{Witnessed}(T)$ for commit-full.

For termination we use one additional property:

**RL1. Per-target totality.** For every target $u$, there exists an outcome $X_u$ such that every correct party eventually either directly emits and submits the result $X_u$ or receives the resolver call APPLY-ANCHOR$(u, X_u)$.

This service has agreement power; the definition does not claim that AGB implements it.

THEOREM C.1 (RESOLVED AGB COMPOSITION). *Direct AGB composed with any resolver satisfying RS1–RS3 has at most one terminal outcome at each correct party, and no two correct parties terminally seal different outcomes for a target $u$. If completion exists, every terminal outcome that occurs is non-skip, is backed by the completion's unique payload, and contains its core; for every prior-output set $D$, that core expansion is a prefix of the full expansion. Every non-skip outcome satisfies the eligibility witness promised by RS3 or by the direct AGB path. If RL1 also holds, every correct party eventually terminally seals $u$.*

PROOF. The caller-owned seal arbiter gives local uniqueness. Two direct results agree by Theorem B.7(b)–(c). Two resolver submissions agree by RS1, and a direct result agrees with a resolver submission by RS2 and AGB value agreement. A completion is backed by a ready quorum, so future-closed RS2 excludes skip and binds every non-skip resolution to its payload. Its core is fixed by Theorem B.7(b),(d), and the value-domain expansion contract makes $\mathsf{Core}_D(C)$ a prefix of $\mathsf{Full}_D(C, T)$. Direct outcomes obtain their witness from Theorem B.7(d); resolver outcomes obtain it from RS3. Finally, RL1 feeds the common outcome into the caller-owned arbiter at every party that did not already accept the direct result. □

Call a sequence of instances *input-fair* when every admissible item from a correct source eventually occurs in $\mathsf{Items}(C)$ for the core $C$ of some completed instance.

COROLLARY C.2 (ATOMIC BROADCAST FROM RESOLVED AGB). *Run one Direct AGB instance per increasing view, compose each with a resolver satisfying RS1–RS3 and RL1, and assume input fairness and fair repair. Use the following prefix-cursor*

*discipline: a completed core may be emitted once, no tip or later view crosses an open tip, terminal processing is idempotent, and every root is authorized and retrieved before output. With deterministic view-order expansion and first-occurrence deduplication, correct parties' output sequences are prefix-comparable, have one common limit, and eventually contain every finite prefix of that limit. Every output item is emitted at most once, satisfies the value domain's external-validity predicate, is witnessed as required by Direct AGB or RS3, and is retrieved before local output. Every admissible correct-source item is eventually output. Applying a fixed deterministic transition function to this stream implements state-machine replication.*

Proof. By Theorem C.1, all correct parties use one outcome per view, and a completed open core is a prefix of either compatible non-skip outcome. The cursor rule therefore exposes only prefixes of the same deterministic view-order expansion. RL1 advances every cursor through every finite set of views; RS3, Direct AGB availability, and fair repair provide the referenced prefixes before output. First-occurrence deduplication gives at-most-once output, and input fairness gives correct-input inclusion. Equal input prefixes and a deterministic transition function give equal state prefixes. Under Vantage's eligibility instantiation, external validity and witnessing mean BlockOK, publication by the encoded author to a correct party, and permanent retrievability. □

*Batched resolution entries.* Under crash faults, clean silent views arrive in runs: any $f$+1 consecutive views have distinct round-robin proposers, including at least one correct party, so such a run has length at most $f$. Their all-no-ready evidence is uniform. Vantage therefore permits one special batch: $M_w$ may be a vector of resolve$(u_j, \text{skip})$ entries for strictly increasing targets $u_1 < \cdots < u_k \le w - 3$, with $2 \le k \le f$. A vector with a full or core entry is malformed; those outcomes use one general entry. This restriction is load-bearing for the proved bit bound: a skip vector has $O(f)$ scalar fields and no manifests, whereas allowing $f$ independent full/core payloads would make one statement $O(n^2\lambda)$.

For a skip vector, AuxRefs and the annotation are empty, the ready guard passes, and TryMetaOK is the conjunction of the per-target no-ready and active-lock checks before latching AuxOK. At an eligible recovery turn, the proposer may take the maximal consecutive prefix of its scan for which every target justifies skip; the prefix stops at the first other target. While its oldest qualifying target remains fixed, every other recovery attempt is the ordinary single-entry attempt for that target. Thus a refusable rider cannot starve a full/core candidate or the oldest target's candidate cycle.

Safety is per coordinate: a carrying proposal-echo quorum for the vector is an echo-accepted skip entry for every included target, so Lemmas D.4 to D.6 apply separately. For each target $u$, $A_u$ remains the smallest applicable control-log position whose metadata contains an entry for $u$; O1 makes it common and contiguous processing applies a batch in increasing target order, idempotently per target. RL1 is preserved because recurring single-entry attempts retain the original liveness proof. The batch is only an opportunistic drain-rate optimization for skip-only fallback and changes no per-view duration bound.

## D CONTROL LOG: FULL CONSTRUCTION AND PROOFS

Direct homogeneous results and grounded skip seals need no further agreement. Residual mixed, silent, or partial views do: locally valid entries may disagree between commit-full and commit-core, while completion is not total. The background log $\mathcal{L}$ therefore orders sparse hashes of resolution-bearing proposals. It gates neither core completion, local direct paths, nor later data-plane proposals, though an open tip can hold the output cursor. Its common order implements one per-target choice for Appendix C.1.

*Completion reports.* When a correct party first fires $\textsc{complete}(w) \to B_w$ for a well-formed proposal with $M_w \neq \emptyset$, it broadcasts

$$\langle \textsc{complete-report}, w, \mathcal{H}(B_w) \rangle$$

and, in the formal protocol, retains and serves $B_w$ indefinitely. A Vantage FAST-SEAL or SKIP-SEAL event alone creates no report: only the genuine R4 completion above is eligible. A recipient counts only the first report from each author for a data-plane view, received directly from that author. Reports are local evidence: they are never relayed or placed in the log as purported statements by somebody else. Every correct party that validates a control value also retains and serves $B_w$ indefinitely. Checkpoint garbage collection is outside the model; a correct implementation may collect only after proving that no unanchored candidate or unconsumed log position depends on the body.

*Validated control broadcast.* The control log is non-speculative Simple-IT with Bracha reliable broadcast [9, 42]. Its rounds and round-robin leaders are independent of data-plane views. A nonempty logged block is $x = (w, h)$; $\perp$ is an empty block. A pair becomes a submitted control block at a party once that party has counted $n-f$ matching completion reports and obtained and verified $B_w$ with $h = \mathcal{H}(B_w)$ and $M_w \neq \emptyset$.

The control leader follows the ordinary Simple-IT parent rule: it first chooses the highest safe control round $r_p$ for which every intervening round is reliably disabled. Write $\mathsf{Log}(r_p)$ for the blocks on $r_p$'s safe parent chain. The leader then proposes the smallest-view submitted pair not already in its delivered sequence or in $\mathsf{Log}(r_p)$, and proposes $\perp$ if no such pair exists. Thus it never repeats a block already on its chosen parent chain; committing the descendant would expose that block as an ancestor instead. The reliable-broadcast identity covers the complete Simple-IT proposal — control round, parent, and block $x$ — while a nonempty INIT also carries $B_w$ as validation data. The log retains only $x$.

We use the following validated form of Bracha broadcast for a control value. A correct party stores the first complete control proposal received directly from the control leader and may ECHO only that proposal. For a proposal with block $x = (w, h)$, it sends its one ECHO only after it has counted $f$+1 matching completion reports and verified that the attached $B_w$ has view $w$, is well formed, satisfies $h = \mathcal{H}(B_w)$, and has $M_w \neq \emptyset$. The predicate is true immediately for $\perp$. Bracha's subsequent rules are unchanged: READY is sent on $n-f$ matching ECHOs or $f$+1 matching READYs, and delivery occurs on $n-f$ matching READYs. In particular, READY relay and delivery do *not* re-check the local report predicate. ECHO and READY carry the complete, short control proposal, so RBC delivery does not depend on recovering a hash preimage; they do not retransmit $B_w$. A party that did not receive the validation data from a Byzantine control leader requests $B_w$ once from every matching REPORT and ECHO author, accepts the first valid response, and processes the anchor before advancing to the next log position. A correct holder answers at most once per requester–$(w, h)$ pair. At least one correct ECHO author exists and eventually answers. The party does not re-impose the $f$+1-report predicate after RBC delivery.

Lemma D.1 (Validated control broadcast). *The validated broadcast has Bracha uniqueness, consistency, and totality. Moreover:*

(i) *every nonempty value delivered at a correct party names a proposal that completed at a correct party;*

(ii) *if a correct control leader proposes after counting $n-f$ matching reports, every correct party eventually delivers its value; and*

(iii) *if that leader sends INIT at $t_0 \geq \mathrm{GST}$, every correct party delivers by $t_0 + 3\delta$; if one correct party delivers at $t \geq \mathrm{GST}$, every correct party delivers by $t + 2\delta$.*

Proof. The first-stored-proposal rule and one-ECHO state give local uniqueness. The standard one-ECHO, READY-amplification argument gives consistency and totality; neither argument requires every party to have sent an ECHO. For (i), consider a correct READY for a delivered nonempty value that is minimal in the happens-before order among correct READYs for it. Triggering by $f$+1 earlier READYs would include a correct one preceding it, contradicting minimality since at most $f$ of those could be Byzantine. It therefore counted $n-f$ ECHOs, including at least $f$+1 correct ECHO authors. Each such author counted $f$+1 direct completion reports, including one from a correct reporter. That reporter sent only after locally completing the named $B_w$.

For (ii), the leader's $n-f$ reports include at least $f$+1 correct reporters. Their broadcasts reach every correct party, and the leader attaches the verified $B_w$ to its INIT. A correct leader sends one proposal, so the first-stored-proposal rule never binds a correct party to a conflicting value. Thus every correct party eventually satisfies the ECHO predicate for the leader's unique value. The usual Bracha validity argument then gives delivery everywhere.

For (iii), the INIT and the $f$+1 correct reports reach every correct party by $t_0 + \delta$. All correct parties then ECHO; their ECHOs arrive by $t_0 + 2\delta$, and their READYs by $t_0 + 3\delta$. For the second bound, a correct delivery at $t$ counted $n-f$ READYs, at least $f$+1 from correct authors. Those READYs reach every correct party by $t + \delta$ and trigger any missing correct READY; the resulting $n-f$ correct READYs arrive by $t + 2\delta$. Carrying the short control proposal in READY makes this the delivery time as well. □

*Simple-IT adapter.* Algorithm 5 instantiates the non-speculative protocol of Simple-IT Figure 2 and the reliable-notification (RN) protocol of its Figure 4 [42]; Vantage does not use the speculative protocol of that paper's Figure 3. A Simple-IT block is a control value $x \in \{\perp\} \cup \{(w, h)\}$. Its local submitted set is exactly the set of pairs passing Vantage's $(n-f)$-report rule. Its RBC identifier is the control round, and its RBC payload is the complete tuple $(r, p, x)$ naming the parent round. The variables proposal, safe, disabled, committed, voted, and timedOut; the SafeParent and Log functions; and the enter, vote, timeout, commit, deliver, and advance events have the same meaning and guards as in Simple-IT Figure 2. A Simple-IT tob-deliver event appends $x$ to Vantage's raw control log. Completion reports, attached proposal bodies, validation, and later body repair are adapter state: they restrict submission and the validated-broadcast ECHO, but do not alter any Simple-IT state transition. At $n = 3f + 1$, the RN thresholds also match Figure 4 exactly. For larger committees, Figure 4's vote and cascade thresholds are already the parametric $n - f$ and $f + 1$; Vantage conservatively raises only RN confirmation from Figure 4's $2f + 1$ to $n - f$, and Lemma D.2 proves the same RN interface and timing bound for this threshold.

Lemma D.2 (Control reliable notification). *The RN rules in Algorithm 5 satisfy Simple-IT Definition 7:* unanimity*, if all correct parties raise $e$, all eventually confirm it;* totality*, if one correct party confirms $e$, all eventually confirm it; and* validity*, a correct confirmation implies that at least $n - 2f \geq f + 1$ correct parties raised $e$. After GST, confirmation at one correct party is followed by confirmation everywhere within $2\delta$.*

Proof. If all correct parties raise $e$, their $n - f$ RN-VOTEs make every correct party send RN-ACCEPT, after which the $n - f$ correct accepts make all confirm. If one correct party confirms, it counted $n-f$ accepts, at least $f + 1$ from correct authors. Those accepts reach every correct party and trigger the cascade rule; the resulting $n - f$ correct accepts make all confirm. This takes at most two post-GST message delays from the first confirmation.

For validity, consider the first correct party to send RN-ACCEPT. It cannot have used the cascade rule, which would require an earlier correct accept among $f + 1$ accepts. It therefore counted $n - f$ RN-VOTEs, at least $n - 2f$ of them from correct parties, and a correct party sends RN-VOTE only after raising $e$. □

**Algorithm 5:** Non-speculative Simple-IT control adapter and reliable notification at party $p_i$

```
/* Count only first-hand messages from distinct authors; Q = n − f; validated-RB is the broadcast defined in Lemma D.1. */
Global variables and constants:
    curr ← 1; submitted, delivered ← ∅; proposal[0] ← (⊥, genesis); safe[0] ← true
    for r > 0: proposal[r] ← ⊥ and safe[r], disabled[r], committed[r] ← false
    rnVoted[e], rnAccepted[e], rnConfirmed[e] ← false for every RN flag e
    enter control round 1
function SafeParent(r, p):
    return 0 ≤ p < r, safe[p], and disabled[q] for every p < q < r
function Log(r):
    if r = 0 then return the empty sequence
    let proposal[r] = (p, x)
    return Log(p) followed by x
upon Q matching direct ⟨complete-report, w, h⟩ messages have been counted and B_w is verified with H(B_w) = h and M_w ≠ ∅ do
    add x = (w, h) and its body B_w to submitted
upon entering control round r do
    curr ← r; voted ← false; timedOut ← false; arm the round-r timer for 6Δ
    if p_i is the round-r control leader then
        p ← max{q : SafeParent(r, q)}
        choose the lexicographically smallest x = (w, h) ∈ submitted \ delivered with x ∉ Log(p), or x ← ⊥ if none exists
        invoke validated-RB-broadcast on (r, p, x), attaching B_w when x ≠ ⊥
upon validated-RB-deliver returns (r, p, x) in control round r do
    proposal[r] ← (p, x)
upon proposal[r] = (p, x) and SafeParent(r, p) do
    safe[r] ← true
upon safe[curr], not timedOut, and not voted do
    voted ← true; broadcast ⟨control-commit, curr⟩
upon the current round timer expires and not voted do
    timedOut ← true; invoke RnRaise(⟨timeout, curr⟩)
upon RnConfirm(⟨timeout, r⟩) do
    disabled[r] ← true
upon Q direct ⟨control-commit, r⟩ messages have been counted do
    committed[r] ← true
upon committed[r] and safe[r] do
    append, in order, every non-⊥ value of Log(r) not already in delivered; expose the resulting new suffix as control-log deliveries
upon safe[curr] and (voted or timedOut), or disabled[curr] do
    enter control round curr + 1
procedure RnRaise(e)
    if not rnVoted[e] then
        rnVoted[e] ← true; broadcast ⟨rn-vote, e⟩
upon either Q direct ⟨rn-vote, e⟩ messages or f + 1 direct ⟨rn-accept, e⟩ messages have been counted, and not rnAccepted[e] do
    rnAccepted[e] ← true; broadcast ⟨rn-accept, e⟩
upon Q direct ⟨rn-accept, e⟩ messages have been counted and not rnConfirmed[e] do
    rnConfirmed[e] ← true; fire RnConfirm(e)
```

*Imported interface and theorems.* Simple-IT Definition 6 requires RBC uniqueness, correct-sender validity, and totality. Lemma D.1 supplies uniqueness and totality for every control leader. It supplies correct-sender validity because a correct leader broadcasts only $\perp$ or a pair for which it has counted $n-f$ reports, exactly the premise of Lemma D.1(ii). Part (iii) gives the worst-case correct-sender delay $d_s = 3$ and totality delay $d_t = 2$. The RN interface is supplied by Lemma D.2, including its validity threshold and two-delay totality.

We use the following results from the non-speculative Simple-IT proof, with the mapping above: Lemma 1 (a committed round is never disabled at a correct party), Lemma 4 and Theorem 1 (committed logs lie on one parent chain and are totally ordered), Lemma 5 and Corollary 1 (each round becomes safe or disabled and correct parties enter every round), Lemma 14 (a post-GST correct-leader round commits when its timeout is strictly greater than $(d_s + d_t)\delta$), and

Theorem 3 (a delivered block is eventually delivered by every correct party) [42]. Vantage arms $6\Delta$, which is strictly greater than $5\delta$ because $\delta \le \Delta$. No other Simple-IT state, theorem, speculative step, or cryptographic assumption is imported.

Let $\mathcal{L} = (\alpha_1, \alpha_2, \ldots)$ be the sequence of nonempty control values delivered by Simple-IT, after deterministically discarding a duplicate $(w, h)$ after its first occurrence. A value $\alpha_k = (w, \mathcal{H}(B_w))$ is an *anchor* for $B_w$. Correct parties consume only contiguous prefixes of $\mathcal{L}$ and obtain $B_w$ before processing its metadata; below, "observe" means completing this processing. This cannot block forever: the proof of Lemma D.1(i) identifies a correct ECHO author that obtained and serves $B_w$ indefinitely.

Theorem D.3 (Control-log properties). *The anchor sequence satisfies:*

- ***O1. Agreement.*** *Correct parties observe prefixes of one anchor sequence, and every correct party eventually observes every anchor.*
- ***O2. Completion backing.*** *Every anchored proposal has a proposal-ready quorum. Hence its resolution entry is echo-accepted; for a non-skip entry, a correct ready sender also counted $f+1$ proposal echoes carrying origin value 1.*
- ***O3. Progress.*** *Every resolution-bearing proposal that completes at every correct party is eventually anchored.*

Proof. For O1, Simple-IT total order and totality give the common raw log. Each log position has only finitely many predecessors, and the serving rule above lets every correct party fetch their bodies in sequence; deterministic duplicate removal therefore gives one common, eventually processed anchor prefix. For O2, Lemma D.1(i) supplies a correct completion reporter. Its local completion counted a proposal-ready quorum; a correct member of that quorum grounded a proposal-echo quorum and, for non-skip metadata, passed the origin-bit ready guard in R3.

For O3, universal completion makes at least $n-f$ correct parties report the same $(w, \mathcal{H}(B_w))$, so the candidate eventually satisfies every correct control leader's $n-f$ threshold. At most one hash per data-plane view can become eligible at any correct leader: two $n-f$ report quorums intersect in at least $n - 2f \ge f+1$ authors, including a correct author, who reports only its unique completion for that view. There are thus only finitely many eligible candidates at smaller views. Correct control leaders recur forever and, after GST, their globally echoable control proposals commit under Simple-IT. At such a leader, a fixed pending candidate is either already in the chosen safe parent's log, in which case committing the descendant anchors it as an ancestor, or it remains selectable. Induction over the finitely many smaller candidates eventually anchors $B_w$ in one of these two ways. □

For each data-plane view $u$, let $A_u$ be the anchor at the smallest log position $k$ whose proposal view is at least $u+3$ and whose metadata resolves $u$. The minimum is over control-log positions, not numerical carrying views; thus later log growth can never change $A_u$. By O2, its entry is echo-accepted. Parties apply anchors only while consuming a contiguous control-log prefix, so a later locally received anchor is never applied ahead of a missing predecessor. When that processing reaches $A_u$, the composition derives $X_u$ from its resolution entry, supplies its non-skip manifests, and invokes apply-anchor$(u, X_u)$. Later anchors targeting $u$ are ignored. The arbiter receives the following compatible submissions:

(i) all $n$ exact grade-1 echoes for a data-only proposal submit the Vantage fast result commit-full$(C, T)$ to try-seal;

(ii) a grade-1 ready quorum submits the direct result commit-full$(C, T)$ to try-seal;

(iii) a grade-0 ready quorum submits the direct result commit-core$(C)$ to try-seal;

(iv) a quorum of grounded SKIP-VOTE statements submits the Vantage SKIP-SEAL result skip to TRY-SEAL;

(v) when $A_u$ is processed, its adapter submits the outcome carried by the resolution entry for $u$.

The rule needs no applicability side conditions: by Lemmas D.4 to D.6, an echo-accepted resolution entry never conflicts with a correct terminal submission for $u$. If $u$ is completed anywhere, the entry is commit-full or commit-core and names the completed payload.

LEMMA D.4 (FAST-SEAL COMPATIBILITY). *For a data-only proposal $B = (C, T, \emptyset)$:*

(i) *any two correct FAST-SEAL events for a view name the same payload;*

(ii) *a correct FAST-SEAL$(v) \longrightarrow$ commit-full$(C, T)$ is compatible with every Direct AGB result for $v$, and $C$ and $T$ are witnessed;*

(iii) *it is compatible with every echo-accepted resolution entry for $v$, regardless of whether the fast seal or the carrying echo quorum occurs first. In particular, such an entry can only resolve* commit-full$(C, T)$.

PROOF. A correct fast observer counted the unique echo-stage response of every author as an exact grade-1 echo for $B$. In particular, every correct party emitted that response and installed $L_i(v, B)$ before doing so. Correct echo-stage responses are broadcast and immutable. Hence two all-$n$ censuses contain the same response from every correct author and cannot name different proposals, proving (i).

No correct grade-0 proposal-ready sender and no correct proposal-ready sender for a different payload can coexist with these correct echoes: either sender's grounding echo quorum contains a correct echoer, whose unique response would have to be grade 0 for $B$ or name a different proposal. Therefore no corresponding ready quorum exists. A grade-1 direct result can only name $(C, T)$, proving direct compatibility. The fast observer itself passed CoreOK and TipOK before its echo; the eligibility contract gives Witnessed$(C)$ and Witnessed$(T)$.

For (iii), no temporal case split is needed: echo-stage responses are immutable, so an all-$n$ fast-seal ECHO set and an echo-accepted entry are facts about the same fixed response assignment. Suppose both a correct FAST-SEAL ECHO set for $B$ and an echo-accepted resolution entry incompatible with commit-full$(C, T)$ exist, and choose a correct member $p_i$ of the entry's carrying echo quorum. By its accepted AuxOK latch, $p_i$ emitted its target-view echo before accepting the entry. The all-$n$ set counts that response as an exact grade-1 echo for $B$, so $p_i$ installed $L_i(v, B)$ and could pass the incompatible entry only after counting $f+1$ echo-stage responses for $v$ that are not matching responses for $B$. At least one was broadcast by a correct author, whose unique immutable response also appears in the all-$n$ set — contradicting that all $n$ counted responses there match $B$. □

LEMMA D.5 (GROUNDED SKIP COMPATIBILITY). *For every target view $u$:*

(i) *if a correct party broadcasts SKIP-VOTE$(u)$, no correct party ever broadcasts a proposal-ready for $u$; hence no completion, Direct AGB result, or Vantage fast seal for $u$ can exist;*

(ii) *a quorum of SKIP-VOTE$(u)$ statements and a carrying-view proposal-echo quorum for a non-skip resolution of $u$ cannot both exist, regardless of which messages are emitted first;*

(iii) *consequently, every correct SKIP-SEAL$(u) \longrightarrow$ skip submission is compatible with every direct, fast, or resolver submission for $u$.*

PROOF. Let $b \le f$ be the actual number of Byzantine parties. A correct skip voter counted a quorum of target-view ECHO-SKIP responses, including at least $n - f - b$ correct echo-skippers. If a correct proposal-ready sender existed, its grounding proposal-echo quorum would likewise contain at least $n - f - b$ correct proposal echoers. The two correct

sets are disjoint because the echo-stage response is one-shot, but

$$2(n - f - b) > n - b,$$

with margin $n - 2f - b \geq f + 1 - b > 0$, contradicting the number of correct parties. Thus no correct proposal-ready exists. Every ready quorum contains a correct sender, so completion and both Direct AGB results are impossible. The counted ECHO-SKIP quorum also contains a correct echo-skipper, excluding the all-$n$ exact grade-1 ECHO set of the fast seal. This proves (i).

For (ii), let $S$ be a skip-vote quorum and $E$ a carrying proposal-echo quorum for a non-skip entry targeting $u$. Both have size $n - f$, so $|S \cap E| \geq n - 2f \geq f + 1$ and the intersection contains a correct party. Before sending its vote that party persistently claimed $z_i(u) =$ skip-voted and would reject the carrier; before sending its carrier ECHO it instead persistently claimed $z_i(u) =$ non-skip and could not vote. The claims are exclusive in either temporal order, a contradiction.

Part (i) excludes every target-view fast or Direct AGB result. By (ii), no non-skip resolution entry can be echo-accepted when a skip-vote quorum exists; an echo-accepted skip entry has the same outcome. O2 of Theorem D.3 permits only echo-accepted entries to become anchors, proving (iii). □

LEMMA D.6 (DIRECT–RESOLUTION COMPATIBILITY). *Let $\xi$ be a resolution entry for view $u$ that is echo-accepted in a carrying view. Then:*

(i) *if $\xi$ resolves $u$ as* commit-full *with payload $(C_u, T_u)$, then no grade-0 proposal-ready quorum and no proposal-ready quorum for a different payload ever exists; every entry of $C_u$ and $T_u$ is retrievable;*

(ii) *if $\xi$ resolves $u$ as* commit-core *with payload $(C_u, T_u)$, then no grade-1 proposal-ready quorum and no proposal-ready quorum for a different payload ever exists; every entry of $C_u$ is retrievable;*

(iii) *if $\xi$ resolves $u$ as* skip, *then no proposal-ready quorum for $u$ ever exists; in particular, no correct party ever completes $u$;*

(iv) *if the carrying proposal has a proposal-ready quorum and $\xi$ is non-skip, then the valid lane prefix named by every entry that $\xi$ outputs was published by its encoded author to a correct party.*

PROOF. Let $Q$ be the carrying-view proposal-echo quorum. Every correct member of $Q$ passed the entry's checks after durably emitting its unique target ECHO and reaching a closed effective ready value for $u$. Any target-view proposal-ready quorum intersects $Q$ in a correct party. For commit-full, that party's check excludes grade 0 and a different payload; for commit-core, it excludes grade 1 and a different payload; for skip, it requires NO-READY. These three intersections prove the respective quorum exclusions, and the last excludes completion. For commit-full and commit-core, correct members of $Q$ also passed the stated local availability checks, so Lemma B.2(i) proves retrievability. Every party that later completes or seals the carrying proposal, or consumes its anchor, authorizes the relevant roots and, by Lemma B.2(ii), eventually obtains them in every fair execution.

For (iv), the carrying proposal's proposal-ready quorum contains a correct sender that passed the $f$+1-origin-bit ready guard in R3. At least one counted bit sender is correct. For a full entry, it previously sent a grade-1 target echo for the exact payload and therefore passed AUTHOROK for $C_u \cup T_u$; for a core entry, it sent a target echo of either grade and passed AUTHOROK for $C_u$. The author-witness lemma gives the claimed validity and provenance. □

LEMMA D.7 (VANTAGE RESOLVER SAFETY). *The first-applicable-anchor adapter satisfies RS1–RS3 of Appendix C.1.*

PROOF. O1 of Theorem D.3, contiguous prefix consumption, and the definition of $A_u$ make every correct party choose the same first applicable anchor, so RS1 holds. O2 makes its resolution entry echo-accepted. Parts (i)–(iii) of

Lemma D.6 then give RS2. For a non-skip entry, parts (i)–(ii) give retrievability and part (iv), using O2's origin backing, gives validity and publication provenance, establishing RS3. □

Theorem D.8 (Prefix and seal agreement). *Each correct party seals a view at most once, and no two correct parties seal different outcomes for any view $u$. If a terminal outcome exists, the common outcome is* commit-full$(C, T)$, commit-core$(C)$, *or* skip, *where $(C, T)$ is the unique payload of Theorem B.7(b) whenever $u$ is completed anywhere; the unique fast payload of Lemma D.4 when a fast seal exists without completion; and otherwise either* skip *from Lemma D.5 or the outcome and any backing payload of the unique first applicable anchor $A_u$ of Section 6.1. Before sealing, a correct completion may expose* open-tip$(C, T)$*; it is compatible with either non-skip terminal outcome and its expanded core is a prefix of both. A party that has not completed or observed a seal may remain unknown. In particular, whenever completion exists, it excludes* skip *and every non-skip seal uses its payload. For any such payload and every prior-output set $D$,* $\mathsf{Core}_D(C)$ *is a prefix of* $\mathsf{Full}_D(C, T)$.

Proof. Apply Theorem C.1 using Lemma D.7 to the Direct AGB and anchor submissions. Lemma D.4 makes every Vantage fast submission compatible with those submissions and with every other fast submission; Lemma D.5 does the same for every skip-seal submission. The caller-owned arbiter gives local uniqueness. The possible pre-seal open-tip and unknown local states are exactly the non-total states permitted by Theorem B.7(c); neither changes the core prefix fixed by completion. □

Theorem D.9 (Sealing liveness). *With the pacemaker of Appendix B.3 and the control log above, every correct party eventually seals every view. In particular, a completed open view is sealed as* commit-full *or* commit-core *at every correct party. Thus Vantage satisfies the per-target totality conclusion of RL1 in Appendix C.1, with* fast-seal *and* skip-seal *as additional compatible local routes.*

Proof. By strong induction on $u$, assume every view below $u$ is eventually sealed at every correct party. If every correct party eventually seals $u$ through a local fast, skip, or Direct AGB result, there is nothing to prove. Otherwise fix a correct party $q$ that never does so.

By Lemma B.4(a) and the two fallback rules, every correct party's echo-stage and initial ready-stage responses for $u$ are eventually emitted and broadcast, and every provisional READY-mix eventually closes at the ready deadline if it does not refine earlier. Let $b \le f$ be the actual number of Byzantine parties. For each exact data-only proposal $B = (C, T, \emptyset)$, let $G_B$ be the set of correct parties whose echo-stage response is a grade-1 echo for $B$. These sets are disjoint. Call $B$ *large* when

$$|G_B| \ge n - f - b.$$

There is at most one large proposal, because $2(n - f - b) > n - b$, the number of correct parties.

First suppose a large $B = (C_u, T_u, \emptyset)$ exists. Set $P_u = (C_u, T_u)$, $Y^* = R_u^{\text{full}}(P_u)$, and $X^* = \text{commit-full}(C_u, T_u)$. Every correct grade-0 ready for this payload, every correct ready for another payload, and every correct ready for a same-payload proposal with different metadata would have a grounding echo quorum containing at least $n - f - b$ correct responses disjoint from $G_B$ — echoes name the full proposal, and $G_B$ counts exact echoes of $(C_u, T_u, \emptyset)$ — contradicting the same cardinality inequality. Thus the ordinary ready-response checks for $Y^*$ eventually pass everywhere. Locks for $B$ accept $Y^*$ while active; every lock for another proposal eventually counts at least $|G_B| \ge f + 1$ correct nonmatching responses and becomes inactive. The correct members of $G_B$ also make $Y^*$ justified, supply at least $f + 1$ origin bits, and establish retrievability and tip anchoring. No correct party can send skip-vote$(u)$ in this case: its grounding echo-skip

quorum would contain at least $n - f - b$ correct echo-skippers disjoint from $G_B$, contradicting the same cardinality inequality. Hence no correct stance is skip-voted, and the new stance check does not obstruct the full entry.

Now suppose no proposal is large. A member of any nonempty $G_B$ eventually counts at least

$$(n - b) - |G_B| \geq f + 1$$

correct nonmatching echo-stage responses. Hence every optimistic lock eventually becomes inactive. There are at least $n - f$ correct initial ready-stage responses, so every correct party eventually counts a quorum of them. Those initial values together with permitted refinements define exactly one of the following three worlds:

(W1) some correct initial or refined response is a grade-1 proposal-ready;

(W2) otherwise, some correct initial response is a proposal-ready (grade 0 or mix, possibly with a grade-0 refinement);

(W3) otherwise, every correct initial response is NO-READY.

In W1 and W2, Theorem B.7(b) gives one proposal $B_u$ with payload $P_u = (C_u, T_u)$. Let $Y^*$ be $R_u^{\text{full}}(P_u)$ in W1, $R_u^{\text{core}}(P_u)$ in W2, and $R_u^{\text{skip}}$ in W3; define $X^*$ correspondingly. In W1, a correct grade-1 sender's grounding quorum supplies $f + 1$ correct grade-1 echoes and excludes correct grade-0 or mismatched-payload readies. In W2, a correct ready sender's grounding quorum supplies $f + 1$ correct echoes for the payload, while the definition of the world excludes every correct grade-1 ready. In W3, a quorum of correct NO-READY responses justifies and accepts skip. Because all locks are now inactive, these are exactly the response-compatibility checks of Section 6. Moreover, W1 and W2 contain a correct proposal-ready sender, so Lemma D.5(i) excludes every correct skip-voted stance. In W3 every correct initial ready-stage response is NO-READY, and the skip entry does not inspect or change the resolution stance. Thus the stance addition preserves the acceptability of $Y^*$ in all three worlds.

For the non-skip cases, the grounding echo quorum makes the required data permanently retrievable by Lemma B.2(i): $C_u$ and $T_u$ in the full case, and $C_u$ in the core case. Here the large case uses $G_B$ as that grounding set. The first carrying attempt in a post-cutoff view that activates everywhere makes every correct party request missing lane prefixes. Correct holders eventually answer, so all sufficiently late attempts pass the data checks. In a full case the fetched paired-tip lane prefixes also satisfy the tip-anchoring walk: at least $f+1$ correct grade-1 echoers verified strict containment when echoing view $u$, and containment is a property of the payload alone, transferred to any fetched copy by collision resistance. With the actual $b \leq f$ Byzantine parties, the relevant grounding set contains at least $f + 1$ correct target-view echoers. In a full case they sent grade-1 echoes for $P_u$; in a core case they sent proposal echoes for $P_u$, of either grade. Once the target ECHO is emitted, the effective ready value is closed, and the eventual data conditions above hold, these parties accept a carrying attempt and set the corresponding full or core origin bit. They therefore supply at least $f + 1$ correct origin-bit echoes. The skip case needs neither data nor bits.

*A correct proposer eventually carries $Y^*$, and it is anchored.* If an anchor resolving $u$ already exists or appears later, O1 makes every correct party observe it, $A_u$ exists, and the sealing rule finishes the induction step. It remains to consider an execution in which no such anchor ever appears. By the induction hypothesis, $u$ eventually becomes at $q$ the oldest view neither sealed nor resolved by an observed anchor. By Lemma B.4(a)–(b), R1, and round-robin assignment, $q$ sends proposals in infinitely many views beyond the cutoff $V$. Non-Zeno execution makes those send times unbounded. Once $u$ is its oldest pending target, the persistent alternation gives $q$ infinitely many recovery turns, and hence infinitely many carrying attempts for $u$. Its justified candidate set is monotone and has at most $2\lfloor n/(f+1) \rfloor + 1$ elements, so it stabilizes; $Y^*$ is eventually justified at $q$ and hence in the stable set, and $q$'s round-robin over the justified candidates carries every element of the stable set — in particular $Y^*$ — infinitely often. Consider such an attempt at a view $w \geq V$

after all the eventual conditions above hold. Every correct party accepts the resolution inside its echo window, and the genuine origin-bit echoes meet the ready-guard premise of Theorem B.8. That theorem makes the view-$w$ proposal complete at every correct party; O3 anchors it and O1 makes every correct party observe it, so $A_u$ exists. The sealing rule then seals $u$ at every correct party that lacks a terminal seal — via $A_u$, which may be an earlier anchored entry for $u$ rather than $Y^*$; any such entry is safe, and for a completed view it is commit-full or commit-core by Lemma D.6 — and Theorem D.8 makes all seals for $u$ equal.

Finally, if $u$ is completed, it has a proposal-ready quorum. A local Direct AGB result is non-skip, a fast result is full, a skip-seal is excluded by Lemma D.5(i), and a skip entry is never echo-accepted by Lemma D.6(iii). Hence every seal of a completed view is commit-full or commit-core. □

Corollary D.10 (Crash-only silent-view bound). *Let $s \geq$ GST be the first correct entry to view $u$, and suppose every correct party enters $u$ by $s + 2\delta$. Suppose* Proposer$(u)$ *and every other faulty party have crashed before sending any view-$u$ or later protocol message, so no view-$u$ proposal or proposal echo exists. Then every correct party submits* skip-seal$(u) \longrightarrow$ skip *by*

$$s + 4\Delta + 3\delta.$$

*If all correct entries to $u$ are aligned at time $e$, the bound is $e + 4\Delta + \delta$.*

Proof. Every correct party emits echo-skip by its absolute echo deadline, at latest $s + 2\delta + 3\Delta$. These correct responses — at least $n - f$ of them — arrive everywhere by $s + 3\delta + 3\Delta$. Every correct party emits no-ready by at latest $s + 2\delta + 4\Delta$. Since $\delta \leq \Delta$, the latter time is no earlier than the former, so by then every correct party has both its own no-ready response and an echo-skip quorum.

No correct proposer can justify a non-skip recovery for $u$, because no target-view proposal echo exists, and no faulty carrier is sent. Thus every correct resolution stance remains free until the vote. A compatible skip anchor learned early does not suppress the vote, so every correct party broadcasts skip-vote$(u)$ by $s + 2\delta + 4\Delta$, and those votes arrive everywhere one actual delay later. The aligned calculation removes the $2\delta$ entry spread. Each later crashed view still has its own entry and deadlines; this corollary does not collapse a burst into one timeout. □

## E END-TO-END CORRECTNESS PROOF

Theorem E.1 (End-to-end correctness). *Every correct party eventually seals every data-plane view with exactly one common terminal outcome. At every time, the output sequences of any two correct parties are prefix-comparable; they have the same limit sequence, and every finite prefix is eventually output by every correct party. Every output block occurs at most once, satisfies BlockOK, was published with its valid lane prefix by its encoded author to at least one correct party, has a permanent correct holder, and is obtained by each correct party before that party locally outputs it. Moreover, every valid data block published by a correct author is eventually output.*

Proof. Theorems D.8 and D.9 give exactly one eventual common seal per view. For the ordered-output claims, Direct AGB supplies the local properties, Lemma D.7 supplies RS1–RS3, Theorem D.9 supplies per-target totality, and fair repair is Lemma B.2(ii). The only extension beyond Corollary C.2 is the pair of caller-side local routes: Lemma D.4 proves that a fast seal names the same witnessed full outcome as every Direct AGB or anchor submission, while Lemma D.5 proves compatibility of a skip seal. Hence the corollary's cursor argument applies unchanged. It remains to establish input fairness.

Let a correct author publish a valid data block $b$. Every correct party eventually receives the full lane prefix through $b$ in the author's PUBLISH messages and acknowledges a lane frontier covering $b$, so every sufficiently late correct proposer counts $n-f$ acknowledgments for a covering lane frontier and includes the newest such frontier in its core; mixed-tip quarantine suppresses only $T$ entries and therefore cannot block this inclusion. By Lemma B.4(a), R1, round-robin assignment, and non-Zeno execution, every correct party has unbounded send times as proposer. The persistent recovery/data alternation gives it infinitely many data-only proposer turns whenever targets remain unsealed; if none is unsealed, its proposal is data-only as well. Choose such a turn after the acknowledgments are common and after the pacemaker cutoff. Its metadata field is empty, so Theorem B.8 makes the proposal complete everywhere. Its core names a lane prefix covering $b$, proving input fairness. All remaining premises of the cursor argument in Corollary C.2 now hold, and it gives the remaining claims. □

COROLLARY E.2 (STATE-MACHINE REPLICATION). *Fix an initial state and a deterministic transition function defined on BLOCKOK blocks. Applying output blocks in order gives correct parties identical states after every common output prefix, and every finite state prefix is eventually reached by every correct party.*

## F IMPLEMENTATION AND EVALUATION DETAILS

### F.1 Implementation and measured encoding

Our Rust implementation uses one deterministic protocol task and separate networking and storage. It performs no public-key or signature operation. Validators retain 32-byte identifiers internally but use canonical one-byte wire indices; benchmark TCP does not bind those identities cryptographically, so the measurements omit the model's channel-authentication cost.

The same binary runs both Autobahn modes and two Simple-IT variants on a shared TCP, batching, worker, storage, client, and metric stack [16, 20, 42]. Simple-IT follows its paper, adds a safe notarization bound, and fetches missing proposals; Bluestreak and Sailfish++ use Starfish's stack [28, 29]. In-tree protocols set $\Delta = 200$ ms, a 100 ms header delay, 64 KiB/5 ms network flushing, and 20 ms worker batching. Simple-IT retains its $8\Delta$ Opt-RBC and $5\Delta$ Bracha timers; Vantage and Simple-IT use BLAKE3 and round-robin leaders.

### F.2 Measurement methodology

*Network and workload.* Validators use private addresses in one availability zone. Netem assigns validator $i$ row $i$ mod 10 of Table 2, placing half the RTT on each direction without jitter [28]. Variants at one committee size reuse instances and rules; sizes use new instances, so cross-size trends include AWS placement variation. Each validator offers $R/n$ open-loop load in 50 ms bursts. Transactions are timestamped, random, effectively incompressible 512-byte values [28, 29].

Workers store and disseminate batches before passing digests to the primary. Vantage puts them in unsigned lane blocks, broadcasts headers every 100 ms, and fetches missing batches; the interval approximates the uncertified-DAG round duration.

*Metrics.* After all validators advance, runs warm up for 30 seconds and measure for 120. Latency spans submission through local sequencing and byte retrieval [28, 42]; throughput is sequenced transactions per window. We report the median validator's p50/p99 latency, total primary-plus-worker CPU, and outbound framed bytes per sequenced byte. Each point is one run, so results are descriptive without confidence intervals.

Table 2. Injected round-trip latency matrix (ms). Row and column order is the committee-index assignment order.

| | USE1 | CAC1 | EUS2 | SAE1 | APSE2 | USW1 | EUW1 | EUN1 | APS1 | APNE1 |
|---|---|---|---|---|---|---|---|---|---|---|
| USE1 | 1 | 14 | 104 | 112 | 198 | 65 | 68 | 110 | 201 | 146 |
| CAC1 | 14 | 1 | 106 | 122 | 196 | 78 | 67 | 103 | 189 | 142 |
| EUS2 | 104 | 106 | 1 | 215 | 281 | 163 | 29 | 50 | 143 | 238 |
| SAE1 | 112 | 122 | 215 | 1 | 309 | 175 | 176 | 220 | 299 | 254 |
| APSE2 | 198 | 196 | 281 | 309 | 1 | 137 | 254 | 268 | 150 | 101 |
| USW1 | 65 | 78 | 163 | 175 | 137 | 1 | 127 | 172 | 226 | 108 |
| EUW1 | 68 | 67 | 29 | 176 | 254 | 127 | 1 | 38 | 125 | 199 |
| EUN1 | 110 | 103 | 50 | 220 | 268 | 172 | 38 | 1 | 148 | 245 |
| APS1 | 201 | 189 | 143 | 299 | 150 | 226 | 125 | 148 | 1 | 140 |
| APNE1 | 146 | 142 | 238 | 254 | 101 | 108 | 199 | 245 | 140 | 1 |

### F.3 Representation optimizations

The proved protocol broadcasts each lane-frontier acknowledgment separately. The implementation instead places them in the authenticated envelope of the sender's next single-proposal ECHO, outside its immutable identity. Since $\mathcal{H}(B_v)$ binds an ordered reference vector, a bitmap marks acknowledged positions and a shorter prefix uses a sparse hash-anchored exception. Every $(a, k, h)$ remains attributed to and counted first-hand from the envelope sender; the envelope is neither a certificate nor a forwarded-MAC vector, and batch-recovery ECHOs remain by value.

The proposal travels by value; later single-proposal ECHOs and READYs name $\mathcal{H}(B_v)$, a grade, and any origin bit. Before a digest quorum acts, the receiver must hold the canonical body and execute the unchanged by-value rule. Statement authors retain and serve it, but served bytes create no provenance or vote. Transport batches to 64 KiB or 5 ms; sequence state checkpoints every 20 views, reconnect uses a 512-view outbox, and GC follows 200 views. These bounded mechanisms remain outside the unbounded-state proof.

With $\lambda$ bounding a digest, identifier, and reference, the proposal and digest statements cost $O(n^2\lambda)$ bits over all deliveries; the $O(n)$-bit ECHO bitmaps add $O(n^3)$. Thus the aligned single-proposal encoding uses $O(n^2\lambda + n^3)$ control bits and $O(n^2)$ logical messages. Sparse exceptions and by-value recovery preserve the proved $O(n^3\lambda)$ worst case.

### F.4 Experiment-specific records

*Committee scaling.* The seven configurations at $n \in \{10, 20, 50, 100\}$ use one on-demand `c5d.xlarge` validator (4 vCPUs, 8 GiB RAM, 100 GB local NVMe) in `eu-west-1a`, the matrix above, 100 aggregate tx/s, five-second Prometheus scrapes, and the common warmup/window. All 28 first attempts completed, sustained at least 99.2% at every validator, and reported no netem drops or panics.

*View anatomy and network-delay control.* The $n = 20$ mechanism study runs all validators in one release binary on an Apple M4 Pro, injects the matrix delays in process, and uses the optimized encoding at 100 tx/s. Each of three 60-second runs sequences all 6,000 submitted transactions; a reported percentile is first taken across validators and then the median across runs.

Fast-seal waits for all 20 matching ECHOs, while Direct AGB may finish from $n-f = 14$ READYs. Under heterogeneous delays, the Direct AGB result sometimes precedes the slowest ECHO, so fast-seal supplies 70.0–70.5% of local seals; at a uniform 143 ms RTT every observed seal uses fast-seal. All seals are full. Uniformization changes coverage from 175 to 196 ms, proposal-to-seal from 182 to 81 ms, and reduces end-to-end sequencing latency by 90 ms.

AGB is the largest typed category; payload plus data-block dissemination is 48.8%, and healthy-run repair, replay, and pacemaker traffic is 0.2%. The representative run carries 24,876 logical messages/s and 3.457 MB/s typed committee-wide; physical outbound is 3.718 MB/s, or 0.186 MB/s per validator. Optional tracing retains at most 4,096 block timestamps

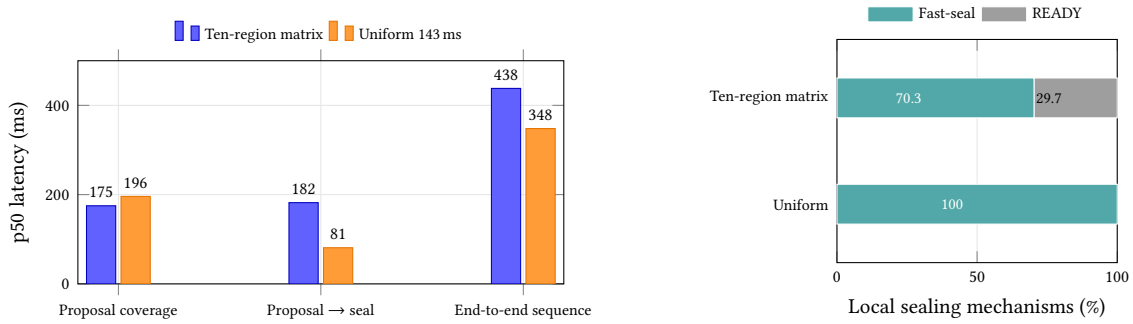


Fig. 8. Matched network-delay control at $n = 20$. Left: p50 stage and end-to-end latency. Right: local sealing mechanisms summed over validators. The uniform value is the 143 ms median off-diagonal RTT after assigning the ten-region matrix to 20 validators.

per validator. A matched 30-second A/B orders all 3,000 transactions in each build, at 438 ms p50 without tracing and 437 ms with it.

*Throughput sweep.* The $n = 100$ sweep offers 100, 10,000, 150,000, 200,000, 225,000, 250,000, and 275,000 tx/s. Most points use `c5d.2xlarge` validators (8 vCPUs, 16 GiB RAM, 200 GB NVMe); compacted Starfish points from 200k use a separate `m5d.2xlarge` fleet (8 vCPUs, 32 GiB RAM, 300 GB NVMe). Both are in `eu-west-1a`, but placement and hardware are not controlled across the two campaigns.

The original campaign strictly validates all validators at 100 and 10k and treats higher loads as exploratory; all Starfish reruns are strict. Acceptance means at least 95% median throughput, with the first lower point retained as overload. Accepted points end with 100 responsive validators and no reported netem drops. Vantage and Bluestreak pass through 250k and overload at 275k; Sailfish++ overloads at 200k. Autobahn passes 100 and 10k but twice fails the 150k progress barrier. These are deployment endpoints, not capacity bounds.

*Reproduction.* In the wan-bench repository, run `wanbench campaign -config configs/NAME.yaml -execute`, replacing `NAME` by `paper-committee-scaling`, `paper-n100-throughput`, or `paper-n100-starfish-m5d-gc`. For the local controls, build the Vantage repository with `cargo build -release -p node -features pipeline-tracing` and run `./target/release/node local-benchmark` with `-nodes N -workers 1 -rate 100 -tx-size 512 -protocol vantage -duration 60 -data-dir PATH`; use `-no-compact-ids` for the ID control or `-mimic-latency-ms 143` for the uniform-RTT control.

## G EXTENDED RELATED WORK

Section 3 gives a normalized quantitative comparison with five baselines. Here we place those protocols and other related work in the broader design space, organized by where a protocol spends strong agreement and whether its evidence is transferable.

*Signature-free BFT.* Agreement without digital signatures is classical: Castro's MAC-vector Algorithm BFT reaches a $3\delta$ good case, with cubic authenticator communication from recipient-specific MAC vectors [12, 13]. Those vectors provide forwarded per-recipient evidence and hence lie outside our channel-only model. Mostéfaoui, Moumen, and Raynal gave signature-free asynchronous binary consensus with $O(n^2)$ messages [27]. Information-Theoretic HotStuff brought the rotating leader-and-view discipline to authenticated channels without signatures or a PKI [3]; TetraBFT

reduces the good case to $5\delta$ [41]; Forget-IT attains the good-case optimum of $3\delta$ [1]. Simple-IT makes the line practical with reliable notification and an optimistic broadcast path under a stronger honesty threshold [42]; speculative pipelining can narrow leader-proposal spacing at the cost of requiring runs of correct leaders. Its shared-mempool variant is compared directly in Section 3. We also reuse non-speculative Simple-IT as the background order for sparse resolution anchors. This dependency can gate output after an open tip but is outside direct completion and homogeneous sealing. The validated-Bracha wrapper of Lemma D.1 supplies the control-record adapter; its validity predicate is separate from Vantage's data-block validity and provenance gates.

*Certified-data designs.* Narwhal separates availability-certified dissemination from ordering [16]; DAG-Rider [21], Bullshark [36], Shoal and Shoal++ [5, 35], and Sailfish [33] order DAGs in which every vertex is reliably broadcast or availability-certified, so ordering adds little communication but every data unit completes a strong-broadcast or certification round before it can be referenced; most practical instantiations also use signatures. DAG-Rider's RBC-DAG is signature-free given an ideal perfect coin, although its standard practical coin realization uses threshold cryptography for liveness. Sailfish++ makes the Sailfish line signature-free by running optimistic signature-free RBC for every vertex [34]; Section 3 details the resulting nonleader classes and the contrast with Vantage's graded tip. On the signed side, Autobahn combines PoA-certified per-author lanes with a PBFT-style core and optionally admits an uncertified current tip [20]; Section 3 compares that optimization and its seamlessness tradeoff directly with Vantage's graded tip. Dumbo-NG separates continuously growing signed data streams from asynchronous agreement [19], and Raptr integrates prefix certification of uncertified batches into leader consensus [37]. Simplex [14] and DispersedSimplex [32] are the signed leader-dissemination baselines against which the signature-free leader line measures itself.

Concurrent work, Multimmit, independently addresses missing data in a signed multi-chain protocol [24]. Its votes report a supported prefix per producer chain and may extend the leader's proposed tips, so one missing producer need not block the other chains, a localisation they prove rather than approximate. Its per-chain report is a finer split than Vantage's single grade bit. Its extension votes omit a step that Vantage retains: a Vantage tip entry must reach the proposer before the proposal. In its good case a block disseminated at $t$ is ordered by $t + 3\delta$ in expectation and $t + 2\delta$ at best, measured from dissemination as ours is. The cost is cryptographic and in resilience: $n \geq 5f + 1$, a PKI, and ordinary plus aggregate signatures, together with two threshold schemes that their optimisation section shows can be replaced by the aggregate scheme alone. Aggregation is what keeps their certificates succinct, and the standardized post-quantum signatures have no comparable aggregate, so a post-quantum instantiation would keep the structure and lose the succinctness. Vantage studies signature-free one-hop admission with $n \geq 3f + 1$. Because it is concurrent work with no reported evaluation and a different fault threshold, it is not included in Table 1.

Other concurrent work takes proposal pipelining beyond the one-round cadence of DAGs: Cadence runs independent signed slots at an arbitrary configured interval $\tau$, with concurrent proposers disseminating coded encrypted proposals against $\tau$-spaced deadlines, and signed include/exclude evidence plus multivalue Byzantine agreement (MVBA) resolving a failed fast path [7]. Cadence can configure $\tau$ independently of $\delta$, a stronger pacing result than our all-correct at-most-$\delta$ fresh-opportunity gap. Its favorable per-slot finality is $\Delta + 2\delta$ from formal slot start $D - \Delta$, although in-order append may wait for earlier slots. Its endpoint and mechanisms — aggregate signatures, per-slot threshold encryption, and proposer-local input rather than a distinct data-block author — place it outside the comparison experiment of Table 1. Vantage instead studies how far pipelined multi-author data can go with hashes and non-transferable first-hand counts alone.

Banyan paired the $n = 3f+2p-1$ two-delay fast path with rotating leaders [38]; concurrent FinWhale brings it to Mysticeti, using evidence blocks to reconcile fast and slow decisions across local DAG views [23]. At $p = 1$, it keeps $n = 3f+1$ and survives one non-voter, unlike Vantage's all-$n$ fast-seal shortcut, but its blocks remain signed. The two systems share the $2\delta$ carrier-relative endpoint; their aligned totals and FinWhale's round bound are in Table 1.

*Recent hybrid and sparse DAGs.* Recent work reduces certified-DAG metadata without removing the data-side dissemination gate. Angelfish replaces some nonleader RBC vertices with one-delay best-effort votes, but those votes carry no transactions; every transaction-carrying vertex remains reliably broadcast, and the protocol uses BLS certificates [40]. Clownfish sparsifies certified-DAG edges in RBC- and signed consistent-broadcast variants and explicitly leaves a signature-free adaptation to future work [39]. DAGs for the Masses uses verifiable sampling to reduce Bullshark's parent set, while every vertex remains reliably broadcast and references carry signed availability evidence [4]. Morpheus is a closer signed endpoint: its Autobahn-style mode lets a leader hash-reference a freshly received author block, but voters proceed only after obtaining that block's signed 0-QC [25]. Hashgraph's constant-degree gossip DAG likewise uses signed two-parent events [8]; the broader certified and uncertified DAG design space is surveyed in [31].

*Uncertified data with signatures.* Cordial Miners introduced leader-based ordering of a best-effort block DAG [22]; Mysticeti improves latency by pipelining leaders, committing a signed uncertified DAG with a three-round leader path [6]. Its idealized aligned cross-author block path is $4\delta$; the familiar $4.5\delta$ starts at transaction arrival and adds the average wait for the author's next block. The Starfish paper presents the pacemaker under which this line is provably live, and its Mysticeti-L variant improves consensus bandwidth with multisignatures. Its explicit all-correct post-GST construction for Mysticeti approaches $5\frac{1}{3}\delta$ average block latency; adding the $\delta/3$ admission induced by that synchronized arrival schedule gives $5\frac{2}{3}\delta$ average transaction latency, a different experiment. Starfish itself further decouples availability through coded dissemination [28]. Bluestreak reduces nonleader metadata without multisignatures [29], and concurrent FinWhale adds a two-round fast path to this family [23]. In these protocols, signatures keep relayed blocks and votes attributable. Vantage instead distinguishes direct author publication from repair and counts author-receipt acknowledgments first-hand.

Starfish-S is the closest mechanism lineage: its optimistic, standard, and pending grades use later committed anchors [30]. Vantage replaces its signed DAG evidence with first-hand response histories and locally validated entries.

*Graded and abortable broadcast.* Feldman–Micali gradecast grades confidence in one value for Byzantine agreement [18]; AGB instead grades optional-suffix inclusion after fixing a common core and omits faulty-proposer totality. BBCA gives an indivisible value Complete–Adopt, and BBCA-Chain recovers a backbone with transferable adopt/no-adopt evidence [26]. AGB's READY/NO-READY and MIX refinement imply a graded counterpart, but Vantage resolves open and general silent views through an ordered resolver, using its grounded shortcut only for clean crash silence. Optimistic RBC retains reliable-broadcast totality [9, 34]. The BBCA-Chain comparison in Section 3 implies no bounded Vantage fallback. GradedDAG grades a different object in an asynchronous signed DAG [15].

*Foundations.* AGB uses Bracha's echo/ready pattern [9]; Abraham, Ren, and Xiang classify unauthenticated broadcast latency [2]. Its pacemaker combines the bounded-space synchronizer of Bravo, Chockler, and Gotsman with Starfish's two-response trigger [10, 30]; verifiable dispersal could compress data and repair independently [11]. Simple-IT grounds its first notification ACCEPT in a vote quorum, yielding $f + 1$ correct timeout raisers excluded from commit voting [42]. Vantage instead excludes overlap between grounded skip votes and ECHOs for non-skip resolution-bearing proposals, without $f + 1$ skip-vote amplification.